

Chapter 43

SPI Controller (SPI)

43.1 Overview

The Serial Peripheral Interface (SPI) is a synchronous serial interface useful for communication with external peripherals. The ESP32-P4 chip integrates four SPI controllers:

- MSPI controller, including two sub-controllers
 - FLASH MSPI controller
 - * FLASH MSPI SPIO
 - * FLASH MSPI SPI1
 - PSRAM MSPI controller
 - * PSRAM MSPI SPIO
 - * PSRAM MSPI SPI1
- General Purpose SPI2 (GP-SPI2)
- General Purpose SPI3 (GP-SPI3)
- Low-Power SPI (LP-SPI)

MSPI controller is primarily reserved for internal use to communicate with external flash and PSRAM memory. This chapter mainly focuses on the GP-SPI and LP-SPI. **In this chapter unless otherwise stated, GP-SPI refers to both GP-SPI2 and GP-SPI3.**

43.2 Glossary

To better illustrate the functions of GP-SPI and LP-SPI, the following terms are used in this chapter.

Master Mode	GP-SPI or LP-SPI acts as an SPI master and initiates SPI transactions.
Slave Mode	GP-SPI or LP-SPI acts as an SPI slave and exchanges data with its master when its CS is asserted.
MISO	Master in, slave out, data transmission from a slave to a master.
MOSI	Master out, slave in, data transmission from a master to a slave.
Transaction	One instance of a master asserting a CS line, transferring data to and from a slave, and de-asserting the CS line. Transactions are atomic, which means they can never be interrupted by another transaction.

SPI Transfer	The whole process of an SPI master exchanging data with a slave. One SPI transfer consists of one or more SPI transactions.
Single Transfer	An SPI transfer that consists of only one transaction.
CPU-Controlled Transfer	A data transfer that happens between CPU configured buffer SPI_WO_REG~SPI_W15_REG (LP_SPI_WO_REG~LP_SPI_W15_REG) and SPI peripherals.
DMA-Controlled Transfer	A data transfer that happens between DMA and SPI peripherals, controlled by the DMA engine.
Configurable Segmented Transfer	A data transfer controlled by DMA when GP-SPI2 works as a master. Such transfer consists of multiple transactions (segments), and each of transaction can be configured independently.
Slave Segmented Transfer	A data transfer controlled by DMA when GP-SPI works as a slave. Such transfer consists of multiple transactions (segments).
Full-duplex	The sending line and receiving line between the master and the slave are independent. Sending data and receiving data happen at the same time.
Half-duplex	Only one side, the master or the slave, sends data first, and the other side receives data. Sending data and receiving data can not happen at the same time.
4-line full-duplex	4-line here means: clock line, CS line, and two data lines. The two data lines can be used to send or receive data simultaneously.
4-line half-duplex	4-line here means: clock line, CS line, and two data lines. The two data lines can not be used simultaneously.
3-line half-duplex	3-line here means: clock line, CS line, and one data line. The data line is used to transmit or receive data.
1-bit SPI	In one clock cycle, one bit can be transferred.
(2-bit) Dual SPI	In one clock cycle, two bits can be transferred.
Dual Output Read	A data mode of Dual SPI. In one clock cycle, one bit of a command, or one bit of an address, or two bits of data can be transferred.
Dual I/O Read	Another data mode of Dual SPI. In one clock cycle, one bit of a command, or two bits of an address, or two bits of data can be transferred.
(4-bit) Quad SPI	In one clock cycle, four bits can be transferred.
Quad Output Read	A data mode of Quad SPI. In one clock cycle, one bit of a command, or one bit of an address, or four bits of data can be transferred.
Quad I/O Read	Another data mode of Quad SPI. In one clock cycle, one bit of a command, or four bits of an address, or four bits of data can be transferred.
QPI	In one clock cycle, four bits of a command, or four bits of an address, or four bits of data can be transferred.
(8-bit) Octal SPI	In one clock cycle, eight bits can be transferred.
Octal Output Read	A data mode of Octal SPI. In one clock cycle, one bit of a command, or one bit of an address, or eight bits of data can be transferred.

Octal I/O Read	Another data mode of Octal SPI. In one clock cycle, one bit of a command, or eight bits of an address, or eight bits of data can be transferred.
OPI	In one clock cycle, eight bits of a command, or eight bits of an address, or eight bits of data can be transferred.
SPI2	The prefix of the signals for GP-SPI2. SPI2 bus signals are routed to GPIO pins via HP GPIO matrix or HP IO MUX.
SPI3	The prefix of the signals for GP-SPI3. SPI3 bus signals are routed to GPIO pins via HP GPIO matrix only.
LP_SPI	The prefix of the signals for LP-SPI. LP_SPI bus signals are routed to GPIO pins via LP GPIO matrix only.

43.3 Features

GP-SPI has the following features:

- Works as master or as slave
- Half- and full-duplex communications
- CPU- and DMA-controlled transfers
- Various data modes
 - **GP-SPI2**
 - * 1-bit SPI mode
 - * 2-bit Dual SPI mode
 - * 4-bit Quad SPI mode
 - * QPI mode
 - * 8-bit Octal SPI mode (available only when GP-SPI2 works as a master)
 - * OPI mode (available only when GP-SPI2 works as a master)
 - **GP-SPI3**
 - * 1-bit SPI mode
 - * 2-bit Dual SPI mode
 - * 4-bit Quad SPI mode
 - * QPI mode
- Configurable module clock frequency
 - Master: up to 80 MHz
 - Slave: up to 60 MHz
- Configurable data length
 - CPU-controlled transfer as master or as slave: 1~64 bytes
 - DMA-controlled single transfer as master: 1~32 KB

- DMA-controlled configurable segmented transfer as master: data length is unlimited
 - DMA-controlled single transfer or segmented transfer as slave: data length is unlimited
- Configurable bit read/write order
- Independent interrupts for CPU-controlled transfer and DMA-controlled transfer
- Configurable clock polarity and phase
- Four SPI clock modes: mode 0~mode 3
- Multiple CS lines as master
 - **GP-SPI2**: CS0~CS5
 - **GP-SPI3**: CS0~CS2
- Able to communicate with SPI devices, such as a sensor, a screen controller, as well as a flash or RAM chip

LP-SPI is a simplified version of GP-SPI and has a subset of GP-SPI's features:

- Works as a master or as a slave
- Half- and full-duplex communications
- CPU-controlled transfer
- 1-bit SPI data mode
- Configurable module clock frequency:
 - Master: up to 40 MHz
 - Slave: up to 40 MHz
- Configurable data length:
 - CPU-controlled transfer as master or as slave: 1~64 bytes
- Configurable bit read/write order
- Interrupts for CPU-controlled transfer
- Configurable clock polarity and phase
- Four SPI clock modes: mode 0~mode 3
- One CS line as master: CS0
- Wake-up feature as slave (the only new feature compared with GP-SPI)

43.4 Architectural Overview

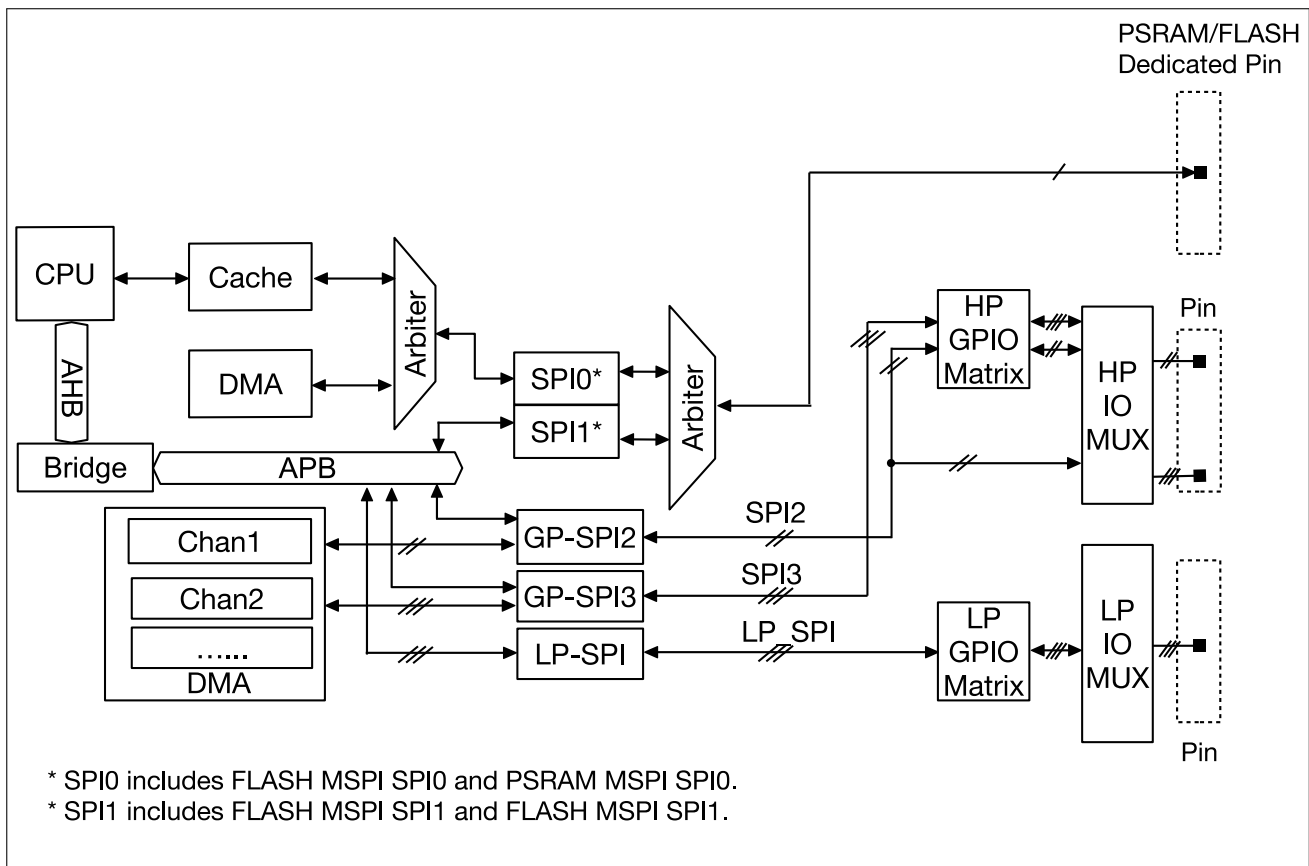


Figure 43.4-1. SPI Module Overview

Figure 43.4-1 shows an overview of this SPI module. GP-SPI2, GP-SPI3, and LP-SPI exchange data with SPI devices by the following ways:

- CPU-controlled transfer:
 - CPU ↔ LP-SPI ↔ SPI devices
 - CPU ↔ GP-SPI2 (GP-SPI3) ↔ SPI devices
- DMA-controlled transfer: DMA ↔ GP-SPI2 (GP-SPI3) ↔ SPI devices

In this chip, “SPI2”, “SPI3”, and “LP_SPI” are used as prefixes of GP-SPI2, GP-SPI3, and LP-SPI input and output signal buses to illustrate the SPI bus functions.

SPI2 bus signals are routed to GPIO pins via either the HP GPIO matrix or the HP IO MUX, while SPI3 bus signals are routed to GPIO pins via the HP GPIO matrix only. LP_SPI bus signals are routed via the LP GPIO matrix only. For more information, see Chapter 9 *GPIO Matrix and IO MUX*.

The functionalities of GP-SPI3 are nearly the same as those of GP-SPI2. The functionalities of LP-SPI are a subset of the GP-SPI functionalities. GP-SPI2’s functionalities are described in Section 43.5. The differences among GP-SPI2, GP-SPI3, and LP-SPI are described in Section 43.5.1 and Section 43.10.

43.5 Functional Description

43.5.1 Data Modes

GP-SPI and LP-SPI can be configured as either a master or a slave to communicate with other SPI devices in the following data modes, see Table 43.5-1. For more information about the data modes used when GP-SPI or LP-SPI works as a master, see Section 43.5.9, and Section 43.5.10 for GP-SPI working as a slave.

Table 43.5-1. Data Modes Supported by GP-SPI and LP-SPI

Data Mode		CMD State	Address State	Data State	GP-SPI2	GP-SPI3	LP-SPI
1-bit SPI		1-bit	1-bit	1-bit	Y	Y	Y
Dual SPI	Dual Output Read	1-bit	1-bit	2-bit	Y	Y	—
	Dual I/O Read	1-bit	2-bit	2-bit	Y	Y	—
Quad SPI	Quad Output Read	1-bit	1-bit	4-bit	Y	Y	—
	Quad I/O Read	1-bit	4-bit	4-bit	Y	Y	—
Octal SPI	Octal Output Read	1-bit	1-bit	8-bit	Y	—	—
	Octal I/O Read	1-bit	8-bit	8-bit	Y	—	—
QPI		4-bit	4-bit	4-bit	Y	Y	—
OPI		8-bit	8-bit	8-bit	Y	—	—

43.5.2 Introduction to Bus Signals

The functional description of SPI2/SPI3 and LP_SPI bus signals is shown in Table 43.5-2. Tables 43.5-3, 43.5-4, and 43.5-5 list the signals used in various SPI modes.

Table 43.5-2. Functional Description of SPI2/SPI3 and LP_SPI Bus Signals

SPI2 Bus Signal	SPI3 Bus Signal	LP-SPI Bus Signal	Function
SPI2CLK	SPI3_CLK	LP_SPI_CLK	Input and output clock as master/slave
SPI2CS0	SPI3_CS0	LP_SPI_CS	Input and output CS signal as master/slave
SPI2CS1~5	SPI3_CS1~2	—	Output CS signal as master
SPI2D	SPI3_D	LP_SPI_D	MOSI/SIO0 (serial data input and output, bit0)
SPI2Q	SPI3_Q	LP_SPI_Q	MISO/SIO1 (serial data input and output, bit1)
SPI2WP	SPI3_WP	—	SIO2 (serial data input and output, bit2)
SPI2HD	SPI3_HD	—	SIO3 (serial data input and output, bit3)
SPI2D4~7	—	—	SIO4~7 (serial data input and output, bit4~7)
SPI2DQS	—	—	Output data mask signal as master

Table 43.5-3. SPI2 Bus Signals Used in Various SPI Modes

SPI2 Bus Signal	Master								Slave					
	FD ¹	1-bit SPI		Dual SPI	Quad SPI	QPI	Octal SPI	OPI	FD	1-bit SPI		Dual SPI	Quad SPI	QPI
		3-line HD ²	4-line HD							3-line HD	4-line HD			
SPI2CLK	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SPI2CS0	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SPI2CS1	Y	Y	Y	Y	Y	Y	Y	Y						
SPI2CS2	Y	Y	Y	Y	Y	Y	Y	Y						
SPI2CS3	Y	Y	Y	Y	Y	Y	Y	Y						
SPI2CS4	Y	Y	Y	Y	Y	Y	Y	Y						
SPI2CS5	Y	Y	Y	Y	Y	Y	Y	Y						
SPI2D	Y	Y	(Y) ³	Y ⁴	Y ⁵	Y	Y	Y	Y	Y	(Y) ⁶	Y ⁷	Y ⁸	Y
SPI2Q	Y		(Y) ³	Y ⁴	Y ⁵	Y	Y	Y	Y		(Y) ⁶	Y ⁷	Y ⁸	Y
SPI2WP					Y ⁵	Y	Y	Y					Y ⁸	Y
SPI2HD					Y ⁵	Y	Y	Y					Y ⁸	Y
SPI2D4~7							Y	Y						
SPI2DQS							Y	Y						

¹ FD: full-duplex² HD: half-duplex³ Only one of the two signals is used at a time.⁴ The two signals are used in parallel.⁵ The four signals are used in parallel.⁶ Only one of the two signals is used at a time.⁷ The two signals are used in parallel.⁸ The four signals are used in parallel.

Table 43.5-4. SPI3 Bus Signals Used in Various SPI Modes

SPI3 Bus Signal	Master						Slave					
	FD ¹	1-bit SPI		Dual SPI	Quad SPI	QPI	FD	1-bit SPI		Dual SPI	Quad SPI	QPI
		3-line HD ²	4-line HD					3-line HD	4-line HD			
SPI3_CLK	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SPI3_CS0	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SPI3_CS1	Y	Y	Y	Y	Y	Y						
SPI3_CS2	Y	Y	Y	Y	Y	Y						
SPI3_D	Y	Y	(Y) ³	Y ⁴	Y ⁵	Y	Y	Y	(Y) ⁶	Y ⁷	Y ⁸	Y
SPI3_Q	Y		(Y) ³	Y ⁴	Y ⁵	Y	Y		(Y) ⁶	Y ⁷	Y ⁸	Y
SPI3_WP					Y ⁵	Y					Y ⁸	Y
SPI3_HD					Y ⁵	Y					Y ⁸	Y

¹ FD: full-duplex² HD: half-duplex³ Only one of the two signals is used at a time.⁴ The two signals are used in parallel.⁵ The four signals are used in parallel.⁶ Only one of the two signals is used at a time.⁷ The two signals are used in parallel.⁸ The four signals are used in parallel.

Table 43.5-5. LP-SPI Bus Signals Used in Various SPI Modes

LP_SPI Bus Signal	Master (1-bit SPI)			Slave (1-bit SPI)		
	FD ¹	3-line HD ²	4-line HD	FD	3-line HD	4-line HD
LP_SPI_CLK	Y	Y	Y	Y	Y	Y
LP_SPI_CS	Y	Y	Y	Y	Y	Y
LP_SPI_D	Y	Y	(Y) ³	Y	Y	(Y) ³
LP_SPI_Q	Y		(Y) ³	Y		(Y) ³

¹ FD: full-duplex² HD: half-duplex³ Only one of the two signals is used at a time.

43.5.3 Bit Read/Write Order Control

- GP-SPI
 - When operating as a master:
 - * The bit order of the command, address and data sent by the GP-SPI master is controlled by [SPI_WR_BIT_ORDER](#).
 - * The bit order of the data received by the master is controlled by [SPI_RD_BIT_ORDER](#).
 - When operating as a slave:
 - * The bit order of the data sent by the GP-SPI slave is controlled by [SPI_WR_BIT_ORDER](#).
 - * The bit order of the command, address and data received by the slave is controlled by [SPI_RD_BIT_ORDER](#).
- LP-SPI
 - When operating as a master:
 - * The bit order of the command, address and data sent by the LP-SPI master is controlled by [LP_SPI_WR_BIT_ORDER](#).
 - * The bit order of the data received by the master is controlled by [LP_SPI_RD_BIT_ORDER](#).
 - When operating as a slave:
 - * The bit order of the data sent by the LP-SPI slave is controlled by [LP_SPI_WR_BIT_ORDER](#).
 - * The bit order of the command, address and data received by the slave is controlled by [LP_SPI_RD_BIT_ORDER](#).

Tables [43.5-6](#) and [43.5-7](#) show the functions of [SPI_RD/WR_BIT_ORDER](#) and [LP_SPI_RD/WR_BIT_ORDER](#).

Table 43.5-6. Bit Order Control in GP-SPI (Take GP-SPI2 as an Example)

Bit Mode	SPI2 Bus Data	SPI_RD/WR_BIT_ORDER = 0 (MSB)	SPI_RD/WR_BIT_ORDER = 2 (MSB)	SPI_RD/WR_BIT_ORDER = 1 (LSB)	SPI_RD/WR_BIT_ORDER = 3 (LSB)
1-bit mode	SPI2D or SPI2Q	B7→B6→B5→B4→B3→B2→B1→B0	B7→B6→B5→B4→B3→B2→B1→B0	B0→B1→B2→B3→B4→B5→B6→B7	B0→B1→B2→B3→B4→B5→B6→B7
2-bit mode	SPI2Q	B7→B5→B3→B1	B6→B4→B2→B0	B1→B3→B5→B7	B0→B2→B4→B6
	SPI2D	B6→B4→B2→B0	B7→B5→B3→B1	B0→B2→B4→B6	B1→B3→B5→B7
4-bit mode	SPI2HD	B7→B3	B4→B0	B3→B7	B0→B4
	SPI2WP	B6→B2	B5→B1	B2→B6	B1→B5
	SPI2Q	B5→B1	B6→B2	B1→B5	B2→B6
	SPI2D	B4→B0	B7→B3	B0→B4	B3→B7
8-bit mode	SPI2D7	B7	B7	B0	B0
	SPI2D6	B6	B6	B1	B1
	SPI2D5	B5	B5	B2	B2
	SPI2D4	B4	B4	B3	B3
	SPI2HD	B3	B3	B4	B4
	SPI2WP	B2	B2	B5	B5
	SPI2Q	B1	B1	B6	B6
	SPI2D	B0	B0	B7	B7

Table 43.5-7. Bit Order Control in LP-SPI

Bit Mode	LP_SPI Bus Signal	LP_SPI_RD/WR_BIT_ORDER = 0 (MSB)	LP_SPI_RD/WR_BIT_ORDER = 1 (LSB)
1-bit mode	LP_SPI_D or LP_SPI_Q	B7→B6→B5→B4→B3→B2→B1→B0	B0→B1→B2→B3→B4→B5→B6→B7

43.5.4 Unaligned Byte Transfer

- When operating as a master, GP-SPI sends and receives data in bits. The bit length is equal to `SPI_MS_DATA_BITLEN` + 1, a multiple of 1/2/4/8 in 1/2/4/8-bit mode.
 - When sending data whose length is not an integer multiple of 8 bits, the software needs to fill the last part of the data less than 8 bits into a full 1-byte data.
 - When receiving data whose length is not an integer multiple of 8 bits, the part less than 1 byte is still received as 1 byte.
- When operating as slave, GP-SPI sends and receives data in bits. The bit length is a multiple of 1/2/4/8 in 1/2/4/8-bit mode.
 - When sending data whose length is not an integer multiple of 8 bits, the software needs to fill the last part of the data less than 8 bits into a full 1-byte data.
 - When receiving data whose length is not an integer multiple of 8 bits, the part less than 1 byte is still received as 1 byte. The total bit length received can be read from `SPI_SLV_DATA_BITLEN`. The valid bits in the last 1 byte is indicated by `SPI_SLV_LAST_BYTE_STRB`.

Note:

- No matter working as master or slave, GP-SPI sends and receives the data parts less than 1 byte following the same bit order as the other data bits as configured.
- LP-SPI does not support this unaligned byte transfer.

43.5.5 Transfer Types

The transfer types supported by GP-SPI working as a master or a slave are shown in Table 43.5-8.

Table 43.5-8. Supported Transfer Types as Master or Slave

Mode		CPU-Controlled Single Transfer 1	DMA-Controlled Single Transfer	DMA-Controlled Configurable Segmented Transfer 2	DMA-Controlled Slave Segmented Transfer
Master	Full-Duplex	Y	Y	Y	—
	Half-Duplex	Y	Y	Y	—
Slave	Full-Duplex	Y	Y	—	Y
	Half-Duplex	Y	Y	—	Y

¹ LP-SPI supports only the CPU-controlled single transfer.

² GP-SPI3 does not support DMA-controlled configurable segmented transfer.

The following sections provide detailed information about the transfer types listed in the table above.

43.5.6 CPU-Controlled Data Transfer

Note:

Both GP-SPI and LP-SPI provide 16 x 32-bit data buffers:

- GP-SPI: [SPI_W0_REG~SPI_W15_REG](#)
- LP-SPI: [LP_SPI_W0_REG~LP_SPI_W15_REG](#)

The following section illustrates the buffer's structure using GP-SPI data buffer as an example.

Figure 43.5-1 shows the buffer's structure. CPU-controlled transfer indicates the transfer, in which the data to send is from GP-SPI data buffer and the received data is stored to GP-SPI data buffer. In such transfer, every single transaction needs to be triggered by the CPU, after its related registers are configured. For such reason, the CPU-controlled transfer is always single transfer (consisting of only one transaction). CPU-controlled transfer supports full-duplex communication and half-duplex communication.

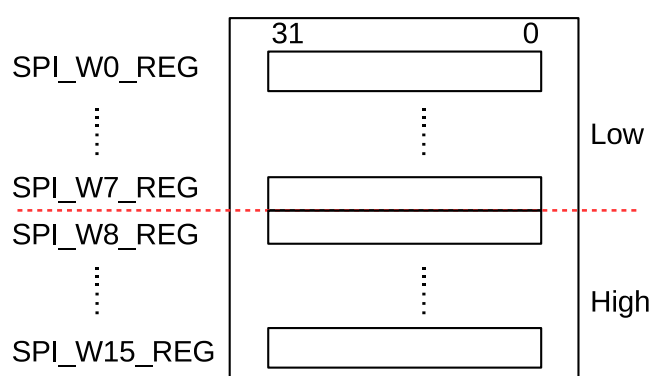


Figure 43.5-1. Data Buffer Used in CPU-Controlled Transfer

43.5.6.1 CPU-Controlled Master Transfer

In a CPU-controlled master full-duplex or half-duplex transfer, the RX or TX data is saved to or sent from [SPI_W0_REG~SPI_W15_REG](#). The bits [SPI_USR_MOSI_HIGHPART](#) and [SPI_USR_MISO_HIGHPART](#) control which buffers are used. See the description below.

- TX data

- When [SPI_USR_MOSI_HIGHPART](#) is cleared, i.e., high part mode is disabled, TX data is read from [SPI_W0_REG~SPI_W15_REG](#) and the data address is incremented by 1 on each byte transferred. **If the data byte length is larger than 64, the data in [SPI_W0_REG~SPI_W15_REG](#) may be sent more than once.**

Take each 256 bytes as a cycle:

- * The first 64 bytes (Byte 0~Byte 63) are read from [SPI_W0_REG~SPI_W15_REG](#), sequentially.
- * Byte 64~Byte 255 are read from [SPI_W15_REG\[31:24\]](#) repeatedly.
- * Byte 256~Byte 319 (the first 64 bytes in another 256 bytes) are read from [SPI_W0_REG~SPI_W15_REG](#) again, sequentially, same as the behaviors described above.

For instance: to send 258 bytes (Byte 0~Byte 257), the data is read from the registers as follows:

- * The first 64 bytes (Byte 0~Byte 63) are read from [SPI_W0_REG~SPI_W15_REG](#), sequentially.
- * Byte 64~Byte 255 are read from [SPI_W15_REG\[31:24\]](#) repeatedly.

- * The other bytes (*Byte 256* and *Byte 257*) are read from [SPI_WO_REG\[7:0\]](#) and [SPI_WO_REG\[15:8\]](#) again, sequentially. The logic is:
 - The address to read data for *Byte 256* is the result of $(256 \% 64 = 0)$, i.e., [SPI_WO_REG\[7:0\]](#).
 - The address to read data for *Byte 257* is the result of $(257 \% 64 = 1)$, i.e., [SPI_WO_REG\[15:8\]](#).
- When [SPI_USR_MOSI_HIGHPART](#) is set, i.e., high part mode is enabled, TX data is read from [SPI_W8_REG~SPI_W15_REG](#) and the data address is incremented by 1 on each byte transferred. **If the data byte length is larger than 32, the data in [SPI_W8_REG~SPI_W15_REG](#) may be sent more than once.**

Take each 256 bytes as a cycle:

- * The first 32 bytes (*Byte 0~Byte 31*) are read from [SPI_W8_REG~SPI_W15_REG](#), sequentially.
- * *Byte 32~Byte 255* are read from [SPI_W15_REG\[31:24\]](#) repeatedly.
- * *Byte 256~Byte 287* (the first 32 bytes in the another 256 bytes) are read from [SPI_W8_REG~SPI_W15_REG](#) again, sequentially, same as the behaviors described above.

For instance: to send 258 bytes (*Byte 0~Byte 257*), the data is read from the registers as follows:

- * The first 32 bytes (*Byte 0~Byte 31*) are read from [SPI_W8_REG~SPI_W15_REG](#), sequentially.
- * *Byte 32~Byte 255* are read from [SPI_W15_REG\[31:24\]](#) repeatedly.
- * The other bytes (*Byte 256* and *Byte 257*) are read from [SPI_W8_REG\[7:0\]](#) and [SPI_W8_REG\[15:8\]](#) again, sequentially. The logic is:
 - The address to read data for *Byte 256* is the result of $(256 \% 32 = 0)$, i.e., [SPI_W8_REG\[7:0\]](#).
 - The address to read data for *Byte 257* is the result of $(257 \% 32 = 1)$, i.e., [SPI_W8_REG\[15:8\]](#).

• RX data

- When [SPI_USR_MISO_HIGHPART](#) is cleared, i.e., high part mode is disabled, RX data is saved to [SPI_WO_REG~SPI_W15_REG](#), and the data address is incremented by 1 on each byte transferred. **If the data byte length is larger than 64, the data in [SPI_WO_REG~SPI_W15_REG](#) may be overwritten.**

Take each 256 bytes as a cycle:

- * The first 64 bytes (*Byte 0~Byte 63*) are saved to [SPI_WO_REG~SPI_W15_REG](#), sequentially.
- * *Byte 64~Byte 255* are saved to [SPI_W15_REG\[31:24\]](#) repeatedly.
- * *Byte 256~Byte 319* (the first 64 bytes in the another 256 bytes) are saved to [SPI_WO_REG~SPI_W15_REG](#) again, sequentially, same as the behaviors described above.

For instance: to receive 258 bytes (*Byte 0~Byte 257*), the data is saved to the registers as follows:

- * The first 64 bytes (*Byte 0~Byte 63*) are saved to [SPI_WO_REG~SPI_W15_REG](#), sequentially.
- * *Byte 64~Byte 255* are saved to [SPI_W15_REG\[31:24\]](#) repeatedly.

- * The other bytes (*Byte 256* and *Byte 257*) are saved to [SPI_WO_REG\[7:0\]](#) and [SPI_WO_REG\[15:8\]](#) again, sequentially. The logic is:
 - The address to save *Byte 256* is the result of $(256 \% 64 = 0)$, i.e., [SPI_WO_REG\[7:0\]](#).
 - The address to save *Byte 257* is the result of $(257 \% 64 = 1)$, i.e., [SPI_WO_REG\[15:8\]](#).
- When [SPI_USR_MISO_HIGHPART](#) is set, i.e., high part mode is enabled, the RX data is saved to [SPI_W8_REG~SPI_W15_REG](#), and the data address is incremented by 1 on each byte transferred. **If the data byte length is larger than 32, the content of [SPI_W8_REG~SPI_W15_REG](#) may be overwritten.**

Take each 256 bytes as a cycle:

- * *Byte 0~Byte 31* are saved to [SPI_W8_REG~SPI_W15_REG](#), sequentially.
- * *Byte 32~Byte 255* are saved to [SPI_W15_REG\[31:24\]](#) repeatedly.
- * *Byte 256~Byte 287* (the first 32 bytes in the another 256 bytes) are saved to [SPI_W8_REG~SPI_W15_REG](#) again, sequentially.

For instance: to receive 258 bytes (*Byte 0~Byte 257*), the data is saved to the registers as follows:

- * The first 32 bytes (*Byte 0~Byte 31*) are saved to [SPI_W8_REG~SPI_W15_REG](#), sequentially.
- * *Byte 32~Byte 255* are saved to [SPI_W15_REG\[31:24\]](#) repeatedly.
- * The other bytes (*Byte 256* and *Byte 257*) are saved to [SPI_W8_REG\[7:0\]](#) and [SPI_W8_REG\[15:8\]](#) again, sequentially. The logic is:
 - The address to save *Byte 256* is the result of $(256 \% 32 = 0)$, i.e., [SPI_W8_REG\[7:0\]](#).
 - The address to save *Byte 257* is the result of $(257 \% 32 = 1)$, i.e., [SPI_W8_REG\[15:8\]](#).

Note:

- TX/RX data address mentioned above both are byte-addressable.
 - If high part mode is disabled, Address 0 stands for [SPI_WO_REG\[7:0\]](#), and Address 1 for [SPI_WO_REG\[15:8\]](#), and so on.
 - If high part mode is enabled, Address 0 stands for [SPI_W8_REG\[7:0\]](#), and Address 1 for [SPI_W8_REG\[15:8\]](#), and so on.

The largest address points to [SPI_W15_REG\[31:24\]](#).

- To avoid any possible error in TX/RX data, such as TX data being sent more than once or RX data being overwritten, please make sure the registers are configured correctly.
- LP-SPI functions similarly, but uses LP-SPI registers and LP-SPI data buffers, as shown in Section [43.12.3](#).

43.5.6.2 CPU-Controlled Slave Transfer

In a CPU-controlled slave full-duplex or half-duplex transfer, the RX or TX data is saved to or sent from [SPI_WO_REG~SPI_W15_REG](#), which are byte-addressable.

- In full-duplex communication, the address of [SPI_WO_REG~SPI_W15_REG](#) starts from 0 and is incremented by 1 on each byte transferred. If the data address is larger than 63, the data in [SPI_WO_REG~SPI_W15_REG](#) will be overwritten, same as the behaviors described in the master transfer when high part mode is disabled.

- In half-duplex communication, the *ADDR* value in [transmission format](#) is the start address of the RX or TX data, corresponding to the registers [SPI_W0_REG~SPI_W15_REG](#). The RX or TX address is incremented by 1 on each byte transferred. If the address is larger than 63 (the highest byte address, i.e., [SPI_W15_REG\[31:24\]](#)), the data in [SPI_W8_REG~SPI_W15_REG](#) will be overwritten, same as the behaviors described in the master transfer when high part mode is enabled.

According to your applications, the registers [SPI_W0_REG~SPI_W15_REG](#) can be used as:

- data buffers only
- data buffers and status buffers
- status buffers only

43.5.7 DMA-Controlled Data Transfer

Note:

LP-SPI does not support this transfer. The description below is for GP-SPI only.

DMA-controlled transfer refers to the transfer in which the DMA RX module receives data and the DMA TX module sends data. This transfer is supported both as master and as slave.

A DMA-controlled transfer can be:

- a single transfer, consisting of only one transaction. GP-SPI supports this transfer both as master and as slave.
- a configurable segmented transfer, consisting of several transactions (segments). Only GP-SPI2 supports this transfer when working as a master. For more information, see [Section 43.5.9.5](#).
- a slave segmented transfer, consisting of several transactions (segments). GP-SPI supports this transfer only when working as a slave. For more information, see [Section 43.5.10.3](#).

A DMA-controlled transfer only needs to be triggered once by CPU. When such a transfer is triggered, data is transferred by the DMA engine from or to the DMA-linked memory, without CPU operation.

DMA-controlled transfer supports full-duplex communication, half-duplex communication and functions described in [Section 43.5.9](#) and [Section 43.5.10](#). Meanwhile, the DMA RX module is independent from the DMA TX module, which means that there are four kinds of full-duplex communications:

- Data is received in DMA-controlled mode and sent in DMA-controlled mode.
- Data is received in DMA-controlled mode but sent in CPU-controlled mode.
- Data is received in CPU-controlled mode but sent in DMA-controlled mode.
- Data is received in CPU-controlled mode and sent in CPU-controlled mode.

43.5.7.1 DMA Configuration

- Select a DMA channel *n* and configure DMA TX/RX descriptor. See [Chapter 4 GDMA Controller \(GDMA-AHB, GDMA-AXI\)](#).

- Set [AXI_DMA_INLINK_START_CH \$n\$](#) or [AXI_DMA_OUTLINK_START_CH \$n\$](#) to start DMA RX or TX engine, respectively.
- Before all the DMA TX buffer is used or the DMA TX engine is reset, if [AXI_DMA_OUTLINK_RESTART_CH \$n\$](#) is set, a new TX buffer will be added to the end of the last TX buffer in use.
- DMA RX buffer is linked in the same way as the DMA TX buffer, by setting [AXI_DMA_INLINK_START_CH \$n\$](#) or [AXI_DMA_INLINK_RESTART_CH \$n\$](#) .
- The TX and RX data lengths are determined by the configured DMA TX and RX buffer respectively, both of which are 0~32 KB.
- Initialize DMA inlink and outlink before DMA starts. The bits [SPI_DMA_RX_ENA](#) and [SPI_DMA_TX_ENA](#) in register [SPI_DMA_CONF_REG](#) should be set, otherwise the read/write data will be stored to/sent from the registers [SPI_WO_REG~SPI_W15_REG](#).

When GP-SPI operating as master, if [AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT_ENA](#) is set, then the interrupt [AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT](#) will be triggered when one single transfer or one configurable segmented transfer is finished.

When GP-SPI operating as slave, if [AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT_ENA](#) is set, then the interrupt [AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT](#) will be triggered when one of the following conditions are met.

Table 43.5-9. Interrupt Trigger Condition on GP-SPI Data Transfer as Slave

Transfer Type	Control Bit ¹	Control Bit ²	Condition
Slave Single Transfer	0	0	A single transfer is done.
	1	0	A single transfer is done. Or the length of the received data is equal to (SPI_MS_DATA_BITLEN + 1).
Slave Segmented Transfer	0	1	CMD7 or End_SEG_TRANS is received correctly.
	1	1	CMD7 or End_SEG_TRANS is received correctly. Or the length of the received data is equal to (SPI_MS_DATA_BITLEN + 1).

¹ [SPI_RX_EOF_EN](#)

² [SPI_DMA_SLV_SEG_TRANS_EN](#)

43.5.7.2 DMA TX/RX Buffer Length Control

It is recommended that the length of the configured DMA TX/RX buffer is equal to the length of actual data transferred.

- If the length of the configured DMA TX buffer is shorter than that of the actual data transferred, the extra data will be the same as the last transferred data. [SPI_OUTFIFO_EMPTY_ERR_INT](#) and [AXI_DMA_OUT_EOF_CH \$n\$ _INT](#) are triggered.
- If the length of the configured DMA TX buffer is longer than that of actual data transferred, the TX buffer is not fully used, and the remaining buffer will be used for the following transaction even if a new TX buffer is linked later. Please keep it in mind. Or save the unused data and reset the DMA.
- If the length of the configured DMA RX buffer is shorter than that of the actual data transferred, the extra data will be lost. [SPI_INFIFO_FULL_ERR_INT](#) and [SPI_TRANS_DONE_INT](#) are triggered. But [AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT](#) is not triggered.

- If the length of the configured DMA RX buffer is longer than that of the actual data transferred, the RX buffer is not fully used, and the remaining buffer is discarded. In the following transaction, a new linked buffer will be used directly.

43.5.8 Data Flow Control (Take GP-SPI as an Example)

CPU-controlled and DMA-controlled transfers are supported in GP-SPI both as master and as slave. CPU-controlled transfer means that data is transferred between registers [SPI_WO_REG~SPI_W15_REG](#) and the SPI device. DMA-controlled transfer means that data is transferred between the configured DMA TX/RX buffer and the SPI device. To select between the two transfer types, configure [SPI_DMA_RX_ENA](#) and [SPI_DMA_TX_ENA](#) before the transfer starts.

43.5.8.1 GP-SPI Functional Blocks

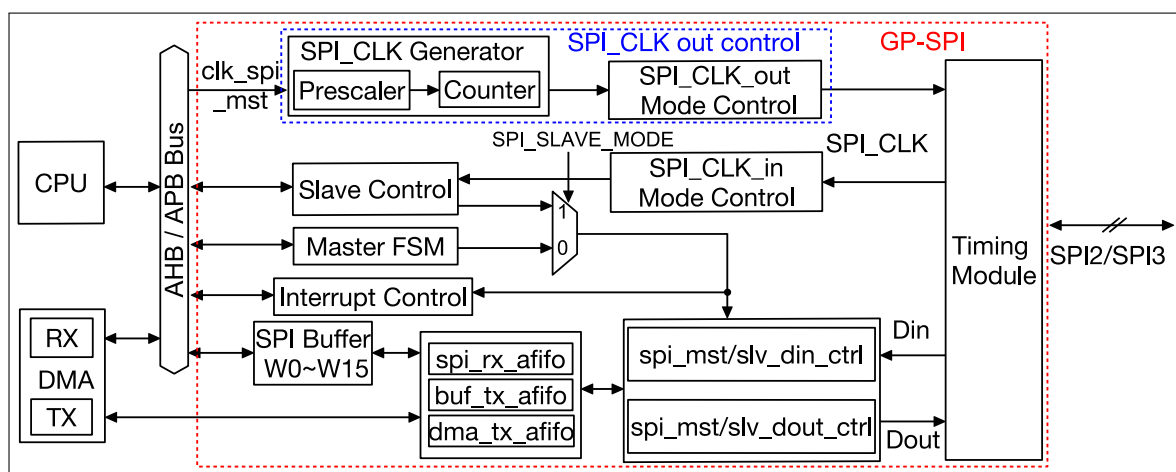


Figure 43.5-2. GP-SPI Functional Blocks

Figure 43.5-2 shows the main functional blocks in GP-SPI, including:

- **Master FSM:** all the features supported in GP-SPI as master are controlled by this state machine together with register configuration.
- **SPI Buffer:** [SPI_WO_REG~SPI_W15_REG](#). See Figure 43.5-1. The data in CPU-controlled transfer is prepared in this buffer.
- **Timing Module:** captures data on SPI2/SPI3 bus.
- **spi_mst/slv_din_ctrl and spi_mst/slv_dout_ctrl:** converts the TX/RX data into bytes.
- **spi_rx_afifo:** stores the received data.
- **buf_tx_afifo:** stores the data to send.
- **dma_tx_afifo:** stores the data from DMA.
- **clk_spi_mst:** this clock is the module clock of GP-SPI and is used in GP-SPI as master to generate SPI_CLK signal for data transfer and for slaves.
- **SPI_CLK Generator:** generates SPI_CLK by dividing clk_spi_mst. The divider is determined by [SPI_CLKCNT_N](#) and [SPI_CLKDIV_PRE](#).

- SPI_CLK_out Mode Control: outputs the SPI_CLK signal for data transfer and for slaves.
- SPI_CLK_in Mode Control: captures the SPI_CLK signal from SPI master when GP-SPI works as a slave.

43.5.8.2 Data Flow Control as Master

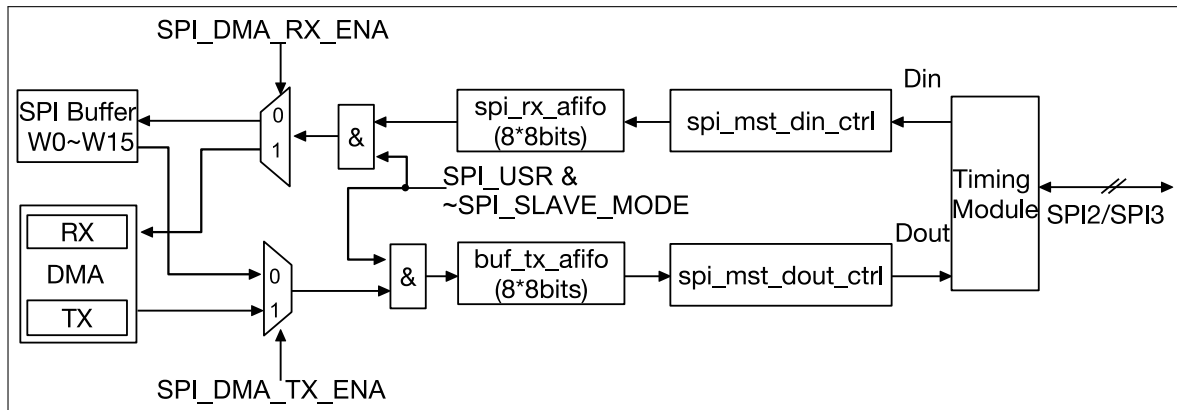


Figure 43.5-3. Data Flow Control in GP-SPI as Master

Figure 43.5-3 shows the data flow of GP-SPI as master. Its control logic is as follows:

- RX data: data bits in SPI2/SPI3 bus is captured by *Timing Module*, converted into bytes by *spi_mst_din_ctrl* module, then buffered in *spi_rx_afifo*, and finally stored in corresponding addresses according to the transfer types.
 - CPU-controlled transfer: the data is stored to registers [SPI_W0_REG ~SPI_W15_REG](#).
 - DMA-controlled transfer: the data is stored to DMA RX buffer.
- TX data: the TX data is from corresponding addresses according to transfer modes and is saved to *buf_tx_afifo*.
 - CPU-controlled transfer: TX data is from [SPI_W0_REG~SPI_W15_REG](#).
 - DMA-controlled transfer: TX data is from DMA TX buffer.

The data in *buf_tx_afifo* is sent out to *Timing Module* in 1/2/4/(8)-bit modes, controlled by GP-SPI state machine. The *Timing Module* can be used for timing compensation. For more information, see Section [43.8](#).

43.5.8.3 Data Flow Control as Slave

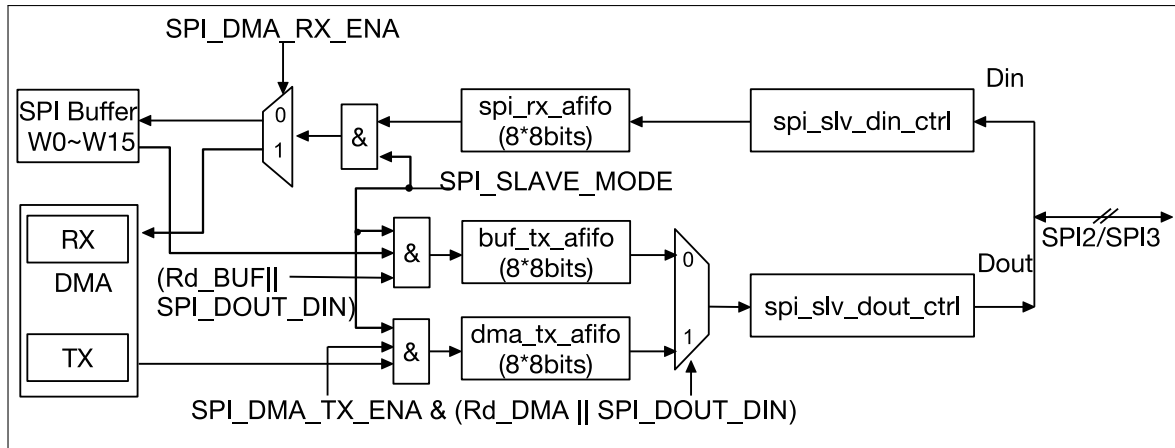


Figure 43.5-4. Data Flow Control in GP-SPI as Slave

Figure 43.5-4 shows the data flow of GP-SPI as slave. Its control logic is as follows:

- In CPU/DMA-controlled full-/half-duplex transfer, when an external SPI master starts the SPI transfer, data on the SPI2/SPI3 bus is captured, converted into bytes by the *spi_slv_din_ctrl* module, and then is stored in *spi_rx_afifo*.
 - In CPU-controlled full-duplex transfer, the received data in *spi_rx_afifo* will be later stored into registers *SPI_WO_REG~SPI_W15_REG*, successively.
 - In half-duplex *Wr_BUF* transfer, when the value of address (*SLV_ADDR[7:0]*) is received, the received data in *spi_rx_afifo* will be stored in the related address of registers *SPI_WO_REG~SPI_W15_REG*.
 - In DMA-controlled full-duplex transfer or in half-duplex *Wr_DMA* transfer, the received data in *spi_rx_afifo* will be stored in the configured DMA RX buffer.
- In CPU-controlled full-/half-duplex transfer, the data to send is stored in *buf_tx_afifo*. In DMA-controlled full-/half-duplex transfer, the data to send is stored in *dma_tx_afifo*. Therefore, *Rd_BUF* transaction controlled by CPU and *Rd_DMA* transaction controlled by DMA can be done in one slave segmented transfer.
 - In CPU-controlled full-duplex transfer, when *SPI_SLAVE_MODE* and *SPI_DOUTDIN* are set and *SPI_DMA_TX_ENA* is cleared, the data in *SPI_WO_REG~SPI_W15_REG* will be stored into *buf_tx_afifo*.
 - In CPU-controlled half-duplex transfer, when *SPI_SLAVE_MODE* is set, *SPI_DOUTDIN* is cleared, *Rd_BUF* command and *SLV_ADDR[7:0]* are received, the data started from the related address of *SPI_WO_REG~SPI_W15_REG* will be stored into *buf_tx_afifo*.
 - In DMA-controlled full-duplex transfer, when *SPI_SLAVE_MODE*, *SPI_DOUTDIN*, and *SPI_DMA_TX_ENA* are set, the data in the configured DMA TX buffer will be stored into *dma_tx_afifo*.
 - In DMA-controlled half-duplex transfer, when *SPI_SLAVE_MODE* is set, *SPI_DOUTDIN* is cleared, and *Rd_DMA* command is received, the data in the configured DMA TX buffer will be stored into *dma_tx_afifo*.

The data in *buf_tx_afifo* or *dma_tx_afifo* is sent out by *splv_dout_ctrl* module in 1/2/4(8)-bit modes.

43.5.9 GP-SPI as a Master

GP-SPI can be configured as a SPI master by clearing the bit [SPI_SLAVE_MODE](#) in [SPI_SLAVE_REG](#). In this operation mode, GP-SPI provides clock signal (the divided clock from GP-SPI module clock) and CS signal.

43.5.9.1 State Machine

When GP-SPI works as a master, the state machine controls GP-SPI's various states during data transfer, including configuration (CONF), preparation (PREP), command (CMD), address (ADDR), dummy (DUMMY), data out (DOUT), and data in (DIN) states. GP-SPI is mainly used to access 1/2/4/8-bit SPI devices, such as flash and external RAM, thus the naming of GP-SPI states keeps consistent with the sequence naming of flash and external RAM. The meaning of each state is described as follows and Figure [43.5-5](#) shows the workflow of GP-SPI state machine.

1. IDLE: GP-SPI is not active or is operating as a slave.
2. CONF (valid for GP-SPI2 only): only used in DMA-controlled [configurable segmented transfer](#). Set [SPI_USR](#) and [SPI_USR_CONF](#) to enable this state. If this state is not enabled, it means the current transfer is a single transfer.
3. PREP: prepare an SPI transaction and control SPI CS setup time. Set [SPI_USR](#) and [SPI_CS_SETUP](#) to enable this state.
4. CMD: send command sequence. Set [SPI_USR](#) and [SPI_USR_COMMAND](#) to enable this state.
5. ADDR: send address sequence. Set [SPI_USR](#) and [SPI_USR_ADDR](#) to enable this state.
6. DUMMY (wait cycle): send dummy sequence. set [SPI_USR](#) and [SPI_USR_DUMMY](#) to enable this state.
7. DATA: transfer data.
 - DOUT: send data sequence. Set [SPI_USR](#) and [SPI_USR_MOSI](#) to enable this state.
 - DIN: receive data sequence. Set [SPI_USR](#) and [SPI_USR_MISO](#) to enable this state.
8. DONE: control SPI CS hold time. Set [SPI_USR](#) to enable this state.

Note:

To start this state machine, set [SPI_USR](#) first. [SPI_MST_FD_WAIT_DMA_TX_DATA](#) controls when [SPI_USR](#) takes effect:

- 0: the configured state takes effect immediately after [SPI_USR](#) and other control registers are configured.
- 1: if DOUT state is configured, the [SPI_USR](#) and other control registers will take effect, and the state machine will start, only when the data is ready in *buf_tx_afifo*.

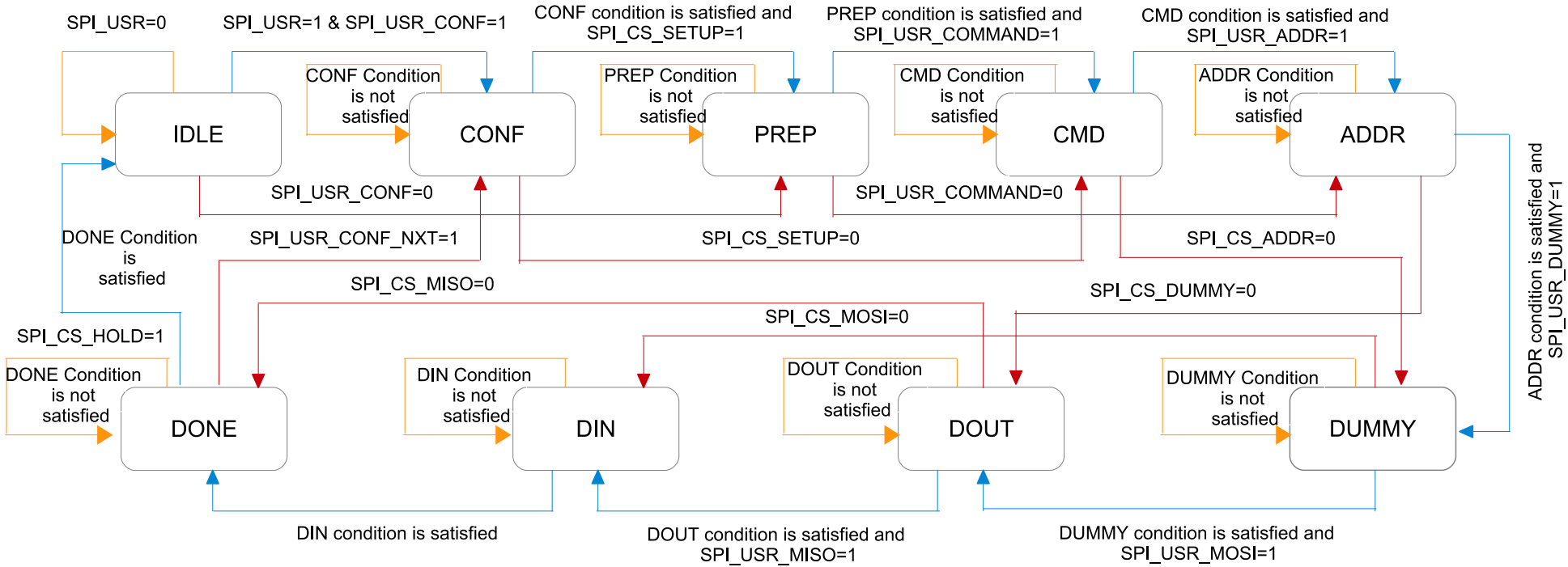


Figure 43.5-5. GP-SPI State Machine as Master

Legend to state flow:

- —: indicates corresponding state condition is not satisfied; repeats current state.
- —: corresponding registers are set and conditions are satisfied; goes to next state.
- —: state registers are not set; skips one or more following states, depending on the registers of the following states are set or not.

Explanation to the conditions listed in the figure above:

- CONF condition: $\text{gpc}[17:0] \geq \text{SPI_CONF_BITLEN}[17:0]$
- PREP condition: $\text{gpc}[4:0] \geq \text{SPI_CS_SETUP_TIME}[4:0]$
- CMD condition: $\text{gpc}[3:0] \geq \text{SPI_USR_COMMAND_BITLEN}[3:0]$
- ADDR condition: $\text{gpc}[4:0] \geq \text{SPI_USR_ADDR_BITLEN}[4:0]$
- DUMMY condition: $\text{gpc}[7:0] \geq \text{SPI_USR_DUMMY_CYCLELEN}[7:0]$
- DOUT condition: $\text{gpc}[17:0] \geq \text{SPI_MS_DATA_BITLEN}[17:0]$
- DIN condition: $\text{gpc}[17:0] \geq \text{SPI_MS_DATA_BITLEN}[17:0]$
- DONE condition: $(\text{gpc}[4:0] \geq \text{SPI_CS_HOLD_TIME}[4:0] \parallel \text{SPI_CS_HOLD} == 1'b0)$

A counter ($\text{gpc}[17:0]$) is used in the state machine to control the cycle length of each state. The states CONF, PREP, CMD, ADDR, DUMMY, DOUT, and DIN can be enabled or disabled independently. The cycle length of each state can also be configured independently.

43.5.9.2 Register Configuration for State and Bit Mode Control

Introduction

The registers, related to GP-SPI state control, are listed in Table 43.5-10. Users can enable QPI mode for GP-SPI by setting the bit `SPI_QPI_MODE` in register `SPI_USER_REG`.

Table 43.5-10. Registers Used for State Control in 1/2/4/8-bit Modes

State	Control Registers for 1-bit Mode SPI2/SPI3 Bus	Control Registers for 2-bit Mode SPI2/SPI3 Bus	Control Registers for 4-bit Mode SPI2/SPI3 Bus	Control Registers for 8-bit Mode SPI2 Bus
CMD	SPI_USR_COMMAND_VALUE SPI_USR_COMMAND_BITLEN SPI_USR_COMMAND	SPI_USR_COMMAND_VALUE SPI_USR_COMMAND_BITLEN SPI_FCMD_DUAL SPI_USR_COMMAND	SPI_USR_COMMAND_VALUE SPI_USR_COMMAND_BITLEN SPI_FCMD_QUAD SPI_USR_COMMAND	SPI_USR_COMMAND_VALUE SPI_USR_COMMAND_BITLEN SPI_FCMD_OCT SPI_USR_COMMAND
ADDR	SPI_USR_ADDR_VALUE SPI_USR_ADDR_BITLEN SPI_USR_ADDR	SPI_USR_ADDR_VALUE SPI_USR_ADDR_BITLEN SPI_USR_ADDR SPI_FADDR_DUAL	SPI_USR_ADDR_VALUE SPI_USR_ADDR_BITLEN SPI_USR_ADDR SPI_FADDR_QUAD	SPI_USR_ADDR_VALUE SPI_USR_ADDR_BITLEN SPI_USR_ADDR SPI_FADDR_OCT
DUMMY	SPI_USR_DUMMY_CYCLELEN SPI_USR_DUMMY	SPI_USR_DUMMY_CYCLELEN SPI_USR_DUMMY	SPI_USR_DUMMY_CYCLELEN SPI_USR_DUMMY	SPI_USR_DUMMY_CYCLELEN SPI_USR_DUMMY
DIN	SPI_USR_MISO SPI_MS_DATA_BITLEN	SPI_USR_MISO SPI_MS_DATA_BITLEN SPI_FREAD_DUAL	SPI_USR_MISO SPI_MS_DATA_BITLEN SPI_FREAD_QUAD	SPI_USR_MISO SPI_MS_DATA_BITLEN SPI_FREAD_OCT
DOUT	SPI_USR_MOSI SPI_MS_DATA_BITLEN	SPI_USR_MOSI SPI_MS_DATA_BITLEN SPI_FWRITE_DUAL	SPI_USR_MOSI SPI_MS_DATA_BITLEN SPI_FWRITE_QUAD	SPI_USR_MOSI SPI_MS_DATA_BITLEN SPI_FWRITE_OCT

As shown in Table 43.5-10, the registers in each cell should be configured to set the SPI2/SPI3 bus to corresponding bit mode, i.e., the mode shown in the table header, at a specific state (corresponding to the first column).

Configuration

For instance, when GP-SPI2 reads data, and

- CMD is in 1-bit mode
- ADDR is in 2-bit mode
- DUMMY lasts for 8 clock cycles
- DIN is in 4-bit mode

The register configuration can be as follows:

1. Configure CMD state related registers.
 - Configure the required command value in [SPI_USR_COMMAND_VALUE](#).
 - Configure command bit length in [SPI_USR_COMMAND_BITLEN](#). [SPI_USR_COMMAND_BITLEN](#) = expected bit length - 1.
 - Set [SPI_USR_COMMAND](#).
 - Clear [SPI_FCMD_DUAL](#), [SPI_FCMD_QUAD](#), and [SPI_FCMD_OCT](#).
2. Configure ADDR state related registers.
 - Configure the required address value in [SPI_USR_ADDR_VALUE](#).
 - Configure address bit length in [SPI_USR_ADDR_BITLEN](#). [SPI_USR_ADDR_BITLEN](#) = expected bit length - 1.
 - Set [SPI_USR_ADDR](#) and [SPI_FADDR_DUAL](#).
 - Clear [SPI_FADDR_QUAD](#) and [SPI_FCMD_OCT](#).
3. Configure DUMMY state related registers.
 - Configure DUMMY cycles in [SPI_USR_DUMMY_CYCLELEN](#). [SPI_USR_DUMMY_CYCLELEN](#) = expected clock cycles - 1.
 - Set [SPI_USR_DUMMY](#).
4. Configure DIN state related registers.
 - Configure read data bit length in [SPI_MS_DATA_BITLEN](#). [SPI_MS_DATA_BITLEN](#) = expected bit length - 1.
 - Set [SPI_FREAD_QUAD](#) and [SPI_USR_MISO](#).
 - Clear [SPI_FREAD_DUAL](#) and [SPI_FCMD_OCT](#).
 - Configure DMA for DMA-controlled transfer. For CPU controlled transfer, no action is needed.
5. Clear [SPI_USR_MOSI](#).
6. Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.
7. Set [SPI_USR](#) to start GP-SPI2 transfer.

Note:

Updating the configuration when the GP-SPI works as master described in this and subsequent sections requires to set [SPI_UPDATE](#) accordingly to synchronize the configuration from AHB_CLK domain to clk_spi_mst domain. For more detailed configuration, see the sections above. No operation is required when the GP-SPI works as slave.

When writing data (DOUT state), [SPI_USR_MOSI](#) should be configured instead, while [SPI_USR_MISO](#) should be cleared. The output data bit length is the value of [SPI_MS_DATA_BITLEN](#) + 1. Output data should be configured in GP-SPI2 data buffer ([SPI_WO_REG](#)~[SPI_W15_REG](#)) for CPU-controlled transfer, or DMA TX buffer for DMA-controlled transfer.

Pay special attention to the command value in [SPI_USR_COMMAND_VALUE](#) and to address value in [SPI_USR_ADDR_VALUE](#).

The configuration of command value is as follows:

Table 43.5-11. Sending Sequence of Command Value

COMMAND_BITLEN ¹	COMMAND_VALUE ²	BIT_ORDER ³	Sending Sequence of Command Value
0~7	[7:0]	1	COMMAND_VALUE [COMMAND_BITLEN :0] is sent first.
		0	COMMAND_VALUE [7:7- COMMAND_BITLEN] is sent first.
8~15	[15:0]	1	COMMAND_VALUE [7:0] is sent first, and then COMMAND_VALUE [COMMAND_BITLEN :8].
		0	COMMAND_VALUE [7:0] is sent first, and then COMMAND_VALUE [15:15- COMMAND_BITLEN].

¹ [SPI_USR_COMMAND_BITLEN](#): this field is used to configure the bit length of the command.

² [SPI_USR_COMMAND_VALUE](#): command value is written into this field. For which part of this field is used, see the table above.

³ [SPI_WR_BIT_ORDER](#): 0: LSB first; 1: MSB first.

The configuration of address value is as follows:

Table 43.5-12. Sending Sequence of Address Value

ADDR_BITLEN ¹	ADDR_VALUE ²	BIT_ORDER ³	Sending Sequence of Address Value
0~7	[31:24]	1	ADDR_VALUE [ADDR_BITLEN + 24:24] is sent first.
		0	ADDR_VALUE [31:31- ADDR_BITLEN] is sent first.
8~15	[31:16]	1	ADDR_VALUE [31:24] is sent first, and then ADDR_VALUE [ADDR_BITLEN + 8:16].
		0	ADDR_VALUE [31:24] is sent first, and then ADDR_VALUE [23:31- ADDR_BITLEN].
16~23	[31:8]	1	ADDR_VALUE [31:16] is sent first, and then ADDR_VALUE [ADDR_BITLEN -8:8].

		0	ADDR_VALUE[31:16] is sent first, and then ADDR_VALUE[15:31-ADDR_BITLEN] .
24~31	[31:0]	1	ADDR_VALUE[31:8] is sent first, and then ADDR_VALUE[ADDR_BITLEN-24:0] .
		0	ADDR_VALUE[31:8] is sent first, and then ADDR_VALUE[7:31-ADDR_BITLEN] .

¹ [SPI_USR_ADDR_BITLEN](#): this field is used to configure the bit length of the address.

² [SPI_USR_ADDR_VALUE](#): address value is written into this field. For which part of this field is used, see the table above.

³ [SPI_WR_BIT_ORDER](#): 0: LSB first; 1: MSB first.

43.5.9.3 Full-Duplex Communication (1-bit Mode Only)

Introduction

GP-SPI supports SPI full-duplex communication. In this mode, SPI master provides CLK and CS signals, exchanging data with SPI slave in 1-bit mode via MOSI (SPI2D/SPI3_D, sending) and MISO (SPI2Q/SPI3_Q, receiving) at the same time. To enable this communication mode, set the bit [SPI_DOUTDIN](#) in register [SPI_USER_REG](#). Figure 43.5-6 illustrates the connection of GP-SPI2 with its slave in full-duplex communication.

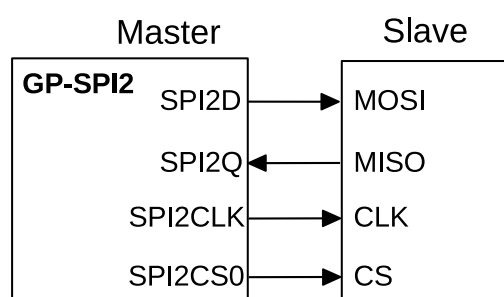


Figure 43.5-6. Full-Duplex Communication Between GP-SPI2 Master and a Slave

In full-duplex communication, the behavior of states CMD, ADDR, DUMMY, DOUT and DIN are configurable. Usually, the states CMD, ADDR and DUMMY are not used in this communication. The bit length of transferred data is configured in [SPI_MS_DATA_BITLEN](#). The actual bit length used in communication equals to ([SPI_MS_DATA_BITLEN](#) + 1).

Configuration (Take GP-SPI2 as an example)

To start a data transfer, follow the steps below:

- Configure the IO path via IO MUX or GPIO matrix between GP-SPI2 and an external SPI device.
- Configure AHB clock (AHB_CLK, see Chapter 10 [Reset and Clock](#)) and module clock (clk_spi_mst) for the GP-SPI2 module.
- Set [SPI_DOUTDIN](#) and clear [SPI_SLAVE_MODE](#), to enable master full-duplex communication.
- Configure GP-SPI2 registers listed in Table 43.5-10.
- Configure SPI CS setup time and hold time according to Section 43.6.

- Set the property of SPI2CLK according to Section 43.7.
- Prepare data according to the selected transfer type:
 - In CPU-controlled MOSI transfer, prepare data in registers [SPI_W0_REG~SPI_W15_REG](#).
 - In DMA-controlled transfer,
 - * configure [SPI_DMA_TX_ENA/SPI_DMA_RX_ENA](#),
 - * configure DMA TX/RX link,
 - * and start DMA TX/RX engine, as described in Section 43.5.7 and Section 43.5.8.
- Configure interrupts and wait for SPI slave to get ready for transfer.
- Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.
- Set [SPI_USR](#) in register [SPI_CMD_REG](#) to start the transfer and wait for the configured interrupts.

43.5.9.4 Half-Duplex Communication (1/2/4/(8)-bit Mode)

Introduction

In this mode, GP-SPI provides CLK and CS signals. Only one side (SPI master or slave) can send data at a time, while the other side receives the data. To enable this communication mode, clear the bit [SPI_DOUTDIN](#) in register [SPI_USER_REG](#). The standard format of SPI half-duplex communication is CMD + [ADDR +] [DUMMY +] [DOUT or DIN]. The states ADDR, DUMMY, DOUT, and DIN are optional, and can be disabled or enabled independently.

As described in Section 43.5.9.2, the properties of GP-SPI states: CMD, ADDR, DUMMY, DOUT and DIN, such as cycle length, value, and parallel bus bit mode, can be set independently. For the register configuration, see Table 43.5-10.

The detailed properties of half-duplex GP-SPI are as follows:

1. CMD: 0~16 bits, master output, slave input.
2. ADDR: 0~32 bits, master output, slave input.
3. DUMMY: 0~256 SPI2CLK/SPI3_CLK cycles, master output, slave input.
4. DOUT: 0~512 bits (64 bytes) in CPU-controlled transfer, 0~256 Kbits (32 KB) in DMA-controlled single transfer, and unlimited length in DMA-controlled configurable segmented transfer, master output, slave input.
5. DIN: 0~512 bits (64 bytes) in CPU-controlled transfer, 0~256 Kbits (32 KB) in DMA-controlled single transfer, and unlimited length in DMA-controlled configurable segmented transfer, slave output, master input.

GP-SPI2 also supports Double Transfer Rate (DTR) in 1/2/4/8-bit modes, in which data is sent and received at the rising and the falling edges of SPI clock:

- If [SPI_CMD_DTR_EN](#) is set, the CMD value will be sent in DTR mode; otherwise CMD phase will be in Single Transfer Rate (STR) mode.
- If [SPI_ADDR_DTR_EN](#) is set, the ADDR value will be sent in DTR mode; otherwise ADDR phase will be in STR mode.

- If [SPI_DATA_DTR_EN](#) is set, the input data in state DIN or output data in state DOUT will be sent in DTR mode; otherwise the data will be in STR mode.

The control bits [SPI_CMD_DTR_EN](#), [SPI_ADDR_DTR_EN](#), and [SPI_DATA_DTR_EN](#) can be configured independently, which means CMD in STR mode, ADDR and DOUT or DIN in DTR mode are supported.

GP-SPI2 can only output SPI2DQS signal, but can not receive the signal. Therefore, only flash or external RAM working in fixed dummy mode (dummy cycles in a read sequence is fixed) are supported.

Configuration (Take GP-SPI2 as an example)

The register configuration can be as follows:

1. Configure the IO path via HP IO MUX or HP GPIO matrix between GP-SPI2 and an external SPI device.
2. Configure AHB clock (AHB_CLK) and module clock (clk_spi_mst) for the GP-SPI2 module.
3. Clear [SPI_DOUTDIN](#) and [SPI_SLAVE_MODE](#), to enable master half-duplex communication.
4. Configure GP-SPI2 registers listed in Table [43.5-10](#).
5. Configure SPI CS setup time and hold time according to Section [43.6](#).
6. Set the property of SPI2CLK according to Section [43.7](#).
7. Prepare data according to the selected transfer type:
 - In CPU-controlled MOSI transfer, prepare data in registers [SPI_W0_REG~SPI_W15_REG](#).
 - In DMA-controlled transfer,
 - configure [SPI_DMA_TX_ENA/SPI_DMA_RX_ENA](#),
 - configure DMA TX/RX link,
 - and start DMA TX/RX engine, as described in Section [43.5.7](#) and Section [43.5.8](#).
8. Configure interrupts and wait for SPI slave to get ready for transfer.
9. Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.
10. Set [SPI_USR](#) in register [SPI_CMD_REG](#) to start the transfer and wait for the configured interrupts.

Application Example (Take GP-SPI2 as an example)

The following example shows how GP-SPI2 accesses flash and external RAM in master half-duplex communication.

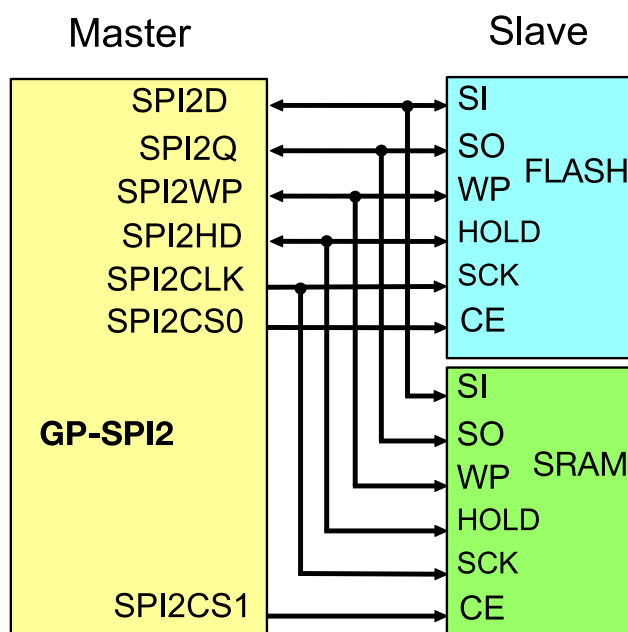


Figure 43.5-7. Connection of GP-SPI2 to Flash and External RAM in 4-bit Mode

Figure 43.5-8 indicates GP-SPI2 Quad I/O Read sequence according to standard flash specification. Other GP-SPI2 command sequences are implemented in accordance with the requirements of SPI slaves.

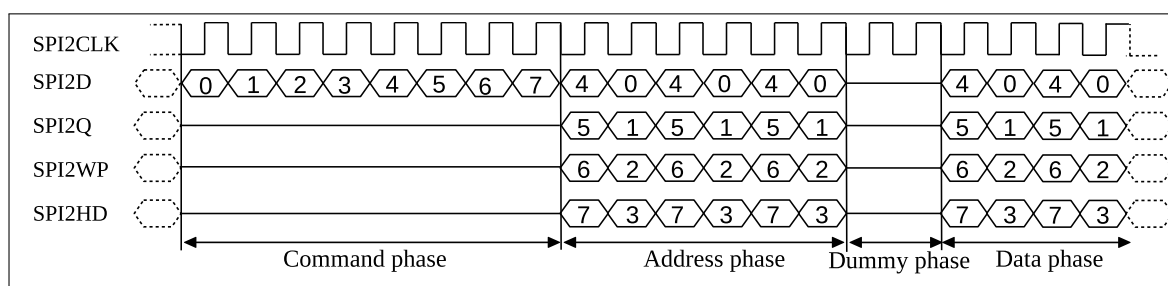


Figure 43.5-8. SPI Quad I/O Read Command Sequence Sent by GP-SPI2 to Flash

43.5.9.5 DMA-Controlled Configurable Segmented Transfer

Note:

- LP-SPI and GP-SPI3 do not support DMA-controlled configurable segmented transfer.
- Note that there is no separate section on how to configure a single transfer as master, since the CONF state of a configurable segmented transfer can be skipped to implement a single transfer.

Introduction

When GP-SPI2 works as a master, it provides a feature named configurable segmented transfer controlled by DMA.

A DMA-controlled transfer as master can be:

- a single transfer, consisting of only one transaction;
- or a configurable segmented transfer, consisting of several transactions (segments).

In a configurable segmented transfer, the registers of each single transaction (segment) are configurable. This feature enables GP-SPI2 to do as many transactions (segments) as configured after such transfer is triggered once by the CPU. Figure 43.5-9 shows how this feature works.

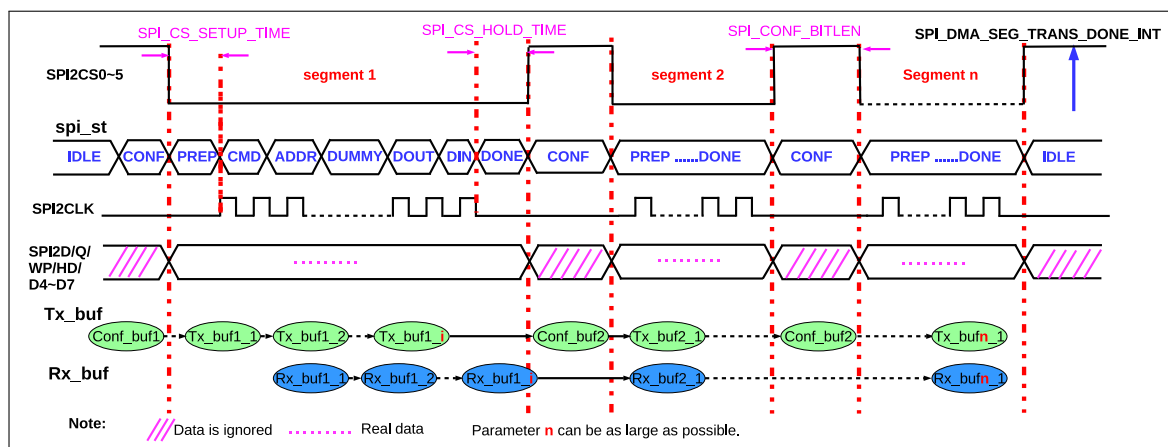


Figure 43.5-9. Configurable Segmented Transfer as Master

As shown in Figure 43.5-9, the registers for one transaction (segment *n*) can be reconfigured by GP-SPI2 hardware according to the content in its Conf_buf*n* during the CONF state, before this segment starts.

It is recommended to provide separate DMA CONF links and CONF buffers (Conf_buf*i* in Figure 43.5-9) for each CONF state. A DMA TX link is used to connect all the CONF buffers and TX data buffers (Tx_buf*i* in Figure 43.5-9) into a chain. Hence, the behavior of the SPI2 bus in each segment can be controlled independently.

For example, in a configurable segmented transfer, its segment *i*, segment *j*, and segment *k* can be configured to full-duplex, half-duplex MISO, and half-duplex MOSI, respectively. *i*, *j*, and *k* represent different segment numbers.

Meanwhile, the state of GP-SPI2, the data length and cycle length of the SPI2 bus, and the behavior of the DMA, can be configured independently for each segment. When this whole DMA-controlled transfer (consisting of several segments) has finished, a GP-SPI2 interrupt, SPI_DMA_SEG_TRANS_DONE_INT, is triggered.

Configuration

1. Configure the IO path via HP IO MUX or HP GPIO matrix between GP-SPI2 and an external SPI device.
2. Configure AHB clock (AHB_CLK) and module clock (clk_spi_mst) for the GP-SPI2 module.
3. Clear SPI_DOUTDIN and SPI_SLAVE_MODE, to enable master half-duplex communication.
4. Configure GP-SPI2 registers listed in Table 43.5-10.
5. Configure SPI CS setup time and hold time according to Section 43.6.
6. Set the property of SPI2CLK according to Section 43.7.
7. Prepare descriptors for DMA CONF buffer and TX data (optional) for each segment. Chain the descriptors of CONF buffer and TX buffers of several segments into one linked list.
8. Similarly, prepare descriptors for RX buffers for each segment and chain them into one linked list.

9. Configure all the needed CONF buffers, TX buffers and RX buffers, respectively for each segment before this DMA-controlled transfer begins.
10. Point `AXI_DMA_OUTLINK_ADDR_CH $n$` to the head address of the CONF and TX buffer descriptor linked list, and then set `AXI_DMA_OUTLINK_START_CH $n$` to start the TX DMA.
11. Clear the bit `SPI_RX_EOF_EN` in register `SPI_DMA_CONF_REG`. Point `AXI_DMA_INLINK_ADDR_CH $n$` to the head address of the CONF and RX buffer descriptor linked list, and then set `AXI_DMA_INLINK_START_CH $n$` to start the RX DMA.
12. Set `SPI_USR_CONF` to enable CONF state.
13. Set `SPI_DMA_SEG_TRANS_DONE_INT_ENA` to enable the `SPI_DMA_SEG_TRANS_DONE_INT` interrupt. Configure other interrupts if needed according to Section 43.11.
14. Wait for all the slaves to get ready for transfer.
15. Set `SPI_DMA_AFIFO_RST`, `SPI_BUF_AFIFO_RST`, and `SPI_RX_AFIFO_RST` to reset these buffers.
16. Set `SPI_USR` to start this DMA-controlled transfer.
17. Wait for `SPI_DMA_SEG_TRANS_DONE_INT` interrupt, which means this transfer has finished and the data has been stored into corresponding memory.

Note:

Prepare the data to send for each segment in its PREP state. Shall ensure that:

$$(SPI_CS_SETUP_TIME + 1) \times T_{SPI_CLK} \geq 4 \times (T_{AHB_CLK} + T_{clk_spi_mst})$$

Configuration of CONF Buffer and Magic Value

In a configurable segmented transfer, only registers which change from the last transaction (segment) need to be re-configured to new values in CONF state. The configuration of other registers can be skipped (i.e., kept the same) to save time and chip resources.

The first word in DMA CONF buffer i , called `SPI_BIT_MAP_WORD`, defines whether given GP-SPI2 register is to be updated or not in segment i . The relation of `SPI_BIT_MAP_WORD` and GP-SPI2 registers to update can be seen in Table 43.5-13 Bitmap (BM) Table. If a bit in the BM table is set to 1, its corresponding register value will be updated in this segment. Otherwise, if some registers should be kept from being changed, the related bits should be set to 0.

Table 43.5-13. BM Table for CONF State

BM Bit	Register	BM Bit	Register
0	<code>SPI_ADDR_REG</code>	7	<code>SPI_MISC_REG</code>
1	<code>SPI_CTRL_REG</code>	8	<code>SPI_DIN_MODE_REG</code>
2	<code>SPI_CLOCK_REG</code>	9	<code>SPI_DIN_NUM_REG</code>
3	<code>SPI_USER_REG</code>	10	<code>SPI_DOUT_MODE_REG</code>
4	<code>SPI_USER1_REG</code>	11	<code>SPI_DMA_CONF_REG</code>
5	<code>SPI_USER2_REG</code>	12	<code>SPI_DMA_INT_ENA_REG</code>
6	<code>SPI_MS_DLEN_REG</code>	13	<code>SPI_DMA_INT_CLR_REG</code>

Then new values of all the registers to be modified should be placed right after SPI_BIT_MAP_WORD, in consecutive words in the CONF buffer.

To ensure the correctness of the content in each CONF buffer, the value in SPI_BIT_MAP_WORD[31:28] is used as a “magic value”, and will be compared with SPI_DMA_SEG_MAGIC_VALUE in register SPI_SLAVE_REG. The value of SPI_DMA_SEG_MAGIC_VALUE should be configured before this DMA-controlled transfer starts, and can not be changed during these segments.

- If SPI_BIT_MAP_WORD[31:28] == SPI_DMA_SEG_MAGIC_VALUE, this DMA-controlled transfer continues normally. SPI_DMA_SEG_TRANS_DONE_INT is triggered at the end of this DMA-controlled transfer.
- If SPI_BIT_MAP_WORD[31:28] != SPI_DMA_SEG_MAGIC_VALUE, GP-SPI2 state (spi_st) goes back to IDLE and the transfer is ended immediately. The interrupt SPI_DMA_SEG_TRANS_DONE_INT is still triggered, with SPI_SEG_MAGIC_ERR_INT_RAW bit set to 1.

CONF Buffer Configuration Example

Table 43.5-14 and Table 43.5-15 provide an example to show how to configure a CONF buffer for a transaction (segment *i*) in which SPI_ADDR_REG, SPI_CTRL_REG, SPI_CLOCK_REG, SPI_USER_REG, and SPI_USER1_REG need to be updated.

Table 43.5-14. An Example of CONF buffer *i* in Segment *i*

CONF buffer <i>i</i>	Note
SPI_BIT_MAP_WORD	The first word in this buffer. Its value is 0xA000001F in this example when the SPI_DMA_SEG_MAGIC_VALUE is set to 0xA. As shown in Table 43.5-15, bits 0, 1, 2, 3, and 4 are set, indicating the following registers will be updated.
SPI_ADDR_REG	The second word, stores the new value to SPI_ADDR_REG.
SPI_CTRL_REG	The third word, stores the new value to SPI_CTRL_REG.
SPI_CLOCK_REG	The fourth word, stores the new value to SPI_CLOCK_REG.
SPI_USER_REG	The fifth word, stores the new value to SPI_USER_REG.
SPI_USER1_REG	The sixth word, stores the new value to SPI_USER1_REG.

Table 43.5-15. BM Bit Value and Register to Be Updated in This Example

BM Bit	Value	Register	BM Bit	Value	Register
0	1	SPI_ADDR_REG	7	0	SPI_MISC_REG
1	1	SPI_CTRL_REG	8	0	SPI_DIN_MODE_REG
2	1	SPI_CLOCK_REG	9	0	SPI_DIN_NUM_REG
3	1	SPI_USER_REG	10	0	SPI_DOUT_MODE_REG
4	1	SPI_USER1_REG	11	0	SPI_DMA_CONF_REG
5	0	SPI_USER2_REG	12	0	SPI_DMA_INT_ENA_REG
6	0	SPI_MS_DLEN_REG	13	0	SPI_DMA_INT_CLR_REG

Notes

In a DMA-controlled configurable segmented transfer, please pay special attention to the following bits:

- **SPI_USR_CONF**: set **SPI_USR_CONF** before **SPI_USR** is set, to enable this transfer.
- **SPI_USR_CONF_NXT**: if segment *i* is not the final transaction of this whole DMA-controlled transfer, its **SPI_USR_CONF_NXT** bit should be set to 1.
- **SPI_CONF_BITLEN**: GP-SPI2 CS setup time and hold time are programmable independently in each segment, see Section 43.6 for detailed configuration. The CS high time in each segment is about:

$$(\text{SPI_CONF_BITLEN} + 5) \times T_{\text{AHB_CLK}}$$

The CS high time in CONF state can be set from 62.5 *ns*~3.2768 *ms* when $f_{\text{AHB_CLK}}$ is 80 MHz.

(**SPI_CONF_BITLEN** + 5) will overflow from (0x40000 - **SPI_CONF_BITLEN** - 5) if **SPI_CONF_BITLEN** is larger than 0x3FFFA.

43.5.10 GP-SPI Works as a Slave

GP-SPI can be used as a slave to communicate with an SPI master. As a slave, GP-SPI supports 1-bit SPI, 2-bit dual SPI, 4-bit quad SPI, and QPI modes, with specific communication formats. To enable this mode, set **SPI_SLAVE_MODE** in register **SPI_SLAVE_REG**.

The CS signal must be held low during the transfer, and its falling/rising edges indicate the start/end of a single or segmented transfer. Take CS low-level effective as an example, when GP-SPI is used as a slave, the CS invalid duration (high-level) between each SPI transfer should not be less than $8 T_{\text{AHB_CLK}}$ to ensure that each transfer can end normally.

43.5.10.1 Configurable Communication Formats

When GP-SPI works as a slave, SPI full-duplex and half-duplex communications are available. To select from the two communications, configure **SPI_DOUTDIN** in register **SPI_USER_REG**.

Full-duplex communication means that input data and output data are transmitted simultaneously throughout the entire transaction. All bits are treated as input or output data, which means no command, address or dummy states are expected. The interrupt **SPI_TRANS_DONE_INT** is triggered once the transaction ends.

In half-duplex communication, the format is CMD+ADDR+DUMMY+DATA (DIN or DOUT).

- “DIN” means that an SPI master reads data from GP-SPI.
- “DOUT” means that an SPI master writes data to GP-SPI.

The detailed properties of each state are as follows:

1. CMD:

- Indicate the function of SPI slave.
- One byte from master to slave.
- Only the values in Table 43.5-16 and Table 43.5-17 are valid.
- Can be sent in 1-bit SPI mode or 4-bit QPI mode.

2. ADDR:

- The address for **Wr_BUF** and **Rd_BUF** commands in CPU-controlled transfer, or placeholder bits in other transfers and can be defined by application.

- One byte from master to slave.
- Can be sent in 1-bit, 2-bit or 4-bit modes according to the command.

3. DUMMY:

- Its value is meaningless. SPI slave prepares data in this state.
- Bit mode of SPI2 bus is also meaningless here.
- Last for eight SPI_CLK cycles.

4. DIN or DOUT:

- Data length can be 0~64 bytes in CPU-controlled transfer and unlimited in DMA-controlled transfer.
- Can be sent in 1-bit, 2-bit or 4-bit modes according to the CMD value.

Note:

The states of ADDR and DUMMY can never be skipped in any half-duplex communications.

When a half-duplex transaction is complete, the transferred CMD and ADDR values are latched into [SPI_SLV_LAST_COMMAND](#) and [SPI_SLV_LAST_ADDR](#), respectively. [SPI_SLV_CMD_ERR_INT_RAW](#) will be set if the transferred CMD value is not supported by GP-SPI as slave. [SPI_SLV_CMD_ERR_INT_RAW](#) can only be cleared by software.

43.5.10.2 CMD Values Supported in Half-Duplex Communication

In half-duplex communication, the defined values of CMD determine the transfer types. Unsupported CMD values are disregarded, meanwhile the related transfer is ignored and [SPI_SLV_CMD_ERR_INT_RAW](#) is set. The transfer format is CMD (8 bits) + ADDR (8 bits) + DUMMY (8 SPI_CLK cycles) + DATA (unit in bytes). The detailed description of CMD[3:0] is as follows:

- 0x1 (Wr_BUF): CPU-controlled write operation. Master sends data and GP-SPI receives data. The data is stored in the related address of [SPI_WO_REG~SPI_W15_REG](#).
- 0x2 (Rd_BUF): CPU-controlled read operation. Master receives the data sent by GP-SPI. The data comes from the related address of [SPI_WO_REG~SPI_W15_REG](#).
- 0x3 (Wr_DMA): DMA-controlled write operation. Master sends data and GP-SPI receives data. The data is stored in GP-SPI DMA RX buffer.
- 0x4 (Rd_DMA): DMA-controlled read operation. Master receives the data sent by GP-SPI. The data comes from GP-SPI DMA TX buffer.
- 0x7 (CMD7): used to generate an [SPI_SLV_CMD7_INT](#) interrupt. It can also generate a [AXI_DMA_IN_SUC_EOF_CHn_INT](#) interrupt in a slave segmented transfer when DMA RX link is used. But it will not end GP-SPI's slave segmented transfer.
- 0x8 (CMD8): only used to generate an [SPI_SLV_CMD8_INT](#) interrupt, which will not end GP-SPI's slave segmented transfer.
- 0x9 (CMD9): only used to generate an [SPI_SLV_CMD9_INT](#) interrupt, which will not end GP-SPI's slave segmented transfer.

- 0xA (CMDA): only used to generate an [SPI_SLV_CMDA_INT](#) interrupt, which will not end GP-SPI's slave segmented transfer.

CMD7, CMD8, CMD9, and CMDA commands are reserved for user definition. These commands can be used as handshake signals, as passwords of some specific functions, as trigger signals of some user defined actions, and so on.

1/2/4-bit modes in states of CMD, ADDR, DATA are supported, which are determined by value of CMD[7:4]. The DUMMY state is always in 1-bit mode and lasts for eight SPI_CLK cycles. The definition of CMD[7:4] is as follows:

- 0x0: CMD, ADDR, and DATA states all are in 1-bit mode.
- 0x1: CMD and ADDR are in 1-bit mode. DATA is in 2-bit mode.
- 0x2: CMD and ADDR are in 1-bit mode. DATA is in 4-bit mode.
- 0x5: CMD is in 1-bit mode. ADDR and DATA are in 2-bit mode.
- 0xA: CMD is in 1-bit mode, ADDR and DATA are in 4-bit mode or in QPI mode.

In addition, if the value of CMD[7:0] is 0x05, 0xA5, 0x06, or 0xDD, DUMMY and DATA states are skipped. The definition of CMD[7:0] is as follows:

- 0x05 (End_SEG_TRANS): master sends this command to end slave segmented transfer in SPI mode.
- 0xA5 (End_SEG_TRANS): master sends this command to end slave segmented transfer in QPI mode.
- 0x06 (En_QPI): GP-SPI enters QPI mode when receiving this command and the bit [SPI_QPI_MODE](#) in register [SPI_USER_REG](#) is set.
- 0xDD (Ex_QPI): GP-SPI exits QPI mode when receiving this command and the bit [SPI_QPI_MODE](#) is cleared.

All the CMD values supported by GP-SPI are listed in Table [43.5-16](#) and Table [43.5-17](#). Note that the DUMMY state is always in 1-bit mode and lasts for eight SPI_CLK cycles.

Table 43.5-16. CMD Values Supported in SPI Mode

Transfer Type	CMD[7:0]	CMD State	ADDR State	DATA State
Wr_BUF	0x01	1-bit mode	1-bit mode	1-bit mode
	0x11	1-bit mode	1-bit mode	2-bit mode
	0x21	1-bit mode	1-bit mode	4-bit mode
	0x51	1-bit mode	2-bit mode	2-bit mode
	0xA1	1-bit mode	4-bit mode	4-bit mode
Rd_BUF	0x02	1-bit mode	1-bit mode	1-bit mode
	0x12	1-bit mode	1-bit mode	2-bit mode
	0x22	1-bit mode	1-bit mode	4-bit mode
	0x52	1-bit mode	2-bit mode	2-bit mode
	0xA2	1-bit mode	4-bit mode	4-bit mode
Wr_DMA	0x03	1-bit mode	1-bit mode	1-bit mode
	0x13	1-bit mode	1-bit mode	2-bit mode
	0x23	1-bit mode	1-bit mode	4-bit mode
	0x53	1-bit mode	2-bit mode	2-bit mode

Table 43.5-16. CMD Values Supported in SPI Mode

Transfer Type	CMD[7:0]	CMD State	ADDR State	DATA State
	0xA3	1-bit mode	4-bit mode	4-bit mode
Rd_DMA	0x04	1-bit mode	1-bit mode	1-bit mode
	0x14	1-bit mode	1-bit mode	2-bit mode
	0x24	1-bit mode	1-bit mode	4-bit mode
	0x54	1-bit mode	2-bit mode	2-bit mode
	0xA4	1-bit mode	4-bit mode	4-bit mode
CMD7	0x07	1-bit mode	1-bit mode	-
	0x17	1-bit mode	1-bit mode	-
	0x27	1-bit mode	1-bit mode	-
	0x57	1-bit mode	2-bit mode	-
	0xA7	1-bit mode	4-bit mode	-
CMD8	0x08	1-bit mode	1-bit mode	-
	0x18	1-bit mode	1-bit mode	-
	0x28	1-bit mode	1-bit mode	-
	0x58	1-bit mode	2-bit mode	-
	0xA8	1-bit mode	4-bit mode	-
CMD9	0x09	1-bit mode	1-bit mode	-
	0x19	1-bit mode	1-bit mode	-
	0x29	1-bit mode	1-bit mode	-
	0x59	1-bit mode	2-bit mode	-
	0xA9	1-bit mode	4-bit mode	-
CMDA	0x0A	1-bit mode	1-bit mode	-
	0x1A	1-bit mode	1-bit mode	-
	0x2A	1-bit mode	1-bit mode	-
	0x5A	1-bit mode	2-bit mode	-
	0xAA	1-bit mode	4-bit mode	-
End_SEG_TRANS	0x05	1-bit mode	-	-
En_QPI	0x06	1-bit mode	-	-

Table 43.5-17. CMD Values Supported in QPI Mode

Transfer Type	CMD[7:0]	CMD State	ADDR State	DATA State
Wr_BUF	0xA1	4-bit mode	4-bit mode	4-bit mode
Rd_BUF	0xA2	4-bit mode	4-bit mode	4-bit mode
Wr_DMA	0xA3	4-bit mode	4-bit mode	4-bit mode
Rd_DMA	0xA4	4-bit mode	4-bit mode	4-bit mode
CMD7	0xA7	4-bit mode	4-bit mode	-
CMD8	0xA8	4-bit mode	4-bit mode	-
CMD9	0xA9	4-bit mode	4-bit mode	-
CMDA	0xAA	4-bit mode	4-bit mode	-
End_SEG_TRANS	0xA5	4-bit mode	4-bit mode	-
Ex_QPI	0xDD	4-bit mode	4-bit mode	-

Master sends 0x06 CMD (En_QPI) to set GP-SPI slave to QPI mode and all the states of supported transfer will be in 4-bit mode afterwards. If 0xDD CMD (Ex_QPI) is received, GP-SPI slave will be back to SPI mode.

Other transfer types than these described in Table 43.5-16 and Table 43.5-17 are ignored. But if the CS low-level duration is longer than two APB_CLK cycles, [SPI_TRANS_DONE_INT](#) will be triggered. For more information on interrupts triggered at the end of transmissions, please refer to Section 43.11.

43.5.10.3 Slave Single Transfer and Slave Segmented Transfer

When GP-SPI works as a slave, it supports full-duplex and half-duplex communications controlled by DMA and by CPU. DMA-controlled transfer can be a single transfer, or a slave segmented transfer consisting of several transactions (segments). The CPU-controlled transfer can only be one single transfer, since each CPU-controlled transaction needs to be triggered by CPU.

In a slave segmented transfer, all transfer types listed in Table 43.5-16 and Table 43.5-17 are supported in a single transaction (segment). It means that CPU-controlled transaction and DMA-controlled transaction can be mixed in one slave segmented transfer.

It is recommended that in a slave segmented transfer:

- CPU-controlled transaction is used for handshake communication and short data transfers.
- DMA-controlled transaction is used for large data transfers.

43.5.10.4 Configuration of Slave Single Transfer

When operating as slave, GP-SPI supports CPU/DMA-controlled full-duplex/half-duplex single transfers.

The register configuration procedure is as follows (take GP-SPI2 as an example):

1. Configure the IO path via HP IO MUX or HP GPIO matrix between GP-SPI2 and an external SPI device.
2. Configure AHB clock (AHB_CLK).
3. Set [SPI_SLAVE_MODE](#) to enable slave mode.
4. Configure [SPI_DOUTDIN](#):
 - 1: enable full-duplex communication.
 - 0: enable half-duplex communication.
5. Prepare data:
 - if CPU-controlled transfer is selected and GP-SPI2 is used to send data, then prepare data in registers [SPI_W0_REG~SPI_W15_REG](#).
 - if DMA-controlled transfer is selected,
 - configure [SPI_DMA_TX_ENA/SPI_DMA_RX_ENA](#) and [SPI_RX_EOF_EN](#).
 - configure DMA TX/RX link,
 - and start DMA TX/RX engine, as described in Section 43.5.7 and Section 43.5.8.
6. Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.

7. Clear [SPI_DMA_SLV_SEG_TRANS_EN](#) in register [SPI_DMA_CONF_REG](#) to enable slave single transfer.
8. Set [SPI_TRANS_DONE_INT_ENA](#) in register [SPI_DMA_INT_ENA_REG](#) and wait for the interrupt [SPI_TRANS_DONE_INT](#). In DMA-controlled mode, it is recommended to wait for the interrupt [AXI_DMA_IN_SUC_EOF_CHn_INT](#) when DMA RX buffer is used, which means that data has been stored in the related memory. Other interrupts described in Section 43.11 are optional.

43.5.10.5 Configuration of Slave Segmented Transfer in Half-Duplex

DMA must be used in this mode. The register configuration procedure is as follows (take GP-SPI2 as an example):

1. Configure the IO path via HP IO MUX or HP GPIO matrix between GP-SPI2 and an external SPI device.
2. Configure APB clock (APB_CLK).
3. Set [SPI_SLAVE_MODE](#) to enable slave mode.
4. Clear [SPI_DOUTDIN](#) to enable half-duplex communication.
5. Prepare data in registers [SPI_WO_REG](#)~[SPI_W15_REG](#), if needed.
6. Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.
7. Set bits [SPI_DMA_RX_ENA](#) and [SPI_DMA_TX_ENA](#). Clear the bit [SPI_RX_EOF_EN](#). Configure DMA TX/RX link and start DMA TX/RX engine, as shown in Section 43.5.7 and Section 43.5.8.
8. Set [SPI_DMA_SLV_SEG_TRANS_EN](#) in register [SPI_DMA_CONF_REG](#) to enable slave segmented transfer.
9. Set [SPI_DMA_SEG_TRANS_DONE_INT_ENA](#) in register [SPI_DMA_INT_ENA_REG](#) and wait for the interrupt [SPI_DMA_SEG_TRANS_DONE_INT](#), which means that the segmented transfer has finished and data has been put into the related memory. Other interrupts described in Section 43.11 are optional.

When End_SEG_TRANS (0x05 in SPI mode, 0xA5 in QPI mode) is received by GP-SPI, this slave segmented transfer is ended and the interrupt [SPI_DMA_SEG_TRANS_DONE_INT](#) is triggered.

43.5.10.6 Configuration of Slave Segmented Transfer in Full-Duplex

DMA must be used in this mode. In such transfer, the data is transferred from and to the DMA buffer. The interrupt [AXI_DMA_IN_SUC_EOF_CHn_INT](#) is triggered when the transfer ends.

The register configuration procedure is as follows (take GP-SPI2 as an example):

1. Configure the IO path via HP IO MUX or HP GPIO matrix between GP-SPI2 and an external SPI device.
2. Configure APB clock (APB_CLK).
3. Set [SPI_SLAVE_MODE](#) and [SPI_DOUTDIN](#), to enable slave full-duplex communication.
4. Set [SPI_DMA_AFIFO_RST](#), [SPI_BUF_AFIFO_RST](#), and [SPI_RX_AFIFO_RST](#) to reset these buffers.
5. Set [SPI_DMA_TX_ENA](#)/[SPI_DMA_RX_ENA](#). Configure DMA TX/RX link and start DMA TX/RX engine, as shown in Section 43.5.7 and Section 43.5.8.
6. Set [SPI_RX_EOF_EN](#) in register [SPI_DMA_CONF_REG](#). Configure [SPI_MS_DATA_BITLEN](#)[17:0] in register [SPI_MS_DLEN_REG](#) to the bit length of the received DMA data.
7. Set [SPI_DMA_SLV_SEG_TRANS_EN](#) in register [SPI_DMA_CONF_REG](#) to enable slave segmented transfer.

8. Set `AXI_DMA_IN_SUC_EOF_CH $n$ _INT_ENA` and wait for the interrupt `AXI_DMA_IN_SUC_EOF_CH $n$ _INT`.

43.6 CS Setup Time and Hold Time Control

SPI bus CS (SPI_CS) setup time and hold time are very important to meet the timing requirements of various SPI devices (e.g., flash or PSRAM).

CS setup time is the time between the CS falling edge and the first latch edge of SPI bus CLK (SPI_CLK). The first latch edge for mode 0 and mode 3 is rising edge, and falling edge for mode 1 and mode 2.

CS hold time is the time between the last latch edge of SPI_CLK and the CS rising edge.

When operating as slave, the CS setup time and hold time should be longer than $0.5 \times T_{\text{SPI_CLK}}$, otherwise the SPI transfer may be incorrect. $T_{\text{SPI_CLK}}$ is one cycle of SPI_CLK.

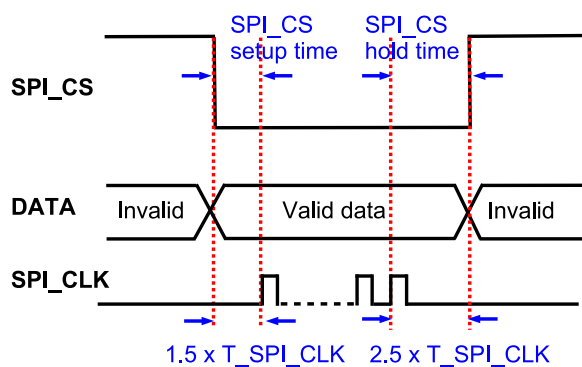
When operating as master, set the CS setup time by specifying `SPI_CS_SETUP` in register `SPI_USER_REG` and `SPI_CS_SETUP_TIME` in register `SPI_USER1_REG`.

- If `SPI_CS_SETUP` is cleared, the SPI CS setup time is $0.5 \times T_{\text{SPI_CLK}}$.
- If `SPI_CS_SETUP` is set, the SPI CS setup time is $(\text{SPI_CS_SETUP_TIME} + 1.5) \times T_{\text{SPI_CLK}}$.

Set the CS hold time by specifying `SPI_CS_HOLD` in register `SPI_USER_REG` and `SPI_CS_HOLD_TIME` in register `SPI_USER1_REG`.

- If `SPI_CS_HOLD` is cleared, the SPI CS hold time is $0.5 \times T_{\text{SPI_CLK}}$.
- If `SPI_CS_HOLD` is set, the SPI CS hold time is $(\text{SPI_CS_HOLD_TIME} + 1.5) \times T_{\text{SPI_CLK}}$.

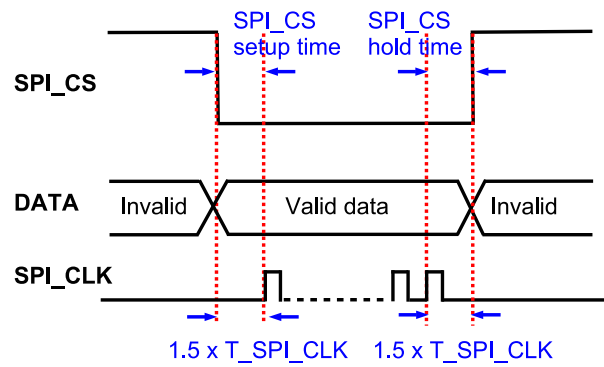
Figure 43.6-1 and Figure 43.6-2 show the recommended CS timing and register configuration to access external RAM and flash.



Register Configurations:

`SPI_CS_SETUP = 1; SPI_CS_SETUP_TIME = 0;`
`SPI_CS_HOLD = 1; SPI_CS_HOLD_TIME = 1.`

Figure 43.6-1. Recommended CS Timing and Settings When Accessing External RAM



Register Configurations:

SPI_CS_SETUP = 1; SPI_CS_SETUP_TIME = 0;
SPI_CS_HOLD = 1; SPI_CS_HOLD_TIME = 0.

Figure 43.6-2. Recommended CS Timing and Settings When Accessing Flash

43.7 Clock Control

43.7.1 GP-SPI Clock Control

GP-SPI has the following clocks:

- `clk_spi_mst`: module clock of GP-SPI, derived from the following clock sources, depending on the configuration of `HP_SYS_CLKRST_GPSPi2/3_CLK_SRC_SEL`:
 - 0: XTAL_CLK
 - 1: RC_FAST_CLK
 - 2: SDIO_PLLO_CLK
 - 3: APLL_CLK
 - 4: SPILL_CLK (480 MHz)

The divider value is equal to `HP_SYS_CLKRST_GPSPi2/3_MST_CLK_DIV_NUM + 1`.

`HP_SYS_CLKRST_GPSPi2/3_SYS_CLK_EN` is used to enable the selected clock source. `clk_spi_mst` is used in GP-SPI as master to generate `SPI_CLK` signal for data transfer and for slaves.

- `clk_hclk`: module timing compensation clock of GP-SPI, which is a frequency-doubled clock derived from the same source as `clk_spi_mst`. Its clock source is also determined by `HP_SYS_CLKRST_GPSPi2/3_CLK_SRC_SEL`. To use this clock, `HP_SYS_CLKRST_GPSPi2/3_HS_CLK_EN` must be set, and the divider value is equal to `HP_SYS_CLKRST_GPSPi2/3_HS_CLK_DIV_NUM + 1`.
- `SPI_CLK`: this is the output clock when the GP-SPI works as master, or IO input clock when the GP-SPI works as slave.
- `AHB_CLK`: clock for register configuration.

When operating as master, the maximum output clock frequency of GP-SPI is $f_{\text{clk_spi_mst}}$. To have slower frequencies, the output clock frequency can be divided as follows:

$$f_{\text{SPI_CLK}} = \frac{f_{\text{clk_spi_mst}}}{(\text{SPI_CLKCNT_N} + 1)(\text{SPI_CLKDIV_PRE} + 1)}$$

The divider is configured by `SPI_CLKCNT_N` and `SPI_CLKDIV_PRE` in register `SPI_CLOCK_REG`. When the bit `SPI_CLK_EQU_SYSCLK` in register `SPI_CLOCK_REG` is set, the output clock frequency of GP-SPI will be $f_{\text{clk_spi_mst}}$. For other integral clock divisions, `SPI_CLK_EQU_SYSCLK` should be cleared.

When operating as slave, the supported input clock frequency ($f_{\text{SPI_CLK}}$) of GP-SPI is:

- If $f_{\text{AHB_CLK}} \geq 60 \text{ MHz}$, $f_{\text{SPI_CLK}} \leq 60 \text{ MHz}$.
- If $f_{\text{AHB_CLK}} < 60 \text{ MHz}$, $f_{\text{SPI_CLK}} \leq f_{\text{AHB_CLK}}$.

43.7.2 LP-SPI Clock Control

LP-SPI has the following clocks:

- `lp_spi_pclk`: LP-SPI module clock and clock for register configuration. `lp_spi_pclk` is used to generate `LP_SPI_CLK` for data transfer and for slaves when LP-SPI works as master. `LPPERI_CLK_EN_LP_SPI` is used to enable this `lp_spi_pclk`.
- `LP_SPI_CLK`: output clock as master.

43.7.3 Clock Phase and Polarity

SPI protocol has four clock modes, i.e., modes 0~3. See Figure 43.7-1 and Figure 43.7-2 (excerpted from SPI protocol):

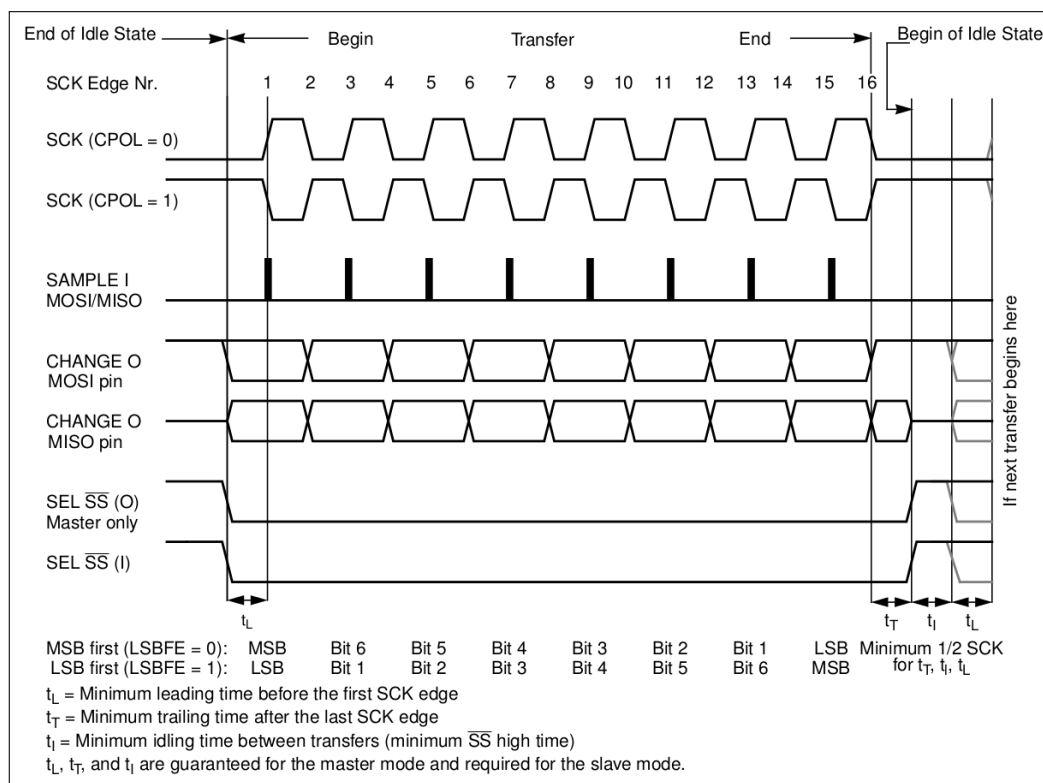


Figure 43.7-1. SPI Clock Mode 0 or 2

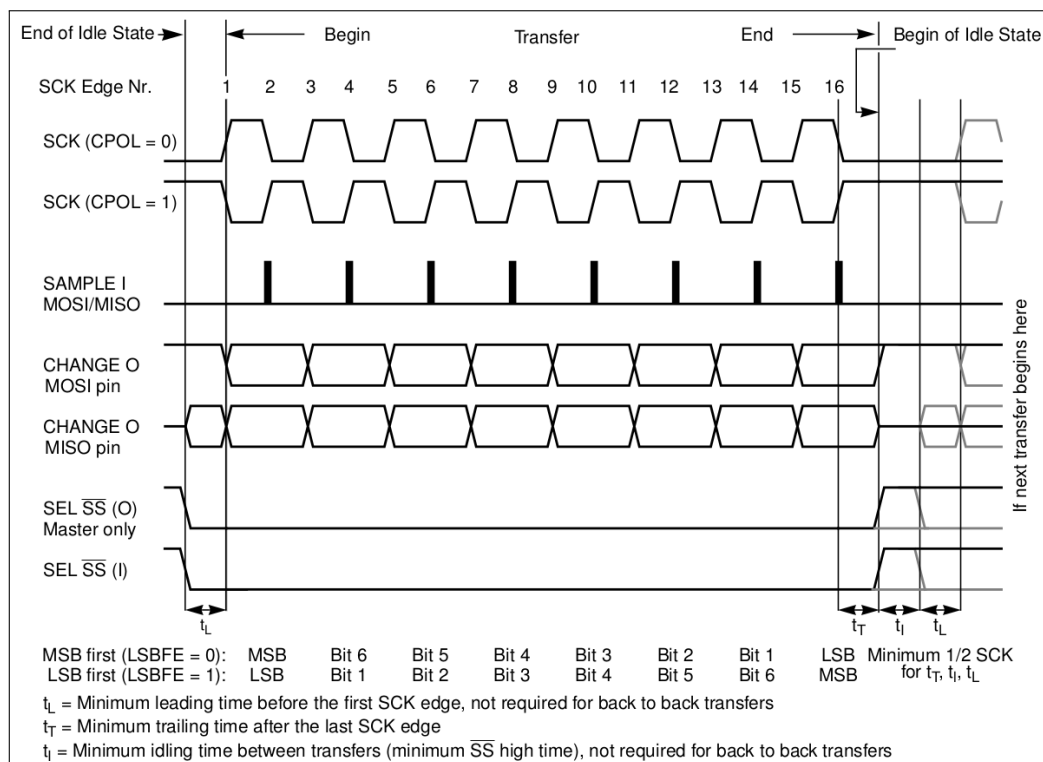


Figure 43.7-2. SPI Clock Mode 1 or 3

1. Mode 0: CPOL = 0, CPHA = 0; SCK is 0 when the SPI is in idle state; data is changed on the falling edge of SCK and sampled on the rising edge. The first data is shifted out before the first falling edge of SCK.
2. Mode 1: CPOL = 0, CPHA = 1; SCK is 0 when the SPI is in idle state; data is changed on the rising edge of SCK and sampled on the falling edge.
3. Mode 2: CPOL = 1, CPHA = 0; SCK is 1 when the SPI is in idle state; data is changed on the rising edge of SCK and sampled on the falling edge. The first data is shifted out before the first rising edge of SCK.
4. Mode 3: CPOL = 1, CPHA = 1; SCK is 1 when the SPI is in idle state; data is changed on the falling edge of SCK and sampled on the rising edge.

43.7.4 Clock Control as Master

The four clock modes 0~3 are supported in GP-SPI as master. The polarity and phase of GP-SPI clock are controlled by the bit [SPI_CLK_IDLE_EDGE](#) in register [SPI_MISC_REG](#) and the bit [SPI_CLK_OUT_EDGE](#) in register [SPI_USER_REG](#). The register configuration for SPI clock modes 0~3 is provided in Table 43.7-1, and can be changed according to the path delay in the application.

Table 43.7-1. Clock Phase and Polarity Configuration as Master

Control Bit	Mode 0	Mode 1	Mode 2	Mode 3
SPI_CLK_IDLE_EDGE	0	0	1	1
SPI_CLK_OUT_EDGE	0	1	1	0

$\square$ [SPI_CLK_MODE](#) is used to select the number of rising edges of SPI_CLK when SPI_CS raises high to be 0, 1, 2 or SPI_CLK always on.

Note:

When [SPI_CLK_MODE](#) is configured to 1 or 2, the bit [SPI_CS_HOLD](#) must be set and the value of [SPI_CS_HOLD_TIME](#) should be larger than 1.

43.7.5 Clock Control as Slave

GP-SPI as slave also supports clock modes 0~3. The polarity and phase are configured by the bits [SPI_TSCK_I_EDGE](#) and [SPI_RSCK_I_EDGE](#) in register [SPI_USER_REG](#). The output edge of data is controlled by [SPI_CLK_MODE_13](#) in register [SPI_SLAVE_REG](#). The detailed register configuration is shown in Table 43.7-2:

Table 43.7-2. Clock Phase and Polarity Configuration as Slave

Control Bit	Mode 0	Mode 1	Mode 2	Mode 3
SPI_TSCK_I_EDGE	0	1	1	0
SPI_RSCK_I_EDGE	0	1	1	0
SPI_CLK_MODE_13	0	1	0	1

43.8 GP-SPI Timing Compensation

Introduction

The I/O lines are mapped via GPIO matrix or IO MUX. But there is no timing adjustment in IO MUX. The input data and output data can be delayed for 1 or 2 IO MUX operating clock cycles at the rising or falling edge in GPIO matrix. For detailed register configuration, see Chapter 9 [GPIO Matrix and IO MUX](#). Timing compensation is supported only by GP-SPI.

Figure 43.8-1 shows the timing compensation control for GP-SPI as master, including the following paths:

- “CLK”: the output path of GP-SPI bus clock. The clock is sent out by [SPI_CLK](#) out control module, passes through GPIO matrix or IO MUX and then goes to an external SPI device.
- “IN”: data input path of GP-SPI (see line 3 path in color purple in Figure 43.8-1). The input data from an external SPI device passes through GPIO matrix or IO MUX, then is adjusted by the Timing Module (see Figure 43.5-2) and finally is stored into [spi_rx_afifo](#).
- “OUT”: data output path of GP-SPI (see line 2 path in color rose-red in Figure 43.8-1). The output data is sent out to the Timing Module, passes through GPIO matrix or IO MUX and is then captured by an external SPI device.

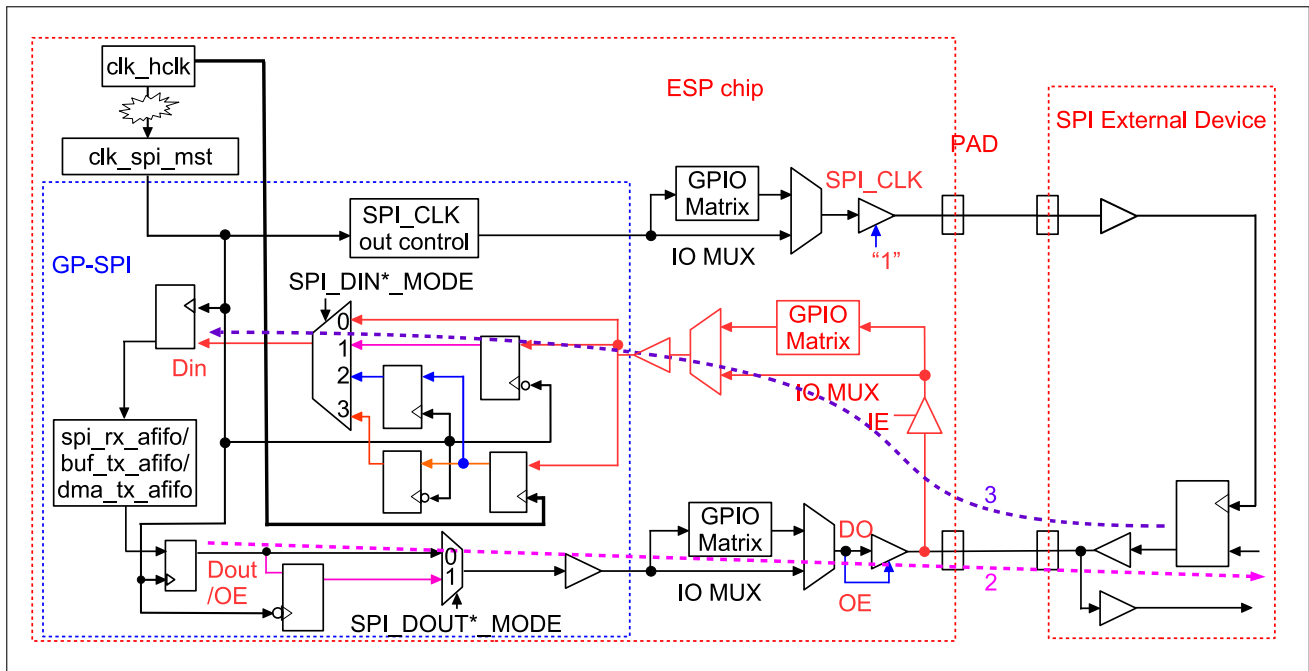


Figure 43.8-1. Timing Compensation Control Diagram in GP-SPI as Master

Every input and output data is passing through the Timing Module and the module can be used to apply delay in units of $T_{\text{clk_spi_mst}}$ (one cycle of clk_spi_mst) on rising or falling edge.

Key Registers

- [SPI_DIN_MODE_REG](#): select the latch edge of input data
- [SPI_DIN_NUM_REG](#): select the delay cycles of input data
- [SPI_DOUT_MODE_REG](#): select the latch edge of output data

Timing Compensation Example

Figure 43.8-2 shows a timing compensation example in GP-SPI2 as master. Note that DUMMY cycle length is configurable to compensate the delay in I/O lines, so as to enhance the performance of GP-SPI2.

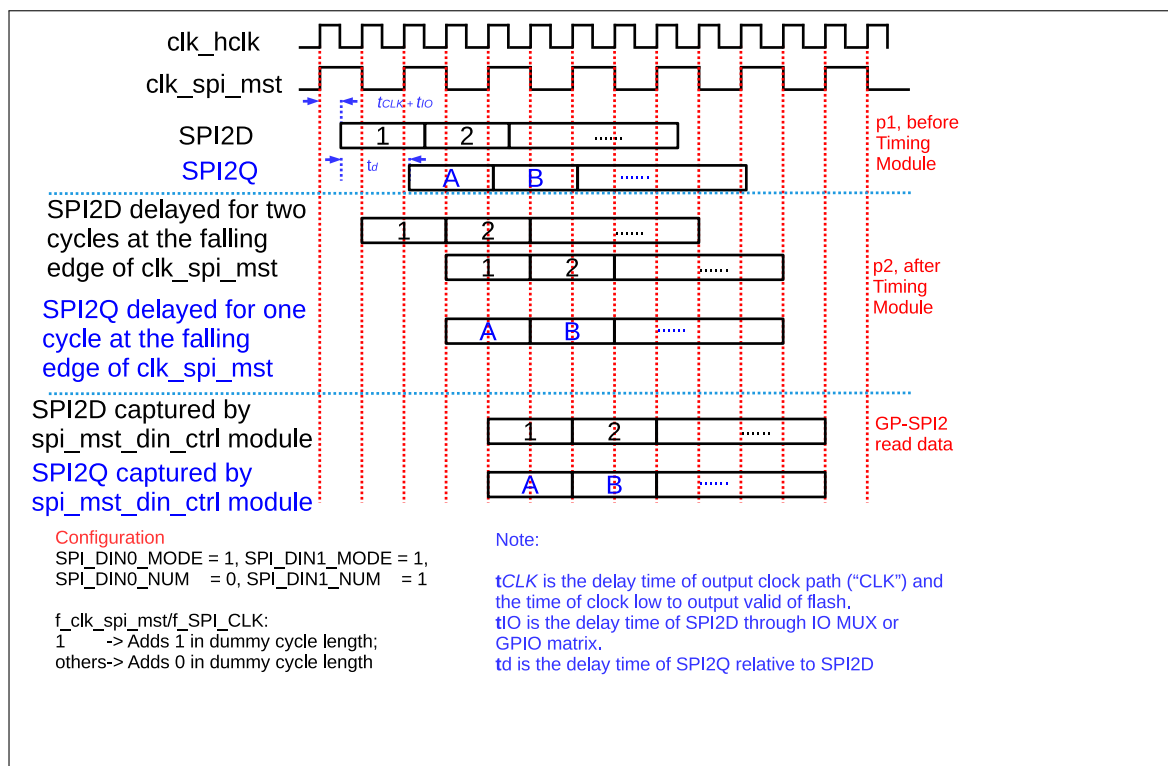


Figure 43.8-2. Timing Compensation Example in GP-SPI2 as Master

In Figure 43.8-2, "p1" is the point of input data of Timing Module, "p2" is the point of output data of Timing Module. Since the input data $SPI2Q$ is unaligned to $SPI2D$, the read data of GP-SPI2 will be wrong without the timing compensation.

To get the correct read data, follow the settings below. Assuming $f_{clk_spi_mst}$ equals to f_{SPI_CLK} :

- Delay $SPI2D$ for two cycles at the falling edge of clk_spi_mst .
- Delay $SPI2Q$ for one cycle at the falling edge of clk_spi_mst .
- Add one extra dummy cycle.

When GP-SPI works as slave, if the bit `SPI_RSCK_DATA_OUT` in register `SPI_SLAVE_REG` is set to 1, the output data is sent at latch edge, which is half an SPI clock cycle earlier. This can be used for slave mode timing compensation.

43.9 LP-SPI Wake-Up

LP-SPI supports wake-up feature when working as a slave. When LP-SPI is in a sleep state (set `LP_SPI_SLEEP_EN` to 1), it can generate a wake_up signal and trigger PMU to wake up the chip by configuring `LP_SPI_SLV_WK_MODE_SEL` to select the wake up mode. For the detailed wake-up flow, see Chapter 14 *Low-Power Management*.

- When `LP_SPI_SLV_WK_MODE_SEL` = 0, LP-SPI wakes up the chip after detecting the start bit.
- When `LP_SPI_SLV_WK_MODE_SEL` = 1, LP-SPI wakes up the chip after receiving a specific character sequence.

- [LP_SPI_SLEEP_WK_DATA_SEL](#) selects to monitor the specific character starting from the CMD state or DIN state.
- [LP_SPI_SLEEP_DIS_RXFIFO_WR_EN](#) selects whether to save data before waking up.
- [LP_SPI_SLV_WK_CHAR0~LP_SPI_SLV_WK_CHAR4](#) define wake up characters. LP-SPI wakes up the chip once it detects the character sequence consisting of CHAR0/CHAR1/CHAR2/CHAR3/CHAR4.
- [LP_SPI_SLV_WK_CHAR_NUM](#) and [LP_SPI_SLV_WK_CHAR_MASK](#) configure various character sequences, see Table 43.9-1. LP-SPI also supports fuzzy detection. For example, [LP_SPI_WK_SLV_CHAR_NUM](#) = 5 and [LP_SPI_SLV_WK_MASK](#) = 0x15 (binary 10101), then the character sequence will be */CHAR1*/CHAR3/*.

Table 43.9-1. LP-SPI Wake-Up Configuration When Working as a Slave

LP_SPI_WK_SLV_CHAR_NUM	LP_SPI_SLV_WK_MASK	Character Sequence
1	0xF	CHAR4
2	0x7	CHAR3/CHAR4
3	0x3	CHAR2/CHAR3/CHAR4
4	0x1	CHAR1/CHAR2/CHAR3/CHAR4
5	0x0	CHAR0/CHAR1/CHAR2/CHAR3/CHAR4

43.10 Differences Among LP-SPI, GP-SPI2, and GP-SPI3

43.10.1 Feature Differences

Table 43.10-1 lists the differences among GP-SPI2, GP-SPI3, and LP-SPI on their supported features.

Table 43.10-1. Feature Differences Among GP-SPI2, GP-SPI3, and LP-SPI

Feature	GP-SPI2	GP-SPI3	LP-SPI
Data modes supported in CMD/ADDR/DOUT/DIN states when working as a master	1/2/4/8-bit	1/2/4-bit	1-bit
Data modes supported in CMD/ADDR/DOUT/DIN states when working as a slave	1/2/4-bit	1/2/4-bit	1-bit
DMA-controlled transfers	Support all DMA-controlled transfers	Not support DMA-controlled configurable segmented transfer	Not support any DMA related transfers
I/O mapping	Via HP GPIO matrix or HP IO MUX	Via HP GPIO matrix	Via LP GPIO matrix or LP IO MUX
CS line amount	6	3	1
Unaligned byte transfer	Support	Support	Not support
DTR feature	Support	Not support	Not support

cont'd on next page

Table 43.10-1 – Cont'd from previous page

Feature	GP-SPI2	GP-SPI3	LP-SPI
Wake-up feature when working as a slave	Not support	Not support	Support
Timing compensation	Support	Support	Not support

43.10.2 Register Differences

Table 43.10-2 lists the registers and fields that differs in GP-SPI2, GP-SPI3, and LP-SPI.

Table 43.10-2. Register Differences

Register	Field	GP-SPI2	GP-SPI3	LP-SPI
CMD_REG	CONF_BITLEN	Y	—	—
USER_REG	OPI_MODE	Y	Y	—
	FWRITE_DUAL	Y	Y	—
	FWRITE_DUAD	Y	Y	—
	FWRITE_OCT	Y	—	—
	USR_CONF_NEXT	Y	—	—
CTRL_REG	FADDR_DUAL	Y	Y	—
	FADDR_QUAD	Y	Y	—
	FADDR_OCT	Y	—	—
	FCMD_DUAL	Y	Y	—
	FCMD_QUAD	Y	Y	—
	FCMD_OCT	Y	—	—
	FREAD_DUAL	Y	Y	—
	FREAD_QUAD	Y	Y	—
	FREAD_OCT	Y	—	—
	HOLD_POL	Y	Y	—
	WP_POL	Y	Y	—
MISC_REG	CS1_DIS	Y	Y	—
	CS2_DIS	Y	Y	—
	CS3_DIS	Y	—	—
	CS4_DIS	Y	—	—
	CS5_DIS	Y	—	—
	CLK_DATA_DTR_EN	Y	—	—
	DATA_DTR_EN	Y	—	—
	ADDR_DTR_EN	Y	—	—
	DQS_IDLE_EDGE	Y	—	—
	QUAD_DIN_PIN_SWAP	Y	—	—

Cont'd on next page

Table 43.10-2 – cont'd from previous page

Register	Field	GP-SPI2	GP-SPI3	LP-SPI
DMA_CONF_REG	DMA_OUTFIFO_EMPTY	Y	Y	—
	DMA_INFIFO_FULL	Y	Y	—
	DMA_SLV_SEG_TRANS_EN	Y	Y	—
	SLV_RX_SEG_TRANS_CLR_EN	Y	Y	—
	SLV_TX_SEG_TRANS_CLR_EN	Y	Y	—
	RX_EOF_EN	Y	Y	—
	DMA_RX_ENA	Y	Y	—
	DMA_TX_ENA	Y	Y	—
	DMA_AFIFO_RST	Y	Y	—
SLEEP_CONFO_REG	SLV_WK_CHAR0	—	—	Y
	SLV_WK_CHAR_NUM	—	—	Y
	SLV_WK_CHAR_MASK	—	—	Y
	SLV_WK_MODE_SEL	—	—	Y
	SLEEP_EN	—	—	Y
	SLEEP_DIS_RXFIFO_WR_EN	—	—	Y
	SLEEP_WK_DATA_SEL	—	—	Y
SLEEP_CONF1_REG	SLV_WK_CHAR1	—	—	Y
	SLV_WK_CHAR2	—	—	Y
	SLV_WK_CHAR3	—	—	Y
	SLV_WK_CHAR4	—	—	Y
SLAVE_REG	SLV_RDDMA_BITLEN_EN	Y	Y	—
	SLV_WRDMA_BITLEN_EN	Y	Y	—
	SLV_LAST_BYTE_STRB	Y	Y	—
	DMA_SEG_MAGIC_VALUE	Y	—	—
	USR_CONF	Y	—	—
	MST_FD_WAIT_DMA_TX_DATA	Y	Y	—
DIN_MODE_REG	DIN4_MODE	Y	—	—
	DIN5_MODE	Y	—	—
	DIN6_MODE	Y	—	—
	DIN7_MODE	Y	—	—
DIN_NUM_REG	DIN4_NUM	Y	—	—
	DIN5_NUM	Y	—	—
	DIN6_NUM	Y	—	—
	DIN7_NUM	Y	—	—
DOUT_MODE_REG	DOUT4_MODE	Y	—	—
	DOUT5_MODE	Y	—	—
	DOUT6_MODE	Y	—	—
	DOUT7_MODE	Y	—	—
	D_DQS_MODE	Y	—	—

Cont'd on next page

Table 43.10-2 – cont'd from previous page

Register	Field	GP-SPI2	GP-SPI3	LP-SPI
DMA_INT_ENA_REG	DMA_INFIFO_FULL_ERR_INT_ENA	Y	Y	—
	DMA_OUTFIFO_EMPTY_ERR_INT_ENA	Y	Y	—
	SLV_EX_QPI_INT_ENA	Y	Y	—
	SLV_EN_QPI_INT_ENA	Y	Y	—
	SLV_CMD7_INT_ENA	Y	Y	—
	SLV_CMD8_INT_ENA	Y	Y	—
	SLV_CMD9_INT_ENA	Y	Y	—
	SLV_CMDA_INT_ENA	Y	Y	—
	SLV_RD_DMA_DONE_INT_ENA	Y	Y	—
	SLV_WR_DMA_DONE_INT_ENA	Y	Y	—
	LP_SPI_WAKEUP_INT_ENA	—	—	Y
	DMA_SEG_TRANS_DONE_INT_ENA	Y	Y	—
	SPI_SEG_MAGIC_ERR_INT_ENA	Y	—	—
DMA_INT_CLR_REG	DMA_INFIFO_FULL_ERR_INT_CLR	Y	Y	—
	DMA_OUTFIFO_EMPTY_ERR_INT_CLR	Y	Y	—
	SLV_EX_QPI_INT_CLR	Y	Y	—
	SLV_EN_QPI_INT_CLR	Y	Y	—
	SLV_CMD7_INT_CLR	Y	Y	—
	SLV_CMD8_INT_CLR	Y	Y	—
	SLV_CMD9_INT_CLR	Y	Y	—
	SLV_CMDA_INT_CLR	Y	Y	—
	SLV_RD_DMA_DONE_INT_CLR	Y	Y	—
	SLV_WR_DMA_DONE_INT_CLR	Y	Y	—
	LP_SPI_WAKEUP_INT_CLR	—	—	Y
	DMA_SEG_TRANS_DONE_INT_CLR	Y	Y	—
	SPI_SEG_MAGIC_ERR_INT_CLR	Y	—	—
DMA_INT_RAW_REG	DMA_INFIFO_FULL_ERR_INT_RAW	Y	Y	—
	DMA_OUTFIFO_EMPTY_ERR_INT_RAW	Y	Y	—
	SLV_EX_QPI_INT_RAW	Y	Y	—
	SLV_EN_QPI_INT_RAW	Y	Y	—
	SLV_CMD7_INT_RAW	Y	Y	—
	SLV_CMD8_INT_RAW	Y	Y	—
	SLV_CMD9_INT_RAW	Y	Y	—
	SLV_CMDA_INT_RAW	Y	Y	—
	SLV_RD_DMA_DONE_INT_RAW	Y	Y	—
	SLV_WR_DMA_DONE_INT_RAW	Y	Y	—
	LP_SPI_WAKEUP_INT_RAW	—	—	Y
	DMA_SEG_TRANS_DONE_INT_RAW	Y	Y	—
	SPI_SEG_MAGIC_ERR_INT_RAW	Y	—	—

Cont'd on next page

Table 43.10-2 – cont'd from previous page

Register	Field	GP-SPI2	GP-SPI3	LP-SPI
DMA_INT_ST_REG	DMA_INFIFO_FULL_ERR_INT_ST	Y	Y	—
	DMA_OUTFIFO_EMPTY_ERR_INT_ST	Y	Y	—
	SLV_EX_QPI_INT_ST	Y	Y	—
	SLV_EN_QPI_INT_ST	Y	Y	—
	SLV_CMD7_INT_ST	Y	Y	—
	SLV_CMD8_INT_ST	Y	Y	—
	SLV_CMD9_INT_ST	Y	Y	—
	SLV_CMDA_INT_ST	Y	Y	—
	SLV_RD_DMA_DONE_INT_ST	Y	Y	—
	SLV_WR_DMA_DONE_INT_ST	Y	Y	—
	LP_SPI_WAKEUP_INT_ST	—	—	Y
	DMA_SEG_TRANS_DONE_INT_ST	Y	Y	—
	SPI_SEG_MAGIC_ERR_INT_ST	Y	—	—
DMA_INT_SET_REG	DMA_INFIFO_FULL_ERR_INT_SET	Y	Y	—
	DMA_OUTFIFO_EMPTY_ERR_INT_SET	Y	Y	—
	SLV_EX_QPI_INT_SET	Y	Y	—
	SLV_EN_QPI_INT_SET	Y	Y	—
	SLV_CMD7_INT_SET	Y	Y	—
	SLV_CMD8_INT_SET	Y	Y	—
	SLV_CMD9_INT_SET	Y	Y	—
	SLV_CMDA_INT_SET	Y	Y	—
	SLV_RD_DMA_DONE_INT_SET	Y	Y	—
	SLV_WR_DMA_DONE_INT_SET	Y	Y	—
	LP_SPI_WAKEUP_INT_SET	—	—	Y
	DMA_SEG_TRANS_DONE_INT_SET	Y	Y	—
	SPI_SEG_MAGIC_ERR_INT_SET	Y	—	—

43.10.3 Interrupt Differences

Table 43.10-3 lists the interrupts supported in GP-SPI2, GP-SPI3, and LP-SPI.

Table 43.10-3. Interrupt Differences

Interrupt	GP-SPI2	GP-SPI3	LP-SPI
SPI_DMA_INFIFO_FULL_ERR_INT	Y	Y	—
SPI_DMA_OUTFIFO_EMPTY_ERR_INT	Y	Y	—
SPI_SLV_EX_QPI_INT	Y	Y	—
SPI_SLV_EN_QPI_INT	Y	Y	—
SPI_SLV_CMD7_INT	Y	Y	—
SPI_SLV_CMD8_INT	Y	Y	—
SPI_SLV_CMD9_INT	Y	Y	—
SPI_SLV_CMDA_INT	Y	Y	—

Table 43.10-3. Interrupt Differences

Interrupt	GP-SPI2	GP-SPI3	LP-SPI
SPI_SLV_RD_DMA_DONE_INT	Y	Y	—
SPI_SLV_WR_DMA_DONE_INT	Y	Y	—
SPI_SLV_RD_BUF_DONE_INT	Y	Y	Y
SPI_SLV_WR_BUF_DONE_INT	Y	Y	Y
SPI_TRANS_DONE_INT	Y	Y	Y
SPI_DMA_SEG_TRANS_DONE_INT	Y	Y	—
SPI_SEG_MAGIC_ERR_INT	Y	—	—
SPI_MST_RX_AFIFO_WFULL_ERR_INT	Y	Y	Y
SPI_MST_TX_AFIFO_REMPTY_ERR_INT	Y	Y	Y
SPI_SLV_CMD_ERR_INT	Y	Y	Y
SPI_APP2_INT	Y	Y	Y
SPI_APP1_INT	Y	Y	Y
LP_SPI_WAKEUP_INT	—	—	Y

43.11 Interrupt

43.11.1 Interrupt Description

ESP32-P4 LP-SPI and GP-SPI can generate the following external interrupt signals:

- LP-SPI: `LP_SPI_INTR`
- GP-SPI2: `SPI_INTR_2`
- GP-SPI3: `SPI_INTR_3`

`LP_SPI_INTR` is directly mapped to LP CPU (see [Low-Power CPU](#)), and `SPI_INTR_2/SPI_INTR_3` can be mapped to HP CPU0/1 through the [Interrupt Matrix](#). These interrupt signals are generated by internal interrupt sources of LP-SPI and GP-SPI. Table 43.11-1 lists the internal interrupt sources, interrupt trigger conditions, and generated interrupt signals of LP-SPI and GP-SPI.

Table 43.11-1. Internal Interrupt Sources of LP-SPI and GP-SPI

Internal Interrupt Source	Trigger Condition	Interrupt Signal
SPI_DMA_INFIFO_FULL_ERR_INT	The length of DMA RX FIFO is shorter than that of actual data transferred	<code>SPI_INTR_2/SPI_INTR_3</code>
SPI_DMA_OUTFIFO_EMPTY_ERR_INT	The length of DMA TX FIFO is shorter than that of actual data transferred	<code>SPI_INTR_2/SPI_INTR_3</code>
SPI_SLV_EX_QPI_INT	<code>Ex_QPI</code> is received correctly in GP-SPI2 as slave and the SPI transfer ends	<code>SPI_INTR_2/SPI_INTR_3</code>
SPI_SLV_EN_QPI_INT	<code>En_QPI</code> is received correctly in GP-SPI2 as slave and the SPI transfer ends	<code>SPI_INTR_2/SPI_INTR_3</code>

Cont'd on next page

Table 43.11-1 – cont'd from previous page

Internal Interrupt Source	Trigger Condition	Interrupt Signal
SPI_SLV_CMD7_INT	CMD7 is received correctly in GP-SPI2 as slave and the SPI transfer ends	SPI_INTR_2/SPI_INTR_3
SPI_SLV_CMD8_INT	CMD8 is received correctly in GP-SPI2 as slave and the SPI transfer ends	SPI_INTR_2/SPI_INTR_3
SPI_SLV_CMD9_INT	CMD9 is received correctly in GP-SPI2 as slave and the SPI transfer ends	SPI_INTR_2/SPI_INTR_3
SPI_SLV_CMDA_INT	CMDA is received correctly in GP-SPI2 as slave and the SPI transfer ends	SPI_INTR_2/SPI_INTR_3
SPI_SLV_RD_DMA_DONE_INT	At the end of Rd_DMA transfer as slave	SPI_INTR_2/SPI_INTR_3
SPI_SLV_WR_DMA_DONE_INT	At the end of Wr_DMA transfer as slave	SPI_INTR_2/SPI_INTR_3
SPI_SLV_RD_BUF_DONE_INT	At the end of Rd_BUF transfer as slave	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_SLV_WR_BUF_DONE_INT	At the end of Wr_BUF transfer as slave	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_TRANS_DONE_INT	At the end of SPI bus transfer in both as master and as slave.	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_DMA_SEG_TRANS_DONE_INT	At the end of End_SEG_TRANS transfer in GP-SPI slave segmented transfer mode or at the end of configurable segmented transfer as master	SPI_INTR_2/SPI_INTR_3
SPI_SEG_MAGIC_ERR_INT	A Magic error occurs in CONF buffer during configurable segmented transfer as master	SPI_INTR_2
SPI_MST_RX_AFIFO_WFULL_ERR_INT	RX AFIFO write-full error in GP-SPI as master	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_MST_TX_AFIFO_EMPTY_ERR_INT	TX AFIFO read-empty error in GP-SPI as master	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_SLV_CMD_ERR_INT	A received command value is not supported in GP-SPI as slave	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_APP2_INT	Set SPI_APP2_INT_SET (Only used for user defined function)	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3
SPI_APP1_INT	Set SPI_APP1_INT_SET (Only used for user defined function)	LP_SPI_INTR/SPI_INTR_2/ SPI_INTR_3

Cont'd on next page

Table 43.11-1 – cont'd from previous page

Internal Interrupt Source	Trigger Condition	Interrupt Signal
LP_SPI_WAKEUP_INT	Pre-defined chars are received and wake_up signal is generated	LP_SPI_INTR

LP-SPI is a simplified version of GP-SPI, which contains only some GP-SPI interrupt sources. Each interrupt source has a common set of configuration registers, see the *Interrupt Registers* in Section [Register Summary](#).

43.11.2 Interrupts Used in Master and in Slave (Take GP-SPI as an Example)

Table 43.11-2 and Table 43.11-3 show the interrupts used in GP-SPI master and slave modes. Set the interrupt enable bit SPI_*_INT_ENA in [SPI_DMA_INT_ENA_REG](#) and wait for the interrupt. When the transfer ends, the related interrupt is triggered and should be cleared by software before the next transfer.

Table 43.11-2. GP-SPI Master Mode Interrupts

Transfer Type	Communication Mode	Controlled by	Interrupt
Single Transfer	Full-duplex	DMA	AXI_DMA_IN_SUC_EOF_CH_n_INT ¹
		CPU	SPI_TRANS_DONE_INT ²
	Half-duplex MOSI Mode	DMA	SPI_TRANS_DONE_INT
		CPU	SPI_TRANS_DONE_INT
	Half-duplex MISO Mode	DMA	AXI_DMA_IN_SUC_EOF_CH_n_INT
		CPU	SPI_TRANS_DONE_INT
Configurable Segmented Transfer	Full-duplex	DMA	SPI_DMA_SEG_TRANS_DONE_INT ³
		CPU	Not supported
	Half-duplex MOSI Mode	DMA	SPI_DMA_SEG_TRANS_DONE_INT
		CPU	Not supported
	Half-duplex MISO	DMA	SPI_DMA_SEG_TRANS_DONE_INT
		CPU	Not supported

¹ If [AXI_DMA_IN_SUC_EOF_CH_n_INT](#) is triggered, it means all the RX data of GP-SPI has been stored in the RX buffer, and the TX data has been transferred to the slave.

² [SPI_TRANS_DONE_INT](#) is triggered when CS is high, which indicates that master has completed the data exchange in [SPI_WO_REG~SPI_W15_REG](#) with slave in this mode.

³ If [SPI_DMA_SEG_TRANS_DONE_INT](#) is triggered, it means that the whole configurable segmented transfer (consisting of several segments) has finished, i.e., the RX data has been stored in the RX buffer completely and all the TX data has been sent out.

Table 43.11-3. GP-SPI Slave Mode Interrupts

Transfer Type	Communication Mode	Controlled by	Interrupt
Single Transfer	Full-duplex	DMA	AXI_DMA_IN_SUC_EOF_CHn_INT ¹
		CPU	SPI_TRANS_DONE_INT ²
	Half-duplex MOSI Mode	DMA (Wr_DMA)	AXI_DMA_IN_SUC_EOF_CHn_INT ³
		CPU (Wr_BUF)	SPI_TRANS_DONE_INT ⁴
	Half-duplex MISO Mode	DMA (Rd_DMA)	SPI_TRANS_DONE_INT ⁵
		CPU (Rd_BUF)	SPI_TRANS_DONE_INT ⁶
Slave Segmented Transfer	Full-duplex	DMA	AXI_DMA_IN_SUC_EOF_CHn_INT ⁷
		CPU	Not supported ⁸
	Half-duplex MOSI Mode	DMA (Wr_DMA)	SPI_DMA_SEG_TRANS_DONE_INT ⁹
		CPU (Wr_BUF)	Not supported ¹⁰
	Half-duplex MISO Mode	DMA (Rd_DMA)	SPI_DMA_SEG_TRANS_DONE_INT ¹¹
		CPU (Rd_BUF)	Not supported ¹²

¹ If [AXI_DMA_IN_SUC_EOF_CHn_INT](#) is triggered, it means all the RX data has been stored in the RX buffer, and the TX data has been sent to the slave.

² [SPI_TRANS_DONE_INT](#) is triggered when CS is high, which indicates that master has completed the data exchange in [SPI_WO_REG~SPI_W15_REG](#) with slave in this mode.

³ [SPI_SLV_WR_DMA_DONE_INT](#) just means that the transmission on the SPI bus is done, but can not ensure that all the push data has been stored in the RX buffer. For this reason, [AXI_DMA_IN_SUC_EOF_CHn_INT](#) is recommended.

⁴ Or wait for [SPI_SLV_WR_BUF_DONE_INT](#).

⁵ Or wait for [SPI_SLV_RD_DMA_DONE_INT](#).

⁶ Or wait for [SPI_SLV_RD_BUF_DONE_INT](#).

⁷ Slave should set the total read data byte length in [SPI_MS_DATA_BITLEN](#) before the transfer begins. Set [SPI_RX_EOF_EN](#) to 1 before the end of the interrupt program.

⁸ Master and slave should define a method to end the segmented transfer, such as via GPIO interrupt.

⁹ Master sends End_SEG_TRAN to end the segmented transfer or slave sets the total read data byte length in [SPI_MS_DATA_BITLEN](#) and waits for [AXI_DMA_IN_SUC_EOF_CHn_INT](#).

¹⁰ Half-duplex Wr_BUF single transfer can be used in a slave segmented transfer.

¹¹ Master sends End_SEG_TRAN to end the segmented transfer.

¹² Half-duplex Rd_BUF single transfer can be used in a slave segmented transfer.

43.12 Register Summary

43.12.1 GP-SPI2 Register Summary

The addresses in this section are relative to GP-SPI2 base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column Access are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
User-defined control registers			

Name	Description	Address	Access
SPI_CMD_REG	Command control register	0x0000	varies
SPI_ADDR_REG	Address value register	0x0004	R/W
SPI_USER_REG	SPI USER control register	0x0010	varies
SPI_USER1_REG	SPI USER control register 1	0x0014	R/W
SPI_USER2_REG	SPI USER control register 2	0x0018	R/W
Control and configuration registers			
SPI_CTRL_REG	SPI control register	0x0008	R/W
SPI_MS_DLEN_REG	SPI data bit length control register	0x001C	R/W
SPI_MISC_REG	SPI misc register	0x0020	R/W
SPI_DMA_CONF_REG	SPI DMA control register	0x0030	varies
SPI_SLAVE_REG	SPI slave control register	0x00E0	varies
SPI_SLAVE1_REG	SPI slave control register 1	0x00E4	R/W/SS
Clock control registers			
SPI_CLOCK_REG	SPI clock control register	0x000C	R/W
SPI_CLK_GATE_REG	SPI module clock and register clock control	0x00E8	R/W
Timing registers			
SPI_DIN_MODE_REG	SPI input delay mode configuration	0x0024	R/W
SPI_DIN_NUM_REG	SPI input delay number configuration	0x0028	R/W
SPI_DOUT_MODE_REG	SPI output delay mode configuration	0x002C	R/W
Interrupt registers			
SPI_DMA_INT_ENA_REG	SPI interrupt enable register	0x0034	R/W
SPI_DMA_INT_CLR_REG	SPI interrupt clear register	0x0038	WT
SPI_DMA_INT_RAW_REG	SPI interrupt raw register	0x003C	R/WTC/SS
SPI_DMA_INT_ST_REG	SPI interrupt status register	0x0040	RO
SPI_DMA_INT_SET_REG	SPI interrupt software set register	0x0044	WT
CPU-controlled data buffer			
SPI_W0_REG	SPI CPU-controlled buffer 0	0x0098	R/W/SS
SPI_W1_REG	SPI CPU-controlled buffer 1	0x009C	R/W/SS
SPI_W2_REG	SPI CPU-controlled buffer 2	0x00A0	R/W/SS
SPI_W3_REG	SPI CPU-controlled buffer 3	0x00A4	R/W/SS
SPI_W4_REG	SPI CPU-controlled buffer 4	0x00A8	R/W/SS
SPI_W5_REG	SPI CPU-controlled buffer 5	0x00AC	R/W/SS
SPI_W6_REG	SPI CPU-controlled buffer 6	0x00B0	R/W/SS
SPI_W7_REG	SPI CPU-controlled buffer 7	0x00B4	R/W/SS
SPI_W8_REG	SPI CPU-controlled buffer 8	0x00B8	R/W/SS
SPI_W9_REG	SPI CPU-controlled buffer 9	0x00BC	R/W/SS
SPI_W10_REG	SPI CPU-controlled buffer 10	0x00C0	R/W/SS
SPI_W11_REG	SPI CPU-controlled buffer 11	0x00C4	R/W/SS
SPI_W12_REG	SPI CPU-controlled buffer 12	0x00C8	R/W/SS
SPI_W13_REG	SPI CPU-controlled buffer 13	0x00CC	R/W/SS
SPI_W14_REG	SPI CPU-controlled buffer 14	0x00D0	R/W/SS
SPI_W15_REG	SPI CPU-controlled buffer 15	0x00D4	R/W/SS
Version register			

Name	Description	Address	Access
SPI_DATE_REG	Version control	0x00F0	R/W

43.12.2 GP-SPI3 Register Summary

The addresses in this section are relative to GP-SPI3 base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column Access are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
User-defined control registers			
SPI_CMD_REG	Command control register	0x0000	varies
SPI_ADDR_REG	Address value register	0x0004	R/W
SPI_USER_REG	SPI USER control register	0x0010	varies
SPI_USER1_REG	SPI USER control register 1	0x0014	R/W
SPI_USER2_REG	SPI USER control register 2	0x0018	R/W
Control and configuration registers			
SPI_CTRL_REG	SPI control register	0x0008	R/W
SPI_MS_DLEN_REG	SPI data bit length control register	0x001C	R/W
SPI_MISC_REG	SPI misc register	0x0020	R/W
SPI_DMA_CONF_REG	SPI DMA control register	0x0030	varies
SPI_SLAVE_REG	SPI slave control register	0x00E0	varies
SPI_SLAVE1_REG	SPI slave control register 1	0x00E4	R/W/SS
Clock control registers			
SPI_CLOCK_REG	SPI clock control register	0x000C	R/W
SPI_CLK_GATE_REG	SPI module clock and register clock control	0x00E8	R/W
Timing registers			
SPI_DIN_MODE_REG	SPI input delay mode configuration	0x0024	R/W
SPI_DIN_NUM_REG	SPI input delay number configuration	0x0028	R/W
SPI_DOUT_MODE_REG	SPI output delay mode configuration	0x002C	R/W
Interrupt registers			
SPI_DMA_INT_ENA_REG	SPI interrupt enable register	0x0034	R/W
SPI_DMA_INT_CLR_REG	SPI interrupt clear register	0x0038	WT
SPI_DMA_INT_RAW_REG	SPI interrupt raw register	0x003C	R/WTC/SS
SPI_DMA_INT_ST_REG	SPI interrupt status register	0x0040	RO
SPI_DMA_INT_SET_REG	SPI interrupt software set register	0x0044	WT
CPU-controlled data buffer			
SPI_W0_REG	SPI CPU-controlled buffer 0	0x0098	R/W/SS
SPI_W1_REG	SPI CPU-controlled buffer 1	0x009C	R/W/SS
SPI_W2_REG	SPI CPU-controlled buffer 2	0x00A0	R/W/SS
SPI_W3_REG	SPI CPU-controlled buffer 3	0x00A4	R/W/SS
SPI_W4_REG	SPI CPU-controlled buffer 4	0x00A8	R/W/SS
SPI_W5_REG	SPI CPU-controlled buffer 5	0x00AC	R/W/SS
SPI_W6_REG	SPI CPU-controlled buffer 6	0x00B0	R/W/SS

Name	Description	Address	Access
SPI_W7_REG	SPI CPU-controlled buffer 7	0x00B4	R/W/SS
SPI_W8_REG	SPI CPU-controlled buffer 8	0x00B8	R/W/SS
SPI_W9_REG	SPI CPU-controlled buffer 9	0x00BC	R/W/SS
SPI_W10_REG	SPI CPU-controlled buffer 10	0x00C0	R/W/SS
SPI_W11_REG	SPI CPU-controlled buffer 11	0x00C4	R/W/SS
SPI_W12_REG	SPI CPU-controlled buffer 12	0x00C8	R/W/SS
SPI_W13_REG	SPI CPU-controlled buffer 13	0x00CC	R/W/SS
SPI_W14_REG	SPI CPU-controlled buffer 14	0x00D0	R/W/SS
SPI_W15_REG	SPI CPU-controlled buffer 15	0x00D4	R/W/SS
Version register			
SPI_DATE_REG	Version control	0x00F0	R/W

43.12.3 LP-SPI Register Summary

The addresses in this section are relative to LP-SPI base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column Access are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
User-defined control registers			
LP_SPI_CMD_REG	Command control register	0x0000	varies
LP_SPI_ADDR_REG	Address value register	0x0004	R/W
LP_SPI_USER_REG	LP-SPI USER control register	0x0010	R/W
LP_SPI_USER1_REG	LP-SPI USER control register 1	0x0014	R/W
LP_SPI_USER2_REG	LP-SPI USER control register 2	0x0018	R/W
Control and configuration registers			
LP_SPI_CTRL_REG	LP-SPI control register	0x0008	R/W
LP_SPI_MS_DLEN_REG	LP-SPI data bit length control register	0x001C	R/W
LP_SPI_MISC_REG	LP-SPI misc register	0x0020	R/W
LP_SPI_DMA_CONF_REG	LP-SPI DMA control register	0x0030	WT
LP_SPI_SLEEP_CONFO_REG	LP-SPI sleep configure register 0	0x0044	R/W
LP_SPI_SLEEP_CONF1_REG	LP-SPI sleep configure register 1	0x0048	R/W
LP_SPI_SLAVE_REG	LP-SPI slave control register	0x00E0	varies
LP_SPI_SLAVE1_REG	LP-SPI slave control register 1	0x00E4	R/W/SS
Clock control registers			
LP_SPI_CLOCK_REG	LP-SPI clock control register	0x000C	R/W
LP_SPI_CLK_GATE_REG	LP-SPI module clock and register clock control	0x00E8	R/W
Timing registers			
LP_SPI_DIN_MODE_REG	LP-SPI input delay mode configuration	0x0024	R/W
LP_SPI_DIN_NUM_REG	LP-SPI input delay number configuration	0x0028	R/W
LP_SPI_DOUT_MODE_REG	LP-SPI output delay mode configuration	0x002C	R/W
Interrupt registers			
LP_SPI_DMA_INT_ENA_REG	LP-SPI interrupt enable register	0x0034	R/W

Name	Description	Address	Access
LP_SPI_DMA_INT_CLR_REG	LP-SPI interrupt clear register	0x0038	WT
LP_SPI_DMA_INT_RAW_REG	LP-SPI interrupt raw register	0x003C	R/WTC/SS
LP_SPI_DMA_INT_ST_REG	LP-SPI interrupt status register	0x0040	RO
LP_SPI_DMA_INT_SET_REG	LP-SPI interrupt software set register	0x004C	WT
CPU-controlled data buffer			
LP_SPI_W0_REG	LP-SPI CPU-controlled buffer 0	0x0098	R/W/SS
LP_SPI_W1_REG	LP-SPI CPU-controlled buffer 1	0x009C	R/W/SS
LP_SPI_W2_REG	LP-SPI CPU-controlled buffer 2	0x00A0	R/W/SS
LP_SPI_W3_REG	LP-SPI CPU-controlled buffer 3	0x00A4	R/W/SS
LP_SPI_W4_REG	LP-SPI CPU-controlled buffer 4	0x00A8	R/W/SS
LP_SPI_W5_REG	LP-SPI CPU-controlled buffer 5	0x00AC	R/W/SS
LP_SPI_W6_REG	LP-SPI CPU-controlled buffer 6	0x00B0	R/W/SS
LP_SPI_W7_REG	LP-SPI CPU-controlled buffer 7	0x00B4	R/W/SS
LP_SPI_W8_REG	LP-SPI CPU-controlled buffer 8	0x00B8	R/W/SS
LP_SPI_W9_REG	LP-SPI CPU-controlled buffer 9	0x00BC	R/W/SS
LP_SPI_W10_REG	LP-SPI CPU-controlled buffer 10	0x00C0	R/W/SS
LP_SPI_W11_REG	LP-SPI CPU-controlled buffer 11	0x00C4	R/W/SS
LP_SPI_W12_REG	LP-SPI CPU-controlled buffer 12	0x00C8	R/W/SS
LP_SPI_W13_REG	LP-SPI CPU-controlled buffer 13	0x00CC	R/W/SS
LP_SPI_W14_REG	LP-SPI CPU-controlled buffer 14	0x00D0	R/W/SS
LP_SPI_W15_REG	LP-SPI CPU-controlled buffer 15	0x00D4	R/W/SS
Version register			
LP_SPI_DATE_REG	Version control	0x00F0	R/W

43.13 Register

43.13.1 GP-SPI2 Register

The addresses in this section are relative to GP-SPI2 base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 43.1. SPI_CMD_REG (0x0000)

(reserved)								SPI_USR SPI_UPDATE				(reserved)					SPI_CONF_BITLEN																	
31								25	24	23	22					18	17	0																
0 0 0 0 0 0 0 0								0	0	0 0 0 0 0 0					0																	Reset		

SPI_CONF_BITLEN Configures the SPI_CLK cycles of SPI CONF state.

Measurement unit: SPI_CLK clock cycle.

Can be configured in CONF state.

(R/W)

SPI_UPDATE Configures whether or not to synchronize SPI registers from APB clock domain into SPI module clock domain.

0: Not synchronize

1: Synchronize

This bit is only used in SPI master transfer.

(WT)

SPI_USR Configures whether or not to enable user-defined command.

0: Not enable

1: Enable

An SPI operation will be triggered when the bit is set. This bit will be cleared once the operation is done. Can not be changed by CONF_buf.

(R/W/SC)

Register 43.2. SPI_ADDR_REG (0x0004)

SPI_USR_ADDR_VALUE																															
31																															0
0																															
Reset																															

SPI_USR_ADDR_VALUE Configures the address to slave. Can be configured in CONF state.

(R/W)

Register 43.3. SPI_USER_REG (0x0010)

SPI_USR_COMMAND																							SPI_USR_ADDR																							SPI_USR_DUMMY																							SPI_USR_MISO																							SPI_USR_MOSI																							SPI_USR_DUMMY_IDLE																							SPI_USR_MOSI_HIGHPART																							SPI_USR_MISO_HIGHPART																							(reserved)																							SPI_SIO																							(reserved)																							SPI_USR_CONF_NXT																							SPI_FWRITE_OCT																							SPI_FWRITE_QUAD																							SPI_FWRITE_DUAL																							(reserved)																							SPI_CLK_OUT_EDGE																							SPI_RSCK_I_EDGE																							SPI_CS_SETUP																							SPI_TSCCK_HOLD																							SPI_OPI_I_EDGE																							SPI_GPI_MODE																							(reserved)																							SPI_DOUTDIN																						
31	30	29	28	27	26	25	24	23						18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					

SPI_DOUTDIN Configures whether or not to enable full-duplex communication.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_QPI_MODE Configures whether or not to enable QPI mode.

0: Disable

1: Enable

This configuration is applicable when the SPI controller works as master or slave. Can be configured in CONF state.

(R/W/SS/SC)

SPI_OPI_MODE Configures whether or not to enable OPI mode.

0: Disable. SPI controller works in other modes.

1: Enable. SPI controller works in OPI mode, all phases of which are in 8-bit mode.

This configuration is only applicable when the SPI controller works as master. Can be configured in CONF state.

(R/W)

SPI_TSCK_I_EDGE Configures whether or not to change the polarity of TSCK in slave transfer.

0: TSCK = SPI_CK_I

```
1: TSCK = !SPI_CK_I
```

(R/W)

SPI_CS_HOLD Configures whether or not to keep SPI CS low when SPI is in DONE state.

0: Not keep low

1: Keep low

Can be configured in CONF state.

(R/W)

SPI_CS_SETUP Configures whether or not to enable SPI CS when SPI is in prepare (PREP) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.3. SPI_USER_REG (0x0010)

Continued from the previous page...

SPI_RSCK_I_EDGE Configures whether or not to change the polarity of RSCK in slave transfer.

0: RSCK = !SPI_CK_I

1: RSCK = SPI_CK_I

(R/W)

SPI_CK_OUT_EDGE Configures SPI clock mode together with SPI_CK_IDLE_EDGE. Can be configured in CONF state. For more information, see Section [43.7.4](#).

(R/W)

SPI_FWRITE_DUAL Configures whether or not to enable the 2-bit mode of read-data phase in write operations.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FWRITE_QUAD Configures whether or not to enable the 4-bit mode of read-data phase in write operations.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FWRITE_OCT Configures whether or not to enable the 8-bit mode of read-data phase in write operations.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_CONF_NXT Configures whether or not to enable the CONF state for the next transaction (segment) in a configurable segmented transfer.

0: This transfer will end after the current transaction (segment) is finished. Or this is not a configurable segmented transfer.

1: This configurable segmented transfer will continue its next transaction (segment).

Can be configured in CONF state.

(R/W)

SPI_SIO Configures whether or not to enable 3-line half-duplex communication, where MOSI and MISO signals share the same pin.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.3. SPI_USER_REG (0x0010)

Continued from the previous page...

SPI_USR_MISO_HIGHPART Configures whether or not to enable High-Part mode in read-data phase, i.e., only access to high-part of the buffers: SPI_W8_REG ~ SPI_W15_REG.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_MOSI_HIGHPART Configures whether or not to enable High-Part mode in write-data phase, i.e., only access to high-part of the buffers: SPI_W8_REG ~ SPI_W15_REG.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_DUMMY_IDLE Configures whether or not to disable SPI clock in DUMMY state.

0: Not disable

1: Disable

Can be configured in CONF state.

(R/W)

SPI_USR_MOSI Configures whether or not to enable the write-data (DOUT) state of an operation.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_MISO Configures whether or not to enable the read-data (DIN) state of an operation.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_DUMMY Configures whether or not to enable the DUMMY state of an operation.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_ADDR Configures whether or not to enable the address (ADDR) state of an operation.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.3. SPI_USER_REG (0x0010)

Continued from the previous page...

SPI_USR_COMMAND Configures whether or not to enable the command (CMD) state of an operation.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

Register 43.4. SPI_USER1_REG (0x0014)

SPI_USR_ADDR_BITLEN																
SPI_CS_HOLD_TIME																
SPI_CS_SETUP_TIME																
SPI_MST_WFULL_ERR_END_EN																
(reserved)																
SPI_USR_DUMMY_CYCLELEN																
31	27	26	22	21	17	16	15	8	7	0						
23		0x1		0		1	0	0	0	0	0	0	0	0	7	Reset

SPI_USR_DUMMY_CYCLELEN Configures the length of DUMMY state.

Measurement unit: SPI_CLK clock cycles.

This value is (the expected cycle number - 1).

Can be configured in CONF state.

(R/W)

SPI_MST_WFULL_ERR_END_EN Configures whether or not to end the SPI transfer when SPI RX AFIFO wfull error occurs in master full-/half-duplex transfers.

0: Not end

1: End

(R/W)

SPI_CS_SETUP_TIME Configures the length of prepare (PREP) state.

Measurement unit: SPI_CLK clock cycles.

This value is equal to the expected cycles - 1. This field is used together with SPI_CS_SETUP.

Can be configured in CONF state.

(R/W)

SPI_CS_HOLD_TIME Configures the delay cycles of CS pin.

Measurement unit: SPI_CLK clock cycles.

This field is used together with SPI_CS_HOLD.

Can be configured in CONF state.

(R/W)

SPI_USR_ADDR_BITLEN Configures the bit length in address state.

This value is (expected bit number - 1).

Can be configured in CONF state.

(R/W)

Register 43.5. SPI_USER2_REG (0x0018)

[illegible]

SPI_USR_COMMAND_VALUE Configures the command value.

Can be configured in CONF state.

(R/W)

SPI_MST_EMPTY_ERR_END_EN Configures whether or not to end the SPI transfer when SPI TX

AFIFO read empty error occurs in master full-/half-duplex transfers.

0: Not end

1: End

(R/W)

SPI_USR_COMMAND_BITLEN Configures the bit length of command state.

This value is (expected bit number - 1).

Can be configured in CONF state.

(R/W)

Register 43.6. SPI_CTRL_REG (0x0008)

[illegible]

SPI_DUMMY_OUT Configures whether or not to output the SPI2 bus signals in DUMMY state.

0: Not output

1: Output

Can be configured in CONF state.

(R/W)

SPI_FADDR_DUAL Configures whether or not to enable 2-bit mode during address (ADDR) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FADDR_QUAD Configures whether or not to enable 4-bit mode during address (ADDR) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FADDR_OCT Configures whether or not to enable 8-bit mode during address (ADDR) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FCMD_DUAL Configures whether or not to enable 2-bit mode during command (CMD) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FCMD_QUAD Configures whether or not to enable 4-bit mode during command (CMD) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.6. SPI_CTRL_REG (0x0008)

Continued from the previous page...

SPI_FCMD_OCT Configures whether or not to enable 8-bit mode during command (CMD) state.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FREAD_DUAL Configures whether or not to enable the 2-bit mode of read-data (DIN) state in read operations.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FREAD_QUAD Configures whether or not to enable the 4-bit mode of read-data (DIN) state in read operations.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_FREAD_OCT Configures whether or not to enable the 8-bit mode of read-data (DIN) state in read operations.

0: Disable

1: Enable

Can be configured in CONF state. (R/W)

SPI_Q_POL Configures MISO line polarity.

0: Low

1: High

Can be configured in CONF state.

(R/W)

SPI_D_POL Configures MOSI line polarity.

0: Low

1: High

Can be configured in CONF state.

(R/W)

SPI_HOLD_POL Configures SPI_HOLD output value when SPI is in idle.

0: Output low

1: Output high

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.6. SPI_CTRL_REG (0x0008)**Continued from the previous page...****SPI_WP_POL** Configures the output value of write-protect signal when SPI is in idle.

0: Output low

1: Output high

Can be configured in CONF state.

(R/W)

SPI_RD_BIT_ORDER Configures the bit order in read-data (MISO) state.

0: MSB first

1: LSB first

2: MSB first

3: LSB first

Can be configured in CONF state. For more information, see Table [43.5-6](#).

(R/W)

SPI_WR_BIT_ORDER Configures the bit order in command (CMD), address (ADDR), and write-data (MOSI) states.

0: MSB first

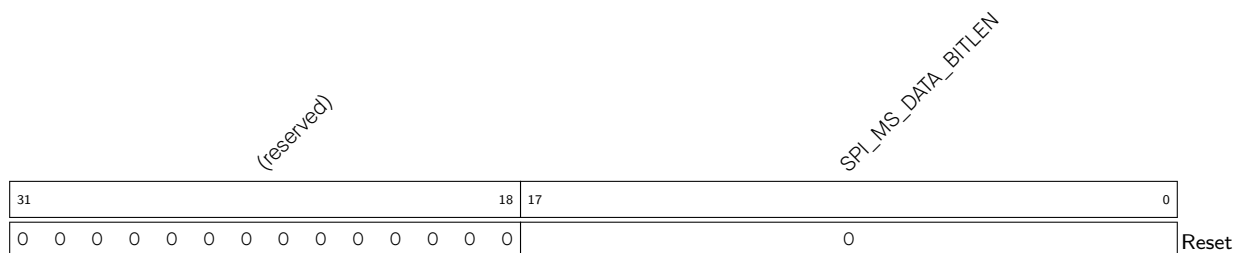
1: LSB first

2: MSB first

3: LSB first

Can be configured in CONF state. For more information, see Table [43.5-6](#).

(R/W)

Register 43.7. SPI_MS_DLEN_REG (0x001C)**SPI_MS_DATA_BITLEN** Configures the data bit length of SPI transfer in DMA-controlled master transfer or in CPU-controlled master transfer. Or configures the bit length of SPI RX transfer in DMA-controlled slave transfer.

This value shall be (expected bit_num - 1).

Can be configured in CONF state.

(R/W)

Register 43.8. SPI_MISC_REG (0x0020)

SPI_QUAD_DIN_PIN_SWAP				SPI_CS_KEEP_ACTIVE				SPI_CLK_IDLE_EDGE				(reserved)				SPI_DQS_IDLE_EDGE				SPI_SLAVE_CS_POL				(reserved)				SPI_CMD_DTR_EN				SPI_ADDR_DTR_EN				SPI_DATA_DTR_EN				SPI_CLK_DATA_DTR_EN				(reserved)				SPI_MASTER_CS_POL				SPI_CLK_DIS				SPI_CS5_DIS				SPI_CS4_DIS				SPI_CS3_DIS				SPI_CS2_DIS				SPI_CS1_DIS				SPI_CS0_DIS			
31	30	29	28	25				24	23	22	20				19	18	17	16	15	13				12	7				6	5	4	3	2	1	0																																												
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				0	1	1	1	1	1	1	0	Reset																																													

Reset

SPI_CS0_DIS Configures whether or not to disable SPI_CS0 pin.

0: SPI_CS0 signal is from/to SPI_CS0 pin.

1: Disable SPI_CS0 pin.

Can be configured in CONF state.

(R/W)

SPI_CS1_DIS Configures whether or not to disable SPI_CS1 pin.

0: SPI_CS1 signal is from/to SPI_CS1 pin.

1: Disable SPI_CS1 pin.

Can be configured in CONF state.

(R/W)

SPI_CS2_DIS Configures whether or not to disable SPI_CS2 pin.

0: SPI_CS2 signal is from/to SPI_CS2 pin.

1: Disable SPI_CS2 pin.

Can be configured in CONF state.

(R/W)

SPI_CS3_DIS Configures whether or not to disable SPI_CS3 pin.

0: SPI_CS3 signal is from/to SPI_CS3 pin.

1: Disable SPI_CS3 pin.

Can be configured in CONF state.

(R/W)

SPI_CS4_DIS Configures whether or not to disable SPI_CS4 pin.

0: SPI_CS4 signal is from/to SPI_CS4 pin.

1: Disable SPI_CS4 pin.

Can be configured in CONF state.

(R/W)

SPI_CS5_DIS Configures whether or not to disable SPI_CS5 pin.

0: SPI_CS5 signal is from/to SPI_CS5 pin.

1: Disable SPI_CS5 pin.

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.8. SPI_MISC_REG (0x0020)

Continued from the previous page...

SPI_CK_DIS Configures whether or not to disable SPI_CLK output.

0: Enable

1: Disable

Can be configured in CONF state.

(R/W)

SPI_MASTER_CS_POL Configures the polarity of SPI_CS n ($n = 0\sim5$) line in master transfer.

0: SPI_CS n is low active.

1: SPI_CS n is high active.

Can be configured in CONF state.

(R/W)

SPI_CLK_DATA_DTR_EN Configures whether or not to enable DTR mode for SPI_CLK, DATA, and SPI_DQS when the SPI works as master.

0: Enable DTR mode only for SPI_DQS

1: Enable DTR mode for SPI_CLK, DATA, and SPI_DQS.

This bit should be used with SPI_DATA_DTR_EN, SPI_ADDR_DTR_EN, and SPI_CMD_DTR_EN.

(R/W)

SPI_DATA_DTR_EN Configures whether or not to enable DTR mode for SPI_CLK and DATA in DOUT and DIN states when the SPI works as master in 1/2/4/8-bit mode.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_ADDR_DTR_EN Configures whether or not to enable DTR mode for SPI_CLK and DATA in ADDR state when the SPI works as master in 1/2/4/8-bit mode.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_CMD_DTR_EN Configures whether or not to enable DTR mode for SPI_CLK and DATA in CMD state when the SPI works as master in 1/2/4/8-bit mode.

0: Not enable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_SLAVE_CS_POL Configures whether or not to invert SPI slave input CS polarity.

0: Not change

1: Invert

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.8. SPI_MISC_REG (0x0020)

Continued from the previous page...

SPI_DQS_IDLE_EDGE Configured the default value of SPI_DQS.

0: Low

1: High

Can be configured in CONF state.

(R/W)

SPI_CK_IDLE_EDGE Configures the level of SPI_CLK line when GP-SPI2 is in idle.

0: Low

1: High

Can be configured in CONF state.

(R/W)

SPI_CS_KEEP_ACTIVE Configures whether or not to keep the SPI_CS line low.

0: Not keep low

1: Keep low

Can be configured in CONF state.

(R/W)

SPI_QUAD_DIN_PIN_SWAP Configures whether or not to swap SPI Quad input pins.

0: Not swap

1: Swap SPI2D with SPI2Q, and SPI2WP with SPI2HD

Can be configured in CONF state.

(R/W)

Register 43.9. SPI_DMA_CONF_REG (0x0030)

SPI_DMA_AFIFO_RST					(reserved)					SPI_RX_EOF_EN					(reserved)					SPI_DMA_INFIFO_FULL				
SPI_BUF_AFIFO_RST					(reserved)					SPI_SLV_TX_SEG_TRANS_CLR_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_RX_AFIFO_RST					(reserved)					SPI_SLV_RX_SEG_TRANS_CLR_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_DMA_TX_ENA					(reserved)					SPI_DMA_SLV_SEG_TRANS_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_DMA_RX_ENA					(reserved)					SPI_DMA_SLV_SEG_TRANS_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
31	30	29	28	27	26						22	21	20	19	18	17						2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
																							1	1

Reset

SPI_DMA_OUTFIFO_EMPTY Represents whether or not the DMA TX FIFO is ready for sending data.

0: Ready
1: Not ready
(RO)

SPI_DMA_INFIFO_FULL Represents whether or not the DMA RX FIFO is ready for receiving data.

0: Ready
1: Not ready
(RO)

SPI_DMA_SLV_SEG_TRANS_EN Configures whether or not to enable DMA-controlled segmented transfer in slave half-duplex communication.

0: Disable
1: Enable
(R/W)

SPI_SLV_RX_SEG_TRANS_CLR_EN In slave segmented transfer, if the size of the DMA RX buffer is smaller than the size of the received data,

1: the data in all the following Wr_DMA transactions will not be received.
0: the data in this Wr_DMA transaction will not be received, but in the following transactions,
- if the size of DMA RX buffer is not 0, the data in following Wr_DMA transactions will be received.
- if the size of DMA RX buffer is 0, the data in following Wr_DMA transactions will not be received.
(R/W)

SPI_SLV_TX_SEG_TRANS_CLR_EN In slave segmented transfer, if the size of the DMA TX buffer is smaller than the size of the transmitted data,

1: the data in the following transactions will not be updated, i.e. the old data is transmitted repeatedly.
0: the data in this transaction will not be updated. But in the following transactions,
- if new data is filled in DMA TX FIFO, new data will be transmitted.
- if no new data is filled in DMA TX FIFO, no new data will be transmitted.
(R/W)

Continued on the next page...

Register 43.9. SPI_DMA_CONF_REG (0x0030)

Continued from the previous page...

SPI_RX_EOF_EN Configures the trigger source of GDMA_IN_SUC_EOF_CH n _INT_RAW.

0: GDMA_IN_SUC_EOF_CH n _INT_RAW is set by SPI_TRANS_DONE_INT event in a single transfer, or by an SPI_DMA_SEG_TRANS_DONE_INT event in a segmented transfer.

1: In a DAM-controlled transfer, if the bit number of transferred data is equal to (SPI_MS_DATA_BITLEN + 1), then GDMA_IN_SUC_EOF_CH n _INT_RAW will be set by hardware.
(R/W)

SPI_DMA_RX_ENA Configures whether or not to enable DMA-controlled receive data transfer.

0: Disable

1: Enable

(R/W)

SPI_DMA_TX_ENA Configures whether or not to enable DMA-controlled send data transfer.

0: Disable

1: Enable

(R/W)

SPI_RX_AFIFO_RST Configures whether or not to reset RX AFIFO (i.e., spi_rx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to receive data in SPI master and slave transfer.

0: Not reset

1: Reset

(WT)

SPI_BUF_AFIFO_RST Configures whether or not to reset BUF TX AFIFO (i.e., buf_tx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to send data out in SPI slave CPU-controlled transfer and master transfer.

0: Not reset

1: Reset

(WT)

SPI_DMA_AFIFO_RST Configures whether or not to reset DMA TX AFIFO (i.e., dma_tx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to send data out in SPI slave DMA-controlled transfer.

0: Not reset

1: Reset

(WT)

[illegible]

- 0: SPI clock is off when CS becomes inactive.
- 1: SPI clock is delayed one cycle after CS becomes inactive.
- 2: SPI clock is delayed two cycles after CS becomes inactive.
- 3: SPI clock is always on.

Can be configured in CONF state.

(R/W)

0: SPI clock mode 1 and mode 3. See Table 43.7-2.
1: SPI clock mode 0 and mode 2. See Table 43.7-2.
(R/W)

0: Output data at TSCK rising edge.
1: Output data at RSCK rising edge.
(R/W)

0: Not use
1: Use
(R/W)

0: Not use
1: Use
(R/W)

0: Not use
1: Use
(R/W)

Continued on the next page...

Register 43.10. SPI_SLAVE_REG (0x00E0)

Continued from the previous page...

SPI_SLV_WRBUF_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Wr_BUF transfer.

0: Not use

1: Use

(R/W)

SPI_SLV_LAST_BYTE_STRB Represents the valid bits of the last received data byte in SPI slave full-duplex and half-duplex transfer.

(R/SS)

SPI_DMA_SEG_MAGIC_VALUE Configures the magic value of BM table in DMA-controlled configurable segmented transfer.

(R/W)

SPI_SLAVE_MODE Configures SPI work mode.

0: Master

1: Slave

(R/W)

SPI_SOFT_RESET Configures whether or not to reset the SPI clock line, CS line, and data line via software.

0: Not reset

1: Reset

Can be configured in CONF state.

(WT)

SPI_USR_CONF Configures whether or not to enable the CONF state of current DMA-controlled configurable segmented transfer.

0: No effect, which means the current transfer is not a configurable segmented transfer.

1: Enable, which means a configurable segmented transfer is started.

(R/W)

SPI_MST_FD_WAIT_DMA_TX_DATA Configures whether or not to wait DMA TX data gets ready before starting SPI transfer in master full-duplex transfer.

0: Not wait

1: Wait

(R/W)

Register 43.11. SPI_SLAVE1_REG (0x00E4)

SPI_SLV_LAST_ADDR										SPI_SLV_LAST_COMMAND										SPI_SLV_DATA_BITLEN										
31	26	25	18	17	0																									
0										0										0										Reset

SPI_SLV_DATA_BITLEN Configures the transferred data bit length in SPI slave full-/half-duplex modes.
(R/W/SS)

SPI_SLV_LAST_COMMAND Configures the command value in slave mode.
(R/W/SS)

SPI_SLV_LAST_ADDR Configures the address value in slave mode.
(R/W/SS)

Register 43.12. SPI_CLOCK_REG (0x000C)

[illegible]

SPI_CLKCNT_L Configures clock duty cycles, together with SPI_CLKCNT_H and SPI_CLKCNT_N.

In master transfer, this field must be equal to SPI_CLKCNT_N.

In slave mode, it must be 0.

Can be configured in CONF state.

(R/W)

SPI_CLKCNT_H Configures the duty cycle of SPI_CLK (high level) in master transfer.

It's recommended to configure this value to $\text{floor}((\text{SPI_CLKCNT_N} + 1)/2 - 1)$. `floor()` here is to round a number down, e.g., $\text{floor}(2.2) = 2$.

In slave mode, it must be 0.

Can be configured in CONF state.

(R/W)

SPI_CLKCNT_N Configures the divider of SPI_CLK in master transfer.

SPI_CLK frequency is $f_{\text{clk_spi_mst}} / (\text{SPI_CLKDIV_PRE} + 1) / (\text{SPI_CLKCNT_N} + 1)$.

Can be configured in CONF state.

(R/W)

SPI_CLKDIV_PRE Configures the pre-divider of SPI_CLK in master transfer.

Can be configured in CONF state.

(R/W)

SPI_CLK_EQU_SYSCLK Configures whether or not the SPI_CLK is equal to clk_spi_mst in master transfer.

0: SPI CLK is divided from clk spi mst.

1: SPI_CLK is equal to clk_spi_mst.

Can be configured in CONF state.

(R/W)

Register 43.13. SPI_CLK_GATE_REG (0x00E8)

(reserved)																															(reserved)		(reserved)		SPI_CLK_EN	
31																															3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

SPI_CLK_EN Configures whether or not to enable clock gate.
0: Disable
1: Enable
(R/W)

Register 43.14. SPI_DIN_MODE_REG (0x0024)

[illegible]

SPI_DINO_MODE Configures the input mode for SPI2D signal.

0: Input without delay

1: Input at the (SPI_DINO_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DINO_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DINO_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_DIN1_MODE Configures the input mode for SPI2Q signal.

0: Input without delay

1: Input at the (SPI_DIN1_NUM+1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_DIN2_MODE Configures the input mode for SPI2WP signal.

0: Input without delay

1: Input at the (SPI_DIN2_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN2_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN2_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.14. SPI_DIN_MODE_REG (0x0024)

Continued from the previous page...

SPI_DIN3_MODE Configures the input mode for SPI2HD signal.

0: Input without delay

1: Input at the (SPI_DIN3_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN3_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN3_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_DIN4_MODE Configures the input mode for SPI2D4 signal.

0: Input without delay

1: Input at the (SPI_DIN4_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN4_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN4_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_DIN5_MODE Configures the input mode for SPI2D5 signal.

0: Input without delay

1: Input at the (SPI_DIN5_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN5_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN5_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_DIN6_MODE Configures the input mode for SPI2D6 signal.

0: Input without delay

1: Input at the (SPI_DIN6_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN6_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN6_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.14. SPI_DIN_MODE_REG (0x0024)

Continued from the previous page...

SPI_DIN7_MODE Configures the input mode for SPI2D7 signal.

0: Input without delay

1: Input at the (SPI_DIN7_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN7_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN7_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

Can be configured in CONF state.

(R/W)

SPI_TIMING_HCLK_ACTIVE Configures whether or not to enable HCLK (high-frequency clock) in SPI input timing module.

0: Disable

1: Enable

Can be configured in CONF state. (R/W)

Register 43.15. SPI_DIN_NUM_REG (0x0028)

(reserved)																SPI_DIN7_NUM		SPI_DIN6_NUM		SPI_DIN5_NUM		SPI_DIN4_NUM		SPI_DIN3_NUM		SPI_DIN2_NUM		SPI_DIN1_NUM		SPI_DINO_NUM		
31																16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset					

SPI_DINO_NUM Configures the delays to input signal SPI2D based on the setting of SPI_DINO_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN1_NUM Configures the delays to input signal SPI2Q based on the setting of SPI_DIN1_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN2_NUM Configures the delays to input signal SPI2WP based on the setting of SPI_DIN2_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN3_NUM Configures the delays to input signal SPI2HD based on the setting of SPI_DIN3_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.15. SPI_DIN_NUM_REG (0x0028)

Continued from the previous page...

SPI_DIN4_NUM Configures the delays to input signal SPI2D4 based on the setting of SPI_DIN4_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN5_NUM Configures the delays to input signal SPI2D5 based on the setting of SPI_DIN5_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN6_NUM Configures the delays to input signal SPI2D6 based on the setting of SPI_DIN6_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

SPI_DIN7_NUM Configures the delays to input signal SPI2D7 based on the setting of SPI_DIN7_MODE.

0: Delayed by 1 clock cycle

1: Delayed by 2 clock cycles

2: Delayed by 3 clock cycles

3: Delayed by 4 clock cycles

Can be configured in CONF state.

(R/W)

Register 43.16. SPI_DOUT_MODE_REG (0x002C)

[illegible]

SPI_DOUT0_MODE Configures the output mode for SPI2D signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI DOUT1 MODE Configures the output mode for SPI2Q signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI_DOUT2_MODE Configures the output mode for SPI2WP signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI_DOUT3_MODE Configures the output mode for SPI2HD signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI_DOUT4_MODE Configures the output mode for SPI2D4 signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI_DOUT5_MODE Configures the output mode for SPI2D5 signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI DOUT6 MODE Configures the output mode for SPI2D6 signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

Continued on the next page...

Register 43.16. SPI_DOUT_MODE_REG (0x002C)

Continued from the previous page...

SPI_DOUT7_MODE Configures the output mode for SPI2D7 signal.

0: Output without delay

1: Output with a delay of a SPI module clock cycle at its falling edge

Can be configured in CONF state.

(R/W)

SPI_D_DQS_MODE Configures the output mode for output SPI_DQS signal.

0: Output without delay

1: Output is delayed for a SPI clock cycle at its falling edge.

Can be configured in CONF state.

(R/W)

Register 43.17. SPI_DMA_INT_ENA_REG (0x0034)

(reserved)												SPI_APP1_INT_ENA SPI_APP2_INT_ENA SPI_MST_TX_INT_ENA SPI_MST_RX_AFIFO_EMPTY_ERR_INT_ENA SPI_SLV_CMD_ERR_INT_ENA SPI_SEG_MAGIC_ERR_INT_ENA SPI_DMA_SEG_TRANS_DONE_INT_ENA SPI_TRANS_DONE_INT_ENA SPI_SLV_WR_BUF_DONE_INT_ENA SPI_SLV_RD_BUF_DONE_INT_ENA SPI_SLV_WR_DMA_DONE_INT_ENA SPI_SLV_RD_DMA_DONE_INT_ENA SPI_SLV_CMD9_INT_ENA SPI_SLV_CMD8_INT_ENA SPI_SLV_CMD7_INT_ENA SPI_SLV_EX_QPI_INT_ENA SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ENA SPI_DMA_INFIFO_FULL_ERR_INT_ENA																					
31												21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset		

SPI_DMA_INFIFO_FULL_ERR_INT_ENA Write 1 to enable SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(R/W)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ENA Write 1 to enable SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(R/W)

SPI_SLV_EX_QPI_INT_ENA Write 1 to enable SPI_SLV_EX_QPI_INT interrupt.
(R/W)

SPI_SLV_EN_QPI_INT_ENA Write 1 to enable SPI_SLV_EN_QPI_INT interrupt.
(R/W)

SPI_SLV_CMD7_INT_ENA Write 1 to enable SPI_SLV_CMD7_INT interrupt.
(R/W)

SPI_SLV_CMD8_INT_ENA Write 1 to enable SPI_SLV_CMD8_INT interrupt.
(R/W)

SPI_SLV_CMD9_INT_ENA Write 1 to enable SPI_SLV_CMD9_INT interrupt.
(R/W)

SPI_SLV_CMDA_INT_ENA Write 1 to enable SPI_SLV_CMDA_INT interrupt.
(R/W)

SPI_SLV_RD_DMA_DONE_INT_ENA Write 1 to enable SPI_SLV_RD_DMA_DONE_INT interrupt.
(R/W)

SPI_SLV_WR_DMA_DONE_INT_ENA Write 1 to enable SPI_SLV_WR_DMA_DONE_INT interrupt.
(R/W)

SPI_SLV_RD_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/W)

SPI_SLV_WR_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_WR_BUF_DONE_INT interrupt.
(R/W)

SPI_TRANS_DONE_INT_ENA Write 1 to enable SPI_TRANS_DONE_INT interrupt.
(R/W)

Continued on the next page...

Register 43.17. SPI_DMA_INT_ENA_REG (0x0034)

Continued from the previous page...

SPI_DMA_SEG_TRANS_DONE_INT_ENA Write 1 to enable SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(R/W)

SPI_SEG_MAGIC_ERR_INT_ENA Write 1 to enable SPI_SEG_MAGIC_ERR_INT interrupt.
(R/W)

SPI_SLV_BUF_ADDR_ERR_INT_ENA Write 1 to enable SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(R/W)

SPI_SLV_CMD_ERR_INT_ENA Write 1 to enable SPI_SLV_CMD_ERR_INT interrupt.
(R/W)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_ENA Write 1 to enable SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(R/W)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ENA Write 1 to enable SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(R/W)

SPI_APP2_INT_ENA Write 1 to enable SPI_APP2_INT interrupt.
(R/W)

SPI_APP1_INT_ENA Write 1 to enable SPI_APP1_INT interrupt.
(R/W)

(reserved)

Reset

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

(WT)

Continued on the next page...

Register 43.18. SPI_DMA_INT_CLR_REG (0x0038)

Continued from the previous page...

SPI_DMA_SEG_TRANS_DONE_INT_CLR Write 1 to clear SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(WT)

SPI_SEG_MAGIC_ERR_INT_CLR Write 1 to clear SPI_SEG_MAGIC_ERR_INT interrupt.
(WT)

SPI_SLV_BUF_ADDR_ERR_INT_CLR Write 1 to clear SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

SPI_SLV_CMD_ERR_INT_CLR Write 1 to clear SPI_SLV_CMD_ERR_INT interrupt.
(WT)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_CLR Write 1 to clear SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_CLR Write 1 to clear SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_APP2_INT_CLR Write 1 to clear SPI_APP2_INT interrupt.
(WT)

SPI_APP1_INT_CLR Write 1 to clear SPI_APP1_INT interrupt.
(WT)

Register 43.19. SPI_DMA_INT_RAW_REG (0x003C)

(reserved)																								SPI_APP1_INT_RAW SPI_APP2_INT_RAW SPI_MST_TX_INT_RAW SPI_MST_RX_INT_RAW SPI_SLV_CMD_EMPTY_ERR_INT_RAW SPI_SLV_CMD_WFULL_ERR_INT_RAW SPI_SEG_BUF_ADDR_ERR_INT_RAW SPI_DMA_SEG_TRANS_DONE_INT_RAW SPI_TRANS_DONE_INT_RAW SPI_SLV_WR_BUF_DONE_INT_RAW SPI_SLV_RD_BUF_DONE_INT_RAW SPI_SLV_RD_DMA_DONE_INT_RAW SPI_SLV_CMD9_INT_RAW SPI_SLV_CMD8_INT_RAW SPI_SLV_CMD7_INT_RAW SPI_SLV_EN_QPI_INT_RAW SPI_DMA_OUTFIFO_EMPTY_ERR_INT_RAW SPI_DMA_INFIFO_FULL_ERR_INT_RAW																		
31																				21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0												

SPI_DMA_INFIFO_FULL_ERR_INT_RAW The raw interrupt status of SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(R/WTC/SS)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_RAW The raw interrupt status of SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(R/WTC/SS)

SPI_SLV_EX_QPI_INT_RAW The raw interrupt status of SPI_SLV_EX_QPI_INT interrupt.
(R/WTC/SS)

SPI_SLV_EN_QPI_INT_RAW The raw interrupt status of SPI_SLV_EN_QPI_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD7_INT_RAW The raw interrupt status of SPI_SLV_CMD7_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD8_INT_RAW The raw interrupt status of SPI_SLV_CMD8_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD9_INT_RAW The raw interrupt status of SPI_SLV_CMD9_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMDA_INT_RAW The raw interrupt status of SPI_SLV_CMDA_INT interrupt.
(R/WTC/SS)

SPI_SLV_RD_DMA_DONE_INT_RAW The raw interrupt status of SPI_SLV_RD_DMA_DONE_INT interrupt.
(R/WTC/SS)

SPI_SLV_WR_DMA_DONE_INT_RAW The raw interrupt status of SPI_SLV_WR_DMA_DONE_INT interrupt.
(R/WTC/SS)

SPI_SLV_RD_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/WTC/SS)

Continued on the next page...

Register 43.19. SPI_DMA_INT_RAW_REG (0x003C)

Continued from the previous page...

SPI_SLV_WR_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.

(R/WTC/SS)

SPI_TRANS_DONE_INT_RAW The raw interrupt status of SPI_TRANS_DONE_INT interrupt.

(R/WTC/SS)

SPI_DMA_SEG_TRANS_DONE_INT_RAW The raw interrupt status of SPI_DMA_SEG_TRANS_DONE_INT interrupt.

(R/WTC/SS)

SPI_SEG_MAGIC_ERR_INT_RAW The raw interrupt status of SPI_SEG_MAGIC_ERR_INT interrupt.

(R/WTC/SS)

SPI_SLV_BUF_ADDR_ERR_INT_RAW The raw interrupt status of SPI_SLV_BUF_ADDR_ERR_INT interrupt.

(R/WTC/SS)

SPI_SLV_CMD_ERR_INT_RAW The raw interrupt status of SPI_SLV_CMD_ERR_INT interrupt.

(R/WTC/SS)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_RAW The raw interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.

(R/WTC/SS)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_RAW The raw interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.

(R/WTC/SS)

SPI_APP2_INT_RAW The raw interrupt status of SPI_APP2_INT interrupt. The value is only controlled by the application.

(R/WTC/SS)

SPI_APP1_INT_RAW The raw interrupt status of SPI_APP1_INT interrupt. The value is only controlled by the application.

(R/WTC/SS)

Register 43.20. SPI_DMA_INT_ST_REG (0x0040)

(reserved)												SPI_APP1_INT_ST SPI_APP2_INT_ST SPI_MST_TX_INT_ST SPI_MST_RX_INT_ST SPI_SLV_CMD_EMPTY_ERR_INT_ST SPI_SLV_CMD_WFULL_ERR_INT_ST SPI_SEG_BUF_ADDR_ERR_INT_ST SPI_DMA_SEG_TRANS_DONE_INT_ST SPI_TRANS_DONE_INT_ST SPI_SLV_WR_BUF_DONE_INT_ST SPI_SLV_RD_BUF_DONE_INT_ST SPI_SLV_RD_DMA_DONE_INT_ST SPI_SLV_WR_DMA_DONE_INT_ST SPI_SLV_CMD9_INT_ST SPI_SLV_CMD8_INT_ST SPI_SLV_CMD7_INT_ST SPI_SLV_EN_QPI_INT_ST SPI_SLV_EX_QPI_INT_ST SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ST SPI_DMA_INFIFO_FULL_ERR_INT_ST											
31	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SPI_DMA_INFIFO_FULL_ERR_INT_ST The interrupt status of SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(RO)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ST The interrupt status of SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(RO)

SPI_SLV_EX_QPI_INT_ST The interrupt status of SPI_SLV_EX_QPI_INT interrupt.
(RO)

SPI_SLV_EN_QPI_INT_ST The interrupt status of SPI_SLV_EN_QPI_INT interrupt.
(RO)

SPI_SLV_CMD7_INT_ST The interrupt status of SPI_SLV_CMD7_INT interrupt.
(RO)

SPI_SLV_CMD8_INT_ST The interrupt status of SPI_SLV_CMD8_INT interrupt.
(RO)

SPI_SLV_CMD9_INT_ST The interrupt status of SPI_SLV_CMD9_INT interrupt.
(RO)

SPI_SLV_CMDA_INT_ST The interrupt status of SPI_SLV_CMDA_INT interrupt.
(RO)

SPI_SLV_RD_DMA_DONE_INT_ST The interrupt status of SPI_SLV_RD_DMA_DONE_INT interrupt.
(RO)

SPI_SLV_WR_DMA_DONE_INT_ST The interrupt status of SPI_SLV_WR_DMA_DONE_INT interrupt.
(RO)

SPI_SLV_RD_BUF_DONE_INT_ST The interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(RO)

SPI_SLV_WR_BUF_DONE_INT_ST The interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.
(RO)

Continued on the next page...

Register 43.20. SPI_DMA_INT_ST_REG (0x0040)

Continued from the previous page...

SPI_TRANS_DONE_INT_ST The interrupt status of SPI_TRANS_DONE_INT interrupt.
(RO)

SPI_DMA_SEG_TRANS_DONE_INT_ST The interrupt status of SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(RO)

SPI_SEG_MAGIC_ERR_INT_ST The interrupt status of SPI_SEG_MAGIC_ERR_INT interrupt.
(RO)

SPI_SLV_BUF_ADDR_ERR_INT_ST The status bit for SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(RO)

SPI_SLV_CMD_ERR_INT_ST The interrupt status of SPI_SLV_CMD_ERR_INT interrupt.
(RO)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_ST The interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(RO)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ST The interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(RO)

SPI_APP2_INT_ST The interrupt status of SPI_APP2_INT interrupt.
(RO)

SPI_APP1_INT_ST The interrupt status of SPI_APP1_INT interrupt.
(RO)

Register 43.21. SPI_DMA_INT_SET_REG (0x0044)

(reserved)										SPI_APP1_INT_SET										SPI_APP2_INT_SET										SPI_MST_TX_INT_SET										SPI_MST_RX_AFIFO_EMPTY_ERR_INT_SET										SPI_SLV_CMD_ERR_INT_SET										SPI_SLV_BUF_ADDR_ERR_INT_SET										SPI_DMA_SEG_MAGIC_ERR_INT_SET										SPI_TRANS_DONE_INT_SET										SPI_SLV_WR_BUF_DONE_INT_SET										SPI_SLV_RD_BUF_DONE_INT_SET										SPI_SLV_WR_DMA_DONE_INT_SET										SPI_SLV_RD_DMA_DONE_INT_SET										SPI_SLV_CMD9_INT_SET										SPI_SLV_CMD8_INT_SET										SPI_SLV_CMD7_INT_SET										SPI_SLV_EN_QPI_INT_SET										SPI_SLV_EX_QPI_INT_SET										SPI_DMA_OUTFIFO_EMPTY_ERR_INT_SET										SPI_DMA_INFIFO_FULL_ERR_INT_SET																																																																																																																																																																																																																																																																																																																																	
31										21										20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
0										0										0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SPI_DMA_INFIFO_FULL_ERR_INT_SET Write 1 to set SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(WT)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_SET Write 1 to set SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_SLV_EX_QPI_INT_SET Write 1 to set SPI_SLV_EX_QPI_INT interrupt.
(WT)

SPI_SLV_EN_QPI_INT_SET Write 1 to set SPI_SLV_EN_QPI_INT interrupt.
(WT)

SPI_SLV_CMD7_INT_SET Write 1 to set SPI_SLV_CMD7_INT interrupt.
(WT)

SPI_SLV_CMD8_INT_SET Write 1 to set SPI_SLV_CMD8_INT interrupt.
(WT)

SPI_SLV_CMD9_INT_SET Write 1 to set SPI_SLV_CMD9_INT interrupt.
(WT)

SPI_SLV_CMDA_INT_SET Write 1 to set SPI_SLV_CMDA_INT interrupt.
(WT)

SPI_SLV_RD_DMA_DONE_INT_SET Write 1 to set SPI_SLV_RD_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_WR_DMA_DONE_INT_SET Write 1 to set SPI_SLV_WR_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_RD_BUF_DONE_INT_SET Write 1 to set SPI_SLV_RD_BUF_DONE_INT interrupt.
(WT)

SPI_SLV_WR_BUF_DONE_INT_SET Write 1 to set SPI_SLV_WR_BUF_DONE_INT interrupt.
(WT)

SPI_TRANS_DONE_INT_SET Write 1 to set SPI_TRANS_DONE_INT interrupt.
(WT)

Continued on the next page...

Register 43.21. SPI_DMA_INT_SET_REG (0x0044)

Continued from the previous page...

SPI_DMA_SEG_TRANS_DONE_INT_SET Write 1 to set SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(WT)

SPI_SEG_MAGIC_ERR_INT_SET Write 1 to set SPI_SEG_MAGIC_ERR_INT interrupt.
(WT)

SPI_SLV_BUF_ADDR_ERR_INT_SET Write 1 to set SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

SPI_SLV_CMD_ERR_INT_SET Write 1 to set SPI_SLV_CMD_ERR_INT interrupt.
(WT)

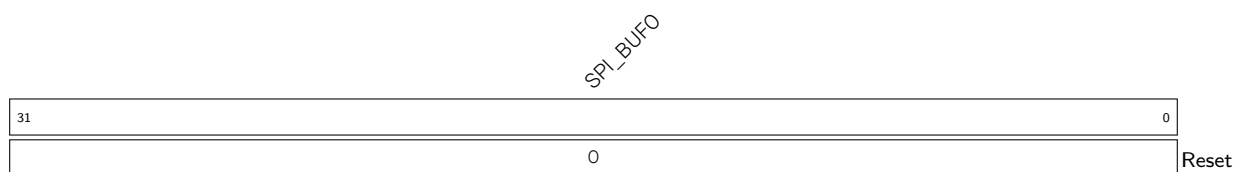
SPI_MST_RX_AFIFO_WFULL_ERR_INT_SET Write 1 to set SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_SET Write 1 to set SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_APP2_INT_SET Write 1 to set SPI_APP2_INT interrupt.
(WT)

SPI_APP1_INT_SET Write 1 to set SPI_APP1_INT interrupt.
(WT)

Register 43.22. SPI_WO_REG (0x0098)



SPI_BUFO 32-bit data buffer 0.
(R/W/SS)

Register 43.23. SPI_W1_REG (0x009C)

SPI_BUF1	
31	0
0	
Reset	

SPI_BUF1 32-bit data buffer 1.
(R/W/SS)

Register 43.24. SPI_W2_REG (0x00A0)

SPI_BUF2	
31	0
0	
Reset	

SPI_BUF2 32-bit data buffer 2.
(R/W/SS)

Register 43.25. SPI_W3_REG (0x00A4)

SPI_BUF3	
31	0
0	
Reset	

SPI_BUF3 32-bit data buffer 3.
(R/W/SS)

Register 43.26. SPI_W4_REG (0x00A8)

SPI_BUF4	
31	0
0	
Reset	

SPI_BUF4 32-bit data buffer 4.
(R/W/SS)

Register 43.27. SPI_W5_REG (0x00AC)

SPI_BUF5	
31	0
0	
Reset	

SPI_BUF5 32-bit data buffer 5.
(R/W/SS)

Register 43.28. SPI_W6_REG (0x00B0)

SPI_BUF6	
31	0
0	
Reset	

SPI_BUF6 32-bit data buffer 6.
(R/W/SS)

Register 43.29. SPI_W7_REG (0x00B4)

SPI_BUF7	
31	0
0	
Reset	

SPI_BUF7 32-bit data buffer 7.
(R/W/SS)

Register 43.30. SPI_W8_REG (0x00B8)

SPI_BUF8	
31	0
0	
Reset	

SPI_BUF8 32-bit data buffer 8.
(R/W/SS)

Register 43.31. SPI_W9_REG (0x00BC)

SPI_BUF9	
31	0
0	
Reset	

SPI_BUF9 32-bit data buffer 9.
(R/W/SS)

Register 43.32. SPI_W10_REG (0x00C0)

SPI_BUF10	
31	0
0	
Reset	

SPI_BUF10 32-bit data buffer 10.
(R/W/SS)

Register 43.33. SPI_W11_REG (0x00C4)

SPI_BUF11	
31	0
0	
Reset	

SPI_BUF11 32-bit data buffer 11.
(R/W/SS)

Register 43.34. SPI_W12_REG (0x00C8)

SPI_BUF12	
31	0
0	
Reset	

SPI_BUF12 32-bit data buffer 12.
(R/W/SS)

Register 43.35. SPI_W13_REG (0x00CC)

SPI_BUF13	
31	0
0	
Reset	

SPI_BUF13 32-bit data buffer 13.
(R/W/SS)

Register 43.36. SPI_W14_REG (0x00D0)

SPI_BUF14	
31	0
0	
Reset	

SPI_BUF14 32-bit data buffer 14.
(R/W/SS)

Register 43.37. SPI_W15_REG (0x00D4)

SPI_BUF15	
31	0
0	
Reset	

SPI_BUF15 32-bit data buffer 15.
(R/W/SS)

Register 43.38. SPI_DATE_REG (0x00F0)

(reserved)		SPI_DATE	
31	28	27	0
0	0	0	0
		0x2207202	
		Reset	

SPI_DATE Version control register.
(R/W)

43.13.2 GP-SPI3 Register

The addresses in this section are relative to GP-SPI3 base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 43.39. SPI_CMD_REG (0x0000)

(reserved)							SPI_USR SPI_UPDATE		(reserved)																																		
31							25							24	23	22																										0	
0							0							0	0	0																										0	Reset

SPI_UPDATE Configures whether or not to synchronize SPI registers from APB clock domain into SPI module clock domain.
0: Not synchronize
1: Synchronize
This bit is only used in SPI master transfer.
(WT)

SPI_USR Configures whether or not to enable user-defined command.
0: Not enable
1: Enable
An SPI operation will be triggered when the bit is set. This bit will be cleared once the operation is done.
(R/W/SC)

Register 43.40. SPI_ADDR_REG (0x0004)

SPI_USR_ADDR_VALUE																															
31																															0
0																															
Reset																															

SPI_USR_ADDR_VALUE Configures the address to slave.
(R/W)

Register 43.41. SPI_USER_REG (0x0010)

SPI_USR_COMMAND																										SPI_USR_ADDR																										SPI_USR_DUMMY																										SPI_USR_MISO																										SPI_USR_MOSI																										SPI_USR_DUMMY_IDLE																										SPI_USR_MOSI_HIGHPART																										SPI_USR_MISO_HIGHPART																										(reserved)																										SPI_SIO																										(reserved)																										SPI_FWRITE_QUAD																										SPI_FWRITE_DUAL																										(reserved)																										SPI_CLK_OUT_EDGE																										SPI_RSCK_I_EDGE																										SPI_CS_SETUP																										SPI_CS_HOLD																										(reserved)																										SPI_GPI_MODE																										(reserved)																										SPI_DOUTDIN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
31	30	29	28	27	26	25	24	23											18	17	16											14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SPI_DOUTDIN Configures whether or not to enable full-duplex communication.

0: Disable

1: Enable

(R/W)

SPI_QPI_MODE Configures whether or not to enable QPI mode.

0: Disable

1: Enable

This configuration is applicable when the SPI controller works as master or slave.

(R/W/SS/SC)

SPI_TSCK_I_EDGE Configures whether or not to change the polarity of TSCK in slave transfer.

0: TSCK = SPI CK I

```
1: TSCK = !SPI_CK_I
```

(R/W)

SPI_CS_HOLD Configures whether or not to keep SPI CS low when SPI is in DONE state.

0: Not keep low

1: Keep low

(R/W)

SPI_CS_SETUP Configures whether or not to enable SPI CS when SPI is in prepare (PREP) state.

0: Disable

1: Enable

(R/W)

SPI_RSCK_I_EDGE Configures whether or not to change the polarity of RSCK in slave transfer.

```
0: RSCK = !SPI_CK_I
```

1: RSCK = SPI_CK_I

(R/W)

SPI_CK_OUT_EDGE Configures SPI clock mode together with SPI_CK_IDLE_EDGE. For more information, see Section 43.7.4.

(R/W)

Continued on the next page...

Register 43.41. SPI_USER_REG (0x0010)

Continued from the previous page...

SPI_FWRITE_DUAL Configures whether or not to enable the 2-bit mode of read-data phase in write operations.

0: Not enable

1: Enable

(R/W)

SPI_FWRITE_QUAD Configures whether or not to enable the 4-bit mode of read-data phase in write operations.

0: Not enable

1: Enable

(R/W)

SPI_SIO Configures whether or not to enable 3-line half-duplex communication, where MOSI and MISO signals share the same pin.

0: Disable

1: Enable

(R/W)

SPI_USR_MISO_HIGHPART Configures whether or not to enable High-Part mode in read-data phase, i.e., only access to high-part of the buffers: SPI_W8_REG ~ SPI_W15_REG.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_MOSI_HIGHPART Configures whether or not to enable High-Part mode in write-data phase, i.e., only access to high-part of the buffers: SPI_W8_REG ~ SPI_W15_REG.

0: Disable

1: Enable

Can be configured in CONF state.

(R/W)

SPI_USR_DUMMY_IDLE Configures whether or not to disable SPI clock in DUMMY state.

0: Not disable

1: Disable

(R/W)

SPI_USR_MOSI Configures whether or not to enable the write-data (DOUT) state of an operation.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 43.41. SPI_USER_REG (0x0010)

Continued from the previous page...

SPI_USR_MISO Configures whether or not to enable the read-data (DIN) state of an operation.

0: Disable

1: Enable

(R/W)

SPI_USR_DUMMY Configures whether or not to enable the DUMMY state of an operation.

0: Disable

1: Enable

(R/W)

SPI_USR_ADDR Configures whether or not to enable the address (ADDR) state of an operation.

0: Disable

1: Enable

(R/W)

SPI_USR_COMMAND Configures whether or not to enable the command (CMD) state of an operation.

0: Disable

1: Enable

(R/W)

Register 43.42. SPI_USER1_REG (0x0014)

SPI_USR_ADDR_BITLEN																SPI_CS_HOLD_TIME																SPI_CS_SETUP_TIME																SPI_MST_WFULL_ERR_END_EN																(reserved)																SPI_USR_DUMMY_CYCLELEN															
31																27	26																22	21																17	16	15																8	7																0										
23																0x1																0																1	0	0	0	0	0	0	0	0	0	7																Reset																					

SPI_USR_DUMMY_CYCLELEN Configures the length of DUMMY state.

Measurement unit: SPI CLK clock cycles.

This value is (the expected cycle number - 1).

(R/W)

SPI_MST_WFULL_ERR_END_EN Configures whether or not to end the SPI transfer when SPI RX

AFIFO wfull error occurs in master full-/half-duplex transfers.

0: Not end

1: End

(R/W)

SPI_CS_SETUP_TIME Configures the length of prepare (PREP) state.

Measurement unit: SPI_CLK clock cycles.

This value is equal to the expected cycles - 1. This field is used together with SPI_CS_SETUP.

(R/W)

SPI_CS_HOLD_TIME Configures the delay cycles of CS pin.

Measurement unit: SPI_CLK clock cycles.

This field is used together with SPI_CS_HOLD.

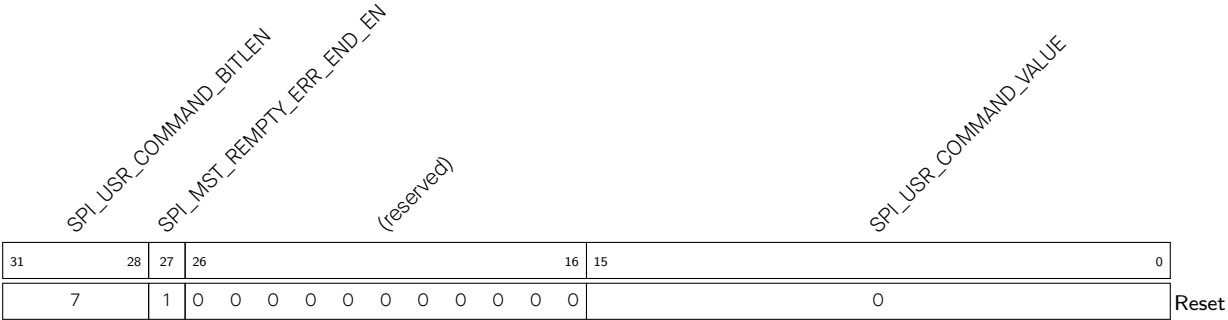
(R/W)

SPI_USR_ADDR_BITLEN Configures the bit length in address state.

This value is (expected bit number - 1).

(R/W)

Register 43.43. SPI_USER2_REG (0x0018)



SPI_USR_COMMAND_VALUE Configures the command value.
(R/W)

SPI_MST_EMPTY_ERR_END_EN Configures whether or not to end the SPI transfer when SPI TX AFIFO read empty error occurs in master full-/half-duplex transfers.
0: Not end
1: End
(R/W)

SPI_USR_COMMAND_BITLEN Configures the bit length of command state. This value is (expected bit number - 1).
(R/W)

Register 43.44. SPI_CTRL_REG (0x0008)

[illegible]

SPI_DUMMY_OUT Configures whether or not to output the SPI3 bus signals in DUMMY state.

0: Not output

1: Output

(R/W)

SPI_FADDR_DUAL Configures whether or not to enable 2-bit mode during address (ADDR) state.

0: Disable

1: Enable

(R/W)

SPI_FADDR_QUAD Configures whether or not to enable 4-bit mode during address (ADDR) state.

0: Disable

1: Enable

(R/W)

SPI_FCMD_DUAL Configures whether or not to enable 2-bit mode during command (CMD) state.

0: Disable

1: Enable

(R/W)

SPI_FCMD_QUAD Configures whether or not to enable 4-bit mode during command (CMD) state.

0: Disable

1: Enable

(R/W)

SPI_FREAD_DUAL Configures whether or not to enable the 2-bit mode of read-data (DIN) state in read operations.

0: Disable

1: Enable

(R/W)

SPI_FREAD_QUAD Configures whether or not to enable the 4-bit mode of read-data (DIN) state in read operations.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 43.44. SPI_CTRL_REG (0x0008)

Continued from the previous page...

SPI_Q_POL Configures MISO line polarity.

0: Low

1: High

(R/W)

SPI_D_POL Configures MOSI line polarity.

0: Low

1: High

(R/W)

SPI_HOLD_POL Configures SPI_HOLD output value when SPI is in idle.

0: Output low

1: Output high

(R/W)

SPI_WP_POL Configures the output value of write-protect signal when SPI is in idle.

0: Output low

1: Output high

(R/W)

SPI_RD_BIT_ORDER Configures the bit order in read-data (MISO) state.

0: MSB first

1: LSB first

2: MSB first

3: LSB first

(R/W)

SPI_WR_BIT_ORDER Configures the bit order in command (CMD), address (ADDR), and write-data (MOSI) states.

0: MSB first

1: LSB first

2: MSB first

3: LSB first

(R/W)

Register 43.45. SPI_MS_DLEN_REG (0x001C)

(reserved)														SPI_MS_DATA_BITLEN																	
3118														170																	
000000000000000														0																	Reset

SPI_MS_DATA_BITLEN Configures the data bit length of SPI transfer in CPU-controlled master transfer. Or configures the bit length of SPI RX transfer in DMA-controlled slave transfer. This value shall be (expected bit_num - 1).
(R/W)

Register 43.46. SPI_MISC_REG (0x0020)

SPI_QUAD_DIN_PIN_SWAP SPI_CS_KEEP_ACTIVE SPI_CLK_IDLE_EDGE (reserved)				SPI_SLAVE_CS_POL (reserved)				SPI_MASTER_CS_POL SPI_CLK_DIS (reserved)				SPI_CS2_DIS SPI_CS1_DIS SPI_CS0_DIS				
31	30	29	28	24	23	22	10	9	7	6	5	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

Reset

Reset

SPI_CS0_DIS Configures whether or not to disable SPI_CS0 pin.

0: SPI_CS0 signal is from/to SPI_CS0 pin.

1: Disable SPI_CS0 pin.

(R/W)

SPI_CS1_DIS Configures whether or not to disable SPI_CS1 pin.

0: SPI_CS1 signal is from/to SPI_CS1 pin.

1: Disable SPI_CS1 pin.

(R/W)

SPI_CS2_DIS Configures whether or not to disable SPI_CS2 pin.

0: SPI_CS2 signal is from/to SPI_CS2 pin.

1: Disable SPI_CS2 pin.

(R/W)

SPI_CLK_DIS Configures whether or not to disable SPI_CLK output.

0: Enable

1: Disable

(R/W)

SPI_MASTER_CS_POL Configures the polarity of SPI_CS n ($n = 0\sim 2$) line in master transfer.0: SPI_CS n is low active.1: SPI_CS n is high active.

(R/W)

SPI_SLAVE_CS_POL Configures whether or not invert SPI slave input CS polarity.

0: Not change

1: Invert

(R/W)

SPI_CLK_IDLE_EDGE Configures the level of SPI_CLK line when GP-SPI3 is in idle.

0: Low

1: High

(R/W)

SPI_CS_KEEP_ACTIVE Configures whether or not to keep the SPI_CS line low.

0: Not keep low

1: Keep low

(R/W)

Continued on the next page...

Register 43.46. SPI_MISC_REG (0x0020)

Continued from the previous page...

SPI_QUAD_DIN_PIN_SWAP Configures whether or not to swap SPI Quad input pins.

0: Not swap

1: Swap SPI3_D with SPI3_Q, and SPI3_WP with SPI3_HD

(R/W)

Register 43.47. SPI_DMA_CONF_REG (0x0030)

SPI_DMA_AFIFO_RST					(reserved)					SPI_RX_EOF_EN					(reserved)					SPI_DMA_INFIFO_FULL				
SPI_BUF_AFIFO_RST					(reserved)					SPI_SLV_TX_SEG_TRANS_CLR_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_RX_AFIFO_RST					(reserved)					SPI_SLV_RX_SEG_TRANS_CLR_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_DMA_TX_ENA					(reserved)					SPI_DMA_SLV_SEG_TRANS_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
SPI_DMA_RX_ENA					(reserved)					SPI_DMA_SLV_SEG_TRANS_EN					(reserved)					SPI_DMA_OUTFIFO_EMPTY				
31	30	29	28	27	26						22	21	20	19	18	17						2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
																							1	1

Reset

SPI_DMA_OUTFIFO_EMPTY Represents whether or not the DMA TX FIFO is ready for sending data.

0: Ready
1: Not ready
(RO)

SPI_DMA_INFIFO_FULL Represents whether or not the DMA RX FIFO is ready for receiving data.

0: Ready
1: Not ready
(RO)

SPI_DMA_SLV_SEG_TRANS_EN Configures whether or not to enable DMA-controlled segmented transfer in slave half-duplex communication.

0: Disable
1: Enable
(R/W)

SPI_SLV_RX_SEG_TRANS_CLR_EN In slave segmented transfer, if the size of the DMA RX buffer is smaller than the size of the received data,

1: the data in all the following Wr_DMA transactions will not be received
0: the data in this Wr_DMA transaction will not be received, but in the following transactions,
- if the size of DMA RX buffer is not 0, the data in following Wr_DMA transactions will be received.
- if the size of DMA RX buffer is 0, the data in following Wr_DMA transactions will not be received.
(R/W)

SPI_SLV_TX_SEG_TRANS_CLR_EN In slave segmented transfer, if the size of the DMA TX buffer is smaller than the size of the transmitted data,

1: the data in the following transactions will not be updated, i.e. the old data is transmitted repeatedly.
0: the data in this transaction will not be updated. But in the following transactions,
- if new data is filled in DMA TX FIFO, new data will be transmitted. - if no new data is filled in DMA TX FIFO, no new data will be transmitted.
(R/W)

Continued on the next page...

Register 43.47. SPI_DMA_CONF_REG (0x0030)

Continued from the previous page...

SPI_RX_EOF_EN Configures the trigger source of GDMA_IN_SUC_EOF_CH n _INT_RAW.

0: GDMA_IN_SUC_EOF_CH n _INT_RAW is set by SPI_TRANS_DONE_INT event in a single transfer, or by an SPI_DMA_SEG_TRANS_DONE_INT event in a segmented transfer.

1: In a DAM-controlled transfer, if the bit number of transferred data is equal to (SPI_MS_DATA_BITLEN + 1), then GDMA_IN_SUC_EOF_CH n _INT_RAW will be set by hardware.
(R/W)

SPI_DMA_RX_ENA Configures whether or not to enable DMA-controlled receive data transfer.

0: Disable

1: Enable

(R/W)

SPI_DMA_TX_ENA Configures whether or not to enable DMA-controlled send data transfer.

0: Disable

1: Enable

(R/W)

SPI_RX_AFIFO_RST Configures whether or not to reset RX AFIFO (i.e., spi_rx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to receive data in SPI master and slave transfer.

0: Not reset

1: Reset

(WT)

SPI_BUF_AFIFO_RST Configures whether or not to reset BUF TX AFIFO (i.e., buf_tx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to send data out in SPI slave CPU-controlled transfer and master transfer.

0: Not reset

1: Reset

(WT)

SPI_DMA_AFIFO_RST Configures whether or not to reset DMA TX AFIFO (i.e., dma_tx_afifo in Figure 43.5-3 and Figure 43.5-4), which is used to send data out in SPI slave DMA-controlled transfer.

0: Not reset

1: Reset

(WT)

Register 43.48. SPI_SLAVE_REG (0x00E0)

[illegible]

SPI_CLK_MODE Configures SPI clock mode.

- 0: SPI clock is off when CS becomes inactive.
- 1: SPI clock is delayed one cycle after CS becomes inactive.
- 2: SPI clock is delayed two cycles after CS becomes inactive.
- 3: SPI clock is always on.

(R/W)

SPI_CLK_MODE_13 Configures clock mode.

- 0: SPI clock mode 1 and mode 3. See Table 43.7-2.
1: SPI clock mode 0 and mode 2. See Table 43.7-2.

(R/W)

SPI_RSCK_DATA_OUT Configures the edge of output data.

- 0: Output data at TSCK rising edge.
1: Output data at RSCK rising edge.

(R/W)

SPI_SLV_RDDMA_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Rd_DMA transfer.

- 0: Not use
1: Use

(R/W)

SPI_SLV_WRDMA_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Wr_DMA transfer.

- 0: Not use
1: Use

(R/W)

SPI_SLV_RDBUF_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Rd_BUF transfer.

- 0: Not use
1: Use

(R/W)

Continued on the next page...

Register 43.48. SPI_SLAVE_REG (0x00E0)

Continued from the previous page...

SPI_SLV_WRBUF_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Wr_BUF transfer.

0: Not use

1: Use

(R/W)

SPI_SLV_LAST_BYTE_STRB Represents the valid bits of the last received data byte in SPI slave full-duplex and half-duplex transfer.

(R/SS)

SPI_SLAVE_MODE Configures SPI work mode.

0: Master

1: Slave

(R/W)

SPI_SOFT_RESET Configures whether to reset the SPI clock line, CS line, and data line via software.

0: Not reset

1: Reset

(WT)

SPI_MST_FD_WAIT_DMA_TX_DATA Configures whether or not to wait DMA TX data gets ready before starting SPI transfer in master full-duplex transfer.

0: Not wait

1: Wait

(R/W)

Register 43.49. SPI_SLAVE1_REG (0x00E4)

SPI_SLV_LAST_ADDR									
SPI_SLV_LAST_COMMAND									
SPI_SLV_DATA_BITLEN									
31	26	25	18	17	0				
0		0		0					
Reset									

SPI_SLV_DATA_BITLEN Configures the transferred data bit length in SPI slave full-/half-duplex modes.
(R/W/SS)

SPI_SLV_LAST_COMMAND Configures the command value in slave mode.
(R/W/SS)

SPI_SLV_LAST_ADDR Configures the address value in slave mode.
(R/W/SS)

Register 43.50. SPI_CLOCK_REG (0x000C)

SPI_CLK_EQU_SYSCLK				(reserved)				SPI_CLKDIV_PRE				SPI_CLKCNT_N				SPI_CLKCNT_H				SPI_CLKCNT_L			
31	30						22	21		18	17				12	11			6	5			0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SPI_CLKCNT_L Configures clock duty cycles, together with SPI_CLKCNT_H and SPI_CLKCNT_N.

In master transfer, this field must be equal to SPI_CLKCNT_N.

In slave mode, it must be 0.

Can be configured in CONF state.

(R/W)

SPI_CLKCNT_H Configures the duty cycle of SPI_CLK (high level) in master transfer.

It's recommended to configure this value to $\text{floor}((\text{SPI_CLKCNT_N} + 1)/2 - 1)$. floor() here is to round a number down, e.g., $\text{floor}(2.2) = 2$.

In slave mode, it must be 0.

(R/W)

SPI_CLKCNT_N Configures the divider of SPI_CLK in master transfer.

SPI_CLK frequency is $f_{\text{clk_spi_mst}}/(\text{SPI_CLKDIV_PRE} + 1)/(\text{SPI_CLKCNT_N} + 1)$.

(R/W)

SPI_CLKDIV_PRE Configures the pre-divider of SPI_CLK in master transfer.

(R/W)

SPI_CLK_EQU_SYSCLK Configures whether or not the SPI_CLK is equal to clk_spi_mst in master transfer.

0: SPI_CLK is divided from clk_spi_mst.

1: SPI_CLK is equal to clk_spi_mst.

(R/W)

Register 43.51. SPI_CLK_GATE_REG (0x00E8)

(reserved)																(reserved)				SPI_CLK_EN			
31																3	2	1	0				
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SPI_CLK_EN Configures whether or not to enable clock gate.

0: Disable

1: Enable

(R/W)

Register 43.52. SPI_DIN_MODE_REG (0x0024)

(reserved)																SPI_TIMING_HCLK_ACTIVE								(reserved)								SPI_DIN3_MODE				SPI_DIN2_MODE				SPI_DIN1_MODE				SPI_DINO_MODE							
31																17	16	15								8	7	6	5	4	3	2	1	0																	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

SPI_DINO_MODE Configures the input mode for SPI3_D signal.

0: Input without delay

1: Input at the (SPI_DINO_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DINO_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DINO_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

SPI_DIN1_MODE Configures the input mode for SPI3_Q signal.

0: Input without delay

1: Input at the (SPI_DIN1_NUM+1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

SPI_DIN2_MODE Configures the input mode for SPI3_WP signal.

0: Input without delay

1: Input at the (SPI_DIN2_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN2_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN2_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

SPI_DIN3_MODE Configures the input mode for SPI3_HD signal.

0: Input without delay

1: Input at the (SPI_DIN3_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (SPI_DIN3_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (SPI_DIN3_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

Continued on the next page...

Register 43.52. SPI_DIN_MODE_REG (0x0024)

Continued from the previous page...

SPI_TIMING_HCLK_ACTIVE Configures whether or not to enable HCLK (high-frequency clock) in SPI input timing module.

0: Disable

1: Enable

(R/W)

Register 43.53. SPI_DIN_NUM_REG (0x0028)

(reserved)																								SPI_DIN3_NUM				SPI_DIN2_NUM		SPI_DIN1_NUM		SPI_DINO_NUM																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
31																								8	7	6	5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SPI_DINO_NUM Configures the delays to input signal SPI3_D based on the setting of SPI_DINO_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

SPI_DIN1_NUM Configures the delays to input signal SPI3_Q based on the setting of SPI_DIN1_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

SPI_DIN2_NUM Configures the delays to input signal SPI3_WP based on the setting of SPI_DIN2_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

SPI_DIN3_NUM Configures the delays to input signal SPI3_HD based on the setting of SPI_DIN3_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

Register 43.54. SPI_DOUT_MODE_REG (0x002C)

(reserved)																				SPI_DOUT3_MODE SPI_DOUT2_MODE SPI_DOUT1_MODE SPI_DOUT0_MODE			
31																	4	3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

- SPI_DOUT0_MODE** Configures the output mode for SPI3_D signal.
0: Output without delay
1: Output with a delay of a SPI module clock cycle at its falling edge
(R/W)
- SPI_DOUT1_MODE** Configures the output mode for SPI3_Q signal.
0: Output without delay
1: Output with a delay of a SPI module clock cycle at its falling edge
(R/W)
- SPI_DOUT2_MODE** Configures the output mode for SPI3_WP signal.
0: Output without delay
1: Output with a delay of a SPI module clock cycle at its falling edge
(R/W)
- SPI_DOUT3_MODE** Configures the output mode for SPI3_HD signal.
0: Output without delay
1: Output with a delay of a SPI module clock cycle at its falling edge
(R/W)

Register 43.55. SPI_DMA_INT_ENA_REG (0x0034)

(reserved)												SPI_APP1_INT_ENA SPI_APP2_INT_ENA SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ENA SPI_MST_RX_AFIFO_WFULL_ERR_INT_ENA SPI_SLV_CMD_ERR_INT_ENA (reserved) SPI_DMA_SEG_TRANS_DONE_INT_ENA SPI_TRANS_DONE_INT_ENA SPI_SLV_WR_BUF_DONE_INT_ENA SPI_SLV_RD_BUF_DONE_INT_ENA SPI_SLV_WR_DMA_DONE_INT_ENA SPI_SLV_RD_DMA_DONE_INT_ENA SPI_SLV_CMD9_INT_ENA SPI_SLV_CMD8_INT_ENA SPI_SLV_CMD7_INT_ENA SPI_SLV_EX_QPI_INT_ENA SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ENA SPI_DMA_INFIFO_FULL_ERR_INT_ENA																					
31												21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset		

SPI_DMA_INFIFO_FULL_ERR_INT_ENA Write 1 to enable SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(R/W)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ENA Write 1 to enable SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(R/W)

SPI_SLV_EX_QPI_INT_ENA Write 1 to enable SPI_SLV_EX_QPI_INT interrupt.
(R/W)

SPI_SLV_EN_QPI_INT_ENA Write 1 to enable SPI_SLV_EN_QPI_INT interrupt.
(R/W)

SPI_SLV_CMD7_INT_ENA Write 1 to enable SPI_SLV_CMD7_INT interrupt.
(R/W)

SPI_SLV_CMD8_INT_ENA Write 1 to enable SPI_SLV_CMD8_INT interrupt.
(R/W)

SPI_SLV_CMD9_INT_ENA Write 1 to enable SPI_SLV_CMD9_INT interrupt.
(R/W)

SPI_SLV_CMDA_INT_ENA Write 1 to enable SPI_SLV_CMDA_INT interrupt.
(R/W)

SPI_SLV_RD_DMA_DONE_INT_ENA Write 1 to enable SPI_SLV_RD_DMA_DONE_INT interrupt.
(R/W)

SPI_SLV_WR_DMA_DONE_INT_ENA Write 1 to enable SPI_SLV_WR_DMA_DONE_INT interrupt.
(R/W)

SPI_SLV_RD_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/W)

SPI_SLV_WR_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_WR_BUF_DONE_INT interrupt.
(R/W)

Continued on the next page...

Register 43.55. SPI_DMA_INT_ENA_REG (0x0034)

Continued from the previous page...

SPI_TRANS_DONE_INT_ENA Write 1 to enable SPI_TRANS_DONE_INT interrupt.
(R/W)

SPI_DMA_SEG_TRANS_DONE_INT_ENA Write 1 to enable SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(R/W)

SPI_SLV_BUF_ADDR_ERR_INT_ENA Write 1 to enable SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(R/W)

SPI_SLV_CMD_ERR_INT_ENA Write 1 to enable SPI_SLV_CMD_ERR_INT interrupt.
(R/W)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_ENA Write 1 to enable SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(R/W)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ENA Write 1 to enable SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(R/W)

SPI_APP2_INT_ENA Write 1 to enable SPI_APP2_INT interrupt.
(R/W)

SPI_APP1_INT_ENA Write 1 to enable SPI_APP1_INT interrupt.
(R/W)

Register 43.56. SPI_DMA_INT_CLR_REG (0x0038)

(reserved)												SPI_APP1_INT_CLR SPI_APP2_INT_CLR SPI_MST_TX_INT_CLR SPI_MST_RX_AFIFO_EMPTY_ERR_INT_CLR SPI_SLV_CMD_ERR_INT_CLR SPI_SLV_BUF_ADDR_ERR_INT_CLR (reserved) SPI_DMA_SEG_TRANS_DONE_INT_CLR SPI_TRANS_DONE_INT_CLR SPI_SLV_WR_BUF_DONE_INT_CLR SPI_SLV_RD_BUF_DONE_INT_CLR SPI_SLV_WR_DMA_DONE_INT_CLR SPI_SLV_RD_DMA_DONE_INT_CLR SPI_SLV_CMD7_INT_CLR SPI_SLV_CMD8_INT_CLR SPI_SLV_CMD9_INT_CLR SPI_SLV_CMDA_INT_CLR SPI_SLV_EN_QPI_INT_CLR SPI_SLV_EX_QPI_INT_CLR SPI_DMA_OUTFIFO_EMPTY_ERR_INT_CLR SPI_DMA_INFIFO_FULL_ERR_INT_CLR																				
31											21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

SPI_DMA_INFIFO_FULL_ERR_INT_CLR Write 1 to clear SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(WT)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_CLR Write 1 to clear SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_SLV_EX_QPI_INT_CLR Write 1 to clear SPI_SLV_EX_QPI_INT interrupt.
(WT)

SPI_SLV_EN_QPI_INT_CLR Write 1 to clear SPI_SLV_EN_QPI_INT interrupt.
(WT)

SPI_SLV_CMD7_INT_CLR Write 1 to clear SPI_SLV_CMD7_INT interrupt.
(WT)

SPI_SLV_CMD8_INT_CLR Write 1 to clear SPI_SLV_CMD8_INT interrupt.
(WT)

SPI_SLV_CMD9_INT_CLR Write 1 to clear SPI_SLV_CMD9_INT interrupt.
(WT)

SPI_SLV_CMDA_INT_CLR Write 1 to clear SPI_SLV_CMDA_INT interrupt.
(WT)

SPI_SLV_RD_DMA_DONE_INT_CLR Write 1 to clear SPI_SLV_RD_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_WR_DMA_DONE_INT_CLR Write 1 to clear SPI_SLV_WR_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_RD_BUF_DONE_INT_CLR Write 1 to clear SPI_SLV_RD_BUF_DONE_INT interrupt.
(WT)

SPI_SLV_WR_BUF_DONE_INT_CLR Write 1 to clear SPI_SLV_WR_BUF_DONE_INT interrupt.
(WT)

Continued on the next page...

Register 43.56. SPI_DMA_INT_CLR_REG (0x0038)

Continued from the previous page...

SPI_TRANS_DONE_INT_CLR Write 1 to clear SPI_TRANS_DONE_INT interrupt.
(WT)

SPI_DMA_SEG_TRANS_DONE_INT_CLR Write 1 to clear SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(WT)

SPI_SLV_BUF_ADDR_ERR_INT_CLR Write 1 to clear SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

SPI_SLV_CMD_ERR_INT_CLR Write 1 to clear SPI_SLV_CMD_ERR_INT interrupt.
(WT)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_CLR Write 1 to clear SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_CLR Write 1 to clear SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_APP2_INT_CLR Write 1 to clear SPI_APP2_INT interrupt.
(WT)

SPI_APP1_INT_CLR Write 1 to clear SPI_APP1_INT interrupt.
(WT)

Register 43.57. SPI_DMA_INT_RAW_REG (0x003C)

(reserved)																																SPI_APP1_INT_RAW SPI_APP2_INT_RAW SPI_MST_TX_AFIFO_EMPTY_ERR_INT_RAW SPI_MST_RX_AFIFO_EMPTY_ERR_INT_RAW SPI_SLV_CMD_ERR_INT_RAW SPI_SLV_BUF_ADDR_ERR_INT_RAW (reserved) SPI_DMA_SEG_TRANS_DONE_INT_RAW SPI_TRANS_DONE_INT_RAW SPI_SLV_WR_BUF_DONE_INT_RAW SPI_SLV_RD_BUF_DONE_INT_RAW SPI_SLV_WR_DMA_DONE_INT_RAW SPI_SLV_RD_DMA_DONE_INT_RAW SPI_SLV_CMD9_INT_RAW SPI_SLV_CMD8_INT_RAW SPI_SLV_CMD7_INT_RAW SPI_SLV_EN_QPI_INT_RAW SPI_DMA_OUTFIFO_EMPTY_ERR_INT_RAW SPI_DMA_INFIFO_FULL_ERR_INT_RAW															
31																								21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0												0												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

SPI_DMA_INFIFO_FULL_ERR_INT_RAW The raw interrupt status of SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(R/WTC/SS)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_RAW The raw interrupt status of SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(R/WTC/SS)

SPI_SLV_EX_QPI_INT_RAW The raw interrupt status of SPI_SLV_EX_QPI_INT interrupt.
(R/WTC/SS)

SPI_SLV_EN_QPI_INT_RAW The raw interrupt status of SPI_SLV_EN_QPI_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD7_INT_RAW The raw interrupt status of SPI_SLV_CMD7_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD8_INT_RAW The raw interrupt status of SPI_SLV_CMD8_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMD9_INT_RAW The raw interrupt status of SPI_SLV_CMD9_INT interrupt.
(R/WTC/SS)

SPI_SLV_CMDA_INT_RAW The raw interrupt status of SPI_SLV_CMDA_INT interrupt.
(R/WTC/SS)

SPI_SLV_RD_DMA_DONE_INT_RAW The raw interrupt status of SPI_SLV_RD_DMA_DONE_INT interrupt.
(R/WTC/SS)

SPI_SLV_WR_DMA_DONE_INT_RAW The raw interrupt status of SPI_SLV_WR_DMA_DONE_INT interrupt.
(R/WTC/SS)

SPI_SLV_RD_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/WTC/SS)

Continued on the next page...

Register 43.57. SPI_DMA_INT_RAW_REG (0x003C)

Continued from the previous page...

SPI_SLV_WR_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.

(R/WTC/SS)

SPI_TRANS_DONE_INT_RAW The raw interrupt status of SPI_TRANS_DONE_INT interrupt.

(R/WTC/SS)

SPI_DMA_SEG_TRANS_DONE_INT_RAW The raw interrupt status of SPI_DMA_SEG_TRANS_DONE_INT interrupt.

(R/WTC/SS)

SPI_SLV_BUF_ADDR_ERR_INT_RAW The raw interrupt status of SPI_SLV_BUF_ADDR_ERR_INT interrupt.

(R/WTC/SS)

SPI_SLV_CMD_ERR_INT_RAW The raw interrupt status of SPI_SLV_CMD_ERR_INT interrupt.

(R/WTC/SS)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_RAW The raw interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.

(R/WTC/SS)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_RAW The raw interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.

(R/WTC/SS)

SPI_APP2_INT_RAW The raw interrupt status of SPI_APP2_INT interrupt. The value is only controlled by the application.

(R/WTC/SS)

SPI_APP1_INT_RAW The raw interrupt status of SPI_APP1_INT interrupt. The value is only controlled by the application.

(R/WTC/SS)

Register 43.58. SPI_DMA_INT_ST_REG (0x0040)

(reserved)												SPI_APP1_INT_ST SPI_APP2_INT_ST SPI_MST_TX_INT_ST SPI_MST_RX_AFIFO_EMPTY_ERR_INT_ST SPI_SLV_CMD_ERR_INT_ST (reserved) SPI_DMA_BUF_ADDR_ERR_INT_ST SPI_DMA_SEG_TRANS_DONE_INT_ST SPI_SLV_WR_BUF_DONE_INT_ST SPI_SLV_RD_BUF_DONE_INT_ST SPI_SLV_RD_DMA_DONE_INT_ST SPI_SLV_WR_DMA_DONE_INT_ST SPI_SLV_CMD9_INT_ST SPI_SLV_CMD8_INT_ST SPI_SLV_CMD7_INT_ST SPI_SLV_EN_QPI_INT_ST SPI_SLV_EX_QPI_INT_ST SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ST SPI_DMA_INFIFO_FULL_ERR_INT_ST																				
31											21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

SPI_DMA_INFIFO_FULL_ERR_INT_ST The interrupt status of SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(RO)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_ST The interrupt status of SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(RO)

SPI_SLV_EX_QPI_INT_ST The interrupt status of SPI_SLV_EX_QPI_INT interrupt.
(RO)

SPI_SLV_EN_QPI_INT_ST The interrupt status of SPI_SLV_EN_QPI_INT interrupt.
(RO)

SPI_SLV_CMD7_INT_ST The interrupt status of SPI_SLV_CMD7_INT interrupt.
(RO)

SPI_SLV_CMD8_INT_ST The interrupt status of SPI_SLV_CMD8_INT interrupt.
(RO)

SPI_SLV_CMD9_INT_ST The interrupt status of SPI_SLV_CMD9_INT interrupt.
(RO)

SPI_SLV_CMDA_INT_ST The interrupt status of SPI_SLV_CMDA_INT interrupt.
(RO)

SPI_SLV_RD_DMA_DONE_INT_ST The interrupt status of SPI_SLV_RD_DMA_DONE_INT interrupt.
(RO)

SPI_SLV_WR_DMA_DONE_INT_ST The interrupt status of SPI_SLV_WR_DMA_DONE_INT interrupt.
(RO)

SPI_SLV_RD_BUF_DONE_INT_ST The interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(RO)

SPI_SLV_WR_BUF_DONE_INT_ST The interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.
(RO)

Continued on the next page...

Register 43.58. SPI_DMA_INT_ST_REG (0x0040)

Continued from the previous page...

SPI_TRANS_DONE_INT_ST The interrupt status of SPI_TRANS_DONE_INT interrupt.
(RO)

SPI_DMA_SEG_TRANS_DONE_INT_ST The interrupt status of SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(RO)

SPI_SLV_BUF_ADDR_ERR_INT_ST The interrupt status of SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(RO)

SPI_SLV_CMD_ERR_INT_ST The interrupt status of SPI_SLV_CMD_ERR_INT interrupt.
(RO)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_ST The interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(RO)

SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ST The interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(RO)

SPI_APP2_INT_ST The interrupt status of SPI_APP2_INT interrupt.
(RO)

SPI_APP1_INT_ST The interrupt status of SPI_APP1_INT interrupt.
(RO)

Register 43.59. SPI_DMA_INT_SET_REG (0x0044)

(reserved)																		SPI_APP1_INT_SET SPI_APP2_INT_SET SPI_MST_TX_INT_SET SPI_MST_RX_AFIFO_EMPTY_ERR_INT_SET SPI_SLV_CMD_ERR_INT_SET SPI_SLV_BUF_ADDR_ERR_INT_SET (reserved) SPI_DMA_SEG_TRANS_DONE_INT_SET SPI_TRANS_DONE_INT_SET SPI_SLV_WR_BUF_DONE_INT_SET SPI_SLV_RD_BUF_DONE_INT_SET SPI_SLV_WR_DMA_DONE_INT_SET SPI_SLV_RD_DMA_DONE_INT_SET SPI_SLV_CMD9_INT_SET SPI_SLV_CMD8_INT_SET SPI_SLV_CMD7_INT_SET SPI_SLV_EN_QPI_INT_SET SPI_SLV_EX_QPI_INT_SET SPI_DMA_OUTFIFO_EMPTY_ERR_INT_SET SPI_DMA_INFIFO_FULL_ERR_INT_SET																	
31											21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset					

SPI_DMA_INFIFO_FULL_ERR_INT_SET Write 1 to set SPI_DMA_INFIFO_FULL_ERR_INT interrupt.
(WT)

SPI_DMA_OUTFIFO_EMPTY_ERR_INT_SET Write 1 to set SPI_DMA_OUTFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_SLV_EX_QPI_INT_SET Write 1 to set SPI_SLV_EX_QPI_INT interrupt.
(WT)

SPI_SLV_EN_QPI_INT_SET Write 1 to set SPI_SLV_EN_QPI_INT interrupt.
(WT)

SPI_SLV_CMD7_INT_SET Write 1 to set SPI_SLV_CMD7_INT interrupt.
(WT)

SPI_SLV_CMD8_INT_SET Write 1 to set SPI_SLV_CMD8_INT interrupt.
(WT)

SPI_SLV_CMD9_INT_SET Write 1 to set SPI_SLV_CMD9_INT interrupt.
(WT)

SPI_SLV_CMDA_INT_SET Write 1 to set SPI_SLV_CMDA_INT interrupt.
(WT)

SPI_SLV_RD_DMA_DONE_INT_SET Write 1 to set SPI_SLV_RD_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_WR_DMA_DONE_INT_SET Write 1 to set SPI_SLV_WR_DMA_DONE_INT interrupt.
(WT)

SPI_SLV_RD_BUF_DONE_INT_SET Write 1 to set SPI_SLV_RD_BUF_DONE_INT interrupt.
(WT)

SPI_SLV_WR_BUF_DONE_INT_SET Write 1 to set SPI_SLV_WR_BUF_DONE_INT interrupt.
(WT)

SPI_TRANS_DONE_INT_SET Write 1 to set SPI_TRANS_DONE_INT interrupt.
(WT)

Continued on the next page...

Register 43.59. SPI_DMA_INT_SET_REG (0x0044)

Continued from the previous page...

SPI_DMA_SEG_TRANS_DONE_INT_SET Write 1 to set SPI_DMA_SEG_TRANS_DONE_INT interrupt.
(WT)

SPI_SLV_BUF_ADDR_ERR_INT_SET Write 1 to set SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

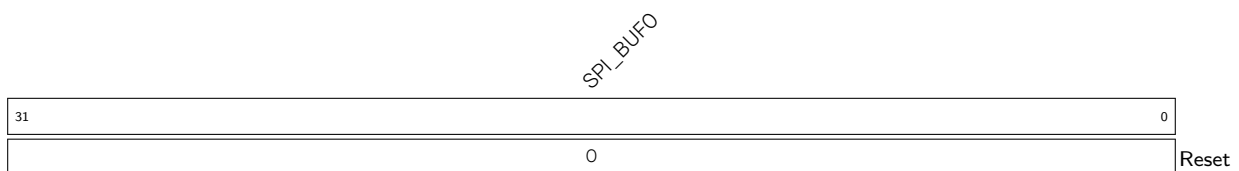
SPI_SLV_CMD_ERR_INT_SET Write 1 to set SPI_SLV_CMD_ERR_INT interrupt.
(WT)

SPI_MST_RX_AFIFO_WFULL_ERR_INT_SET Write 1 to set SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

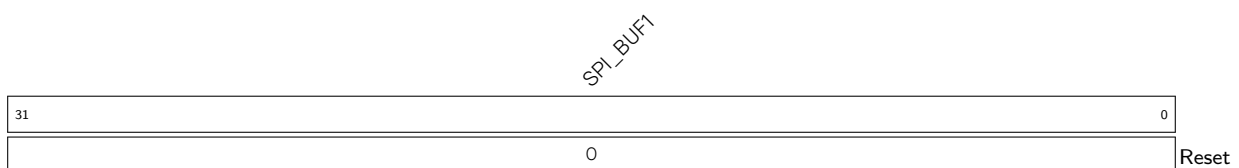
SPI_MST_TX_AFIFO_EMPTY_ERR_INT_SET Write 1 to set SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

SPI_APP2_INT_SET Write 1 to set SPI_APP2_INT interrupt.
(WT)

SPI_APP1_INT_SET Write 1 to set SPI_APP1_INT interrupt.
(WT)

Register 43.60. SPI_W0_REG (0x0098)

SPI_BUF0 32-bit data buffer 0.
(R/W/SS)

Register 43.61. SPI_W1_REG (0x009C)

SPI_BUF1 32-bit data buffer 1.
(R/W/SS)

Register 43.62. SPI_W2_REG (0x00A0)

SPI_BUF2	
31	0
0	
Reset	

SPI_BUF2 32-bit data buffer 2.
(R/W/SS)

Register 43.63. SPI_W3_REG (0x00A4)

SPI_BUF3	
31	0
0	
Reset	

SPI_BUF3 32-bit data buffer 3.
(R/W/SS)

Register 43.64. SPI_W4_REG (0x00A8)

SPI_BUF4	
31	0
0	
Reset	

SPI_BUF4 32-bit data buffer 4.
(R/W/SS)

Register 43.65. SPI_W5_REG (0x00AC)

SPI_BUF5	
31	0
0	
Reset	

SPI_BUF5 32-bit data buffer 5.
(R/W/SS)

SPI_BUF6

SPI_BUF6 32-bit data buffer 6.
(R/W/SS)

SPI_BUF7

SPI_BUF7 32-bit data buffer 7.
(R/W/SS)

SPI_BUF8

SPI_BUF8 32-bit data buffer 8.
(R/W/SS)

SPI_BUF9

SPI_BUF9 32-bit data buffer 9.
(R/W/SS)

Register 43.70. SPI_W10_REG (0x00C0)

<i>SPI_BUF10</i>	
31	0
0	
	Reset

SPI_BUF10 32-bit data buffer 10.
(R/W/SS)

Register 43.71. SPI_W11_REG (0x00C4)

<i>SPI_BUF11</i>	
31	0
0	
	Reset

SPI_BUF11 32-bit data buffer 11.
(R/W/SS)

Register 43.72. SPI_W12_REG (0x00C8)

<i>SPI_BUF12</i>	
31	0
0	
	Reset

SPI_BUF12 32-bit data buffer 12.
(R/W/SS)

Register 43.73. SPI_W13_REG (0x00CC)

<i>SPI_BUF13</i>	
31	0
0	
	Reset

SPI_BUF13 32-bit data buffer 13.
(R/W/SS)

SPI_BUF14

SPI_BUF14 32-bit data buffer 14.
(R/W/SS)

SPI_BUF15

SPI_BUF15 32-bit data buffer 15.
(R/W/SS)

(reserved)

SPI_DATE Version control register.
(R/W)

The addresses in this section are relative to LP-SPI base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

Espressif Systems

Register 43.77. LP_SPI_CMD_REG (0x0000)

(reserved)							LP_SPI_USR LP_SPI_UPDATE		(reserved)																								
31							25	24	23	22																							0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

LP_SPI_UPDATE Configures whether or not to synchronize SPI registers from APB clock domain into SPI module clock domain.

0: Not synchronize

1: Synchronize

This bit is only used in SPI master transfer.

(WT)

LP_SPI_USR Configures whether or not to enable user-defined command.

0: Not enable

1: Enable

An SPI operation will be triggered when the bit is set. This bit will be cleared once the operation is done.

(R/W/SC)

Register 43.78. LP_SPI_ADDR_REG (0x0004)

LP_SPI_USR_ADDR_VALUE																																
31																															0	
0																																Reset

LP_SPI_USR_ADDR_VALUE Configures the address to slave.

(R/W)

Register 43.79. LP_SPI_USER_REG (0x0010)

LP_SPI_USR_COMMAND		LP_SPI_USR_ADDR		LP_SPI_USR_DUMMY		LP_SPI_USR_MISO		LP_SPI_USR_MOSI		LP_SPI_USR_DUMMY_IDLE		LP_SPI_USR_MOSI_HIGHPART		LP_SPI_USR_MISO_HIGHPART		(reserved)		LP_SPI_SIO		(reserved)		LP_SPI_CK_OUT_EDGE		LP_SPI_RSCK_I_EDGE		LP_SPI_CS_SETUP		LP_SPI_CS_HOLD		LP_SPI_TSCK_I_EDGE		(reserved)		LP_SPI_DOUTDIN							
31	30	29	28	27	26	25	24	23	18								17	16	10								9	8	7	6	5	4	1								0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	Reset						

LP_SPI_DOUTDIN Configures whether or not to enable full-duplex communication.

0: Disable

1: Enable

(R/W)

LP_SPI_TSCK_I_EDGE Configures whether or not to change the polarity of TSCK in slave transfer.

0: TSCK = SPI_CK_I

1: TSCK = !SPI_CK_I

(R/W)

LP_SPI_CS_HOLD Configures whether or not to keep SPI CS low when SPI is in DONE state.

0: Not keep low

1: Keep low

(R/W)

LP_SPI_CS_SETUP Configures whether or not to enable SPI CS when SPI is in prepare (PREP) state.

0: Disable

1: Enable

(R/W)

LP_SPI_RSCK_I_EDGE Configures whether or not to change the polarity of RSCK in slave transfer.

0: RSCK = !SPI_CK_I

1: RSCK = SPI_CK_I

(R/W)

LP_SPI_CK_OUT_EDGE Configures SPI clock mode together with LP_SPI_CK_IDLE_EDGE.

For more information, see Section [43.7.4](#). (R/W)

LP_SPI_SIO Configures whether or not to enable 3-line half-duplex communication, where MOSI and MISO signals share the same pin.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 43.79. LP_SPI_USER_REG (0x0010)

Continued from the previous page...

LP_SPI_USR_MISO_HIGHPART Configures whether or not to enable High-Part mode in read-data phase, i.e., only access to high-part of the buffers: LP_SPI_W8_REG ~ LP_SPI_W15_REG.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_MOSI_HIGHPART Configures whether or not to enable High-Part mode in write-data phase, i.e., only access to high-part of the buffers: LP_SPI_W8_REG ~ LP_SPI_W15_REG.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_DUMMY_IDLE Configures whether or not to disable SPI clock in DUMMY state.

0: Not disable

1: Disable

(R/W)

LP_SPI_USR_MOSI Configures whether or not to enable the write-data (DOUT) state of an operation.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_MISO Configures whether or not to enable the read-data (DIN) state of an operation.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_DUMMY Configures whether or not to enable the DUMMY state of an operation.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_ADDR Configures whether or not to enable the address (ADDR) state of an operation.

0: Disable

1: Enable

(R/W)

LP_SPI_USR_COMMAND Configures whether or not to enable the command (CMD) state of an operation.

0: Disable

1: Enable

(R/W)

Register 43.80. LP_SPI_USER1_REG (0x0014)

LP_SPI_USR_ADDR_BITLEN																
LP_SPI_CS_HOLD_TIME																
LP_SPI_CS_SETUP_TIME																
LP_SPI_MST_WFULL_ERR_END_EN																
(reserved)																
LP_SPI_USR_DUMMY_CYCLELEN																
31	27	26	22	21	17	16	15	8	7	0						
23		0x1		0		1	0	0	0	0	0	0	0	0	7	Reset

LP_SPI_USR_DUMMY_CYCLELEN Configures the length of DUMMY state.

Measurement unit: SPI_CLK clock cycles.

This value is (the expected cycle number - 1). (R/W)

LP_SPI_MST_WFULL_ERR_END_EN Configures whether or not to end the LP-SPI transfer when SPI RX AFIFO wfull error occurs in master full-/half-duplex transfers.

0: Not end

1: End

(R/W)

LP_SPI_CS_SETUP_TIME Configures the length of prepare (PREP) state.

Measurement unit: SPI_CLK clock cycles.

This value is equal to the expected cycles - 1. This field is used together with LP_SPI_CS_SETUP.

(R/W)

LP_SPI_CS_HOLD_TIME Configures the delay cycles of CS pin.

Measurement unit: SPI_CLK clock cycles.

This field is used together with LP_SPI_CS_HOLD. (R/W)

LP_SPI_USR_ADDR_BITLEN Configures the bit length in address state.

This value is (expected bit number - 1). (R/W)

Register 43.81. LP_SPI_USER2_REG (0x0018)



- LP_SPI_USR_COMMAND_VALUE

Configures the command value. (R/W)
- LP_SPI_MST_EMPTY_ERR_END_EN

Configures whether or not to end the LP-SPI transfer when SPI TX AFIFO read empty error occurs in master full-/half-duplex transfers.

0: Not end

1: End

(R/W)
- LP_SPI_USR_COMMAND_BITLEN

Configures the bit length of command state.

This value is (expected bit number - 1).

(R/W)

Register 43.82. LP_SPI_CTRL_REG (0x0008)

(reserved)					LP_SPI_WR_BIT_ORDER LP_SPI_RD_BIT_ORDER					(reserved)					LP_SPI_D_POL LP_SPI_Q_POL					(reserved)					LP_SPI_DUMMY_OUT (reserved)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																								
31					27	26	25	24					20	19	18	17											4	3	2	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LP_SPI_DUMMY_OUT Configures whether or not to output the LP_SPI bus signals in DUMMY state.

0: Not output

1: Output

(R/W)

LP_SPI_Q_POL Configures MISO line polarity.

0: Low

1: High

(R/W)

LP_SPI_D_POL Configures MOSI line polarity.

0: Low

1: High

(R/W)

LP_SPI_RD_BIT_ORDER Configures the bit order in read-data (MISO) state.

0: MSB first

1: LSB first

(R/W)

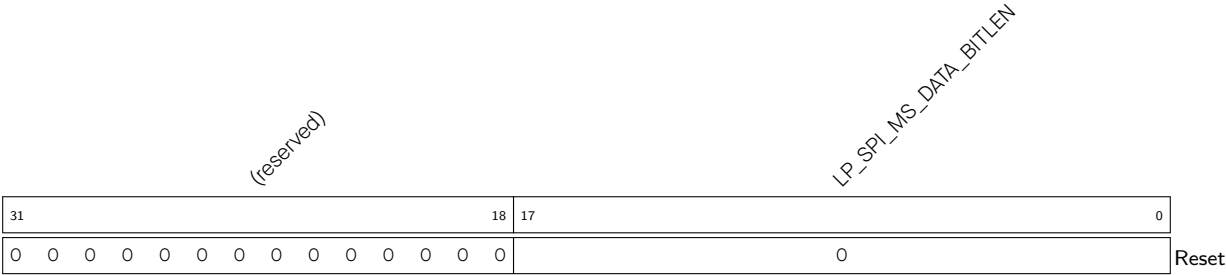
LP_SPI_WR_BIT_ORDER Configures the bit order in command (CMD), address (ADDR), and write-data (MOSI) states.

0: MSB first

1: LSB first

(R/W)

Register 43.83. LP_SPI_MS_DLEN_REG (0x001C)



LP_SPI_MS_DATA_BITLEN Configures the data bit length of SPI transfer in CPU-controlled master transfer.
This value shall be (expected bit_num - 1).
(R/W)

Register 43.84. LP_SPI_MISC_REG (0x0020)

(reserved)				LP_SPI_CS_KEEP_ACTIVE				LP_SPI_CK_IDLE_EDGE				(reserved)				LP_SPI_SLAVE_CS_POL				(reserved)				LP_SPI_MASTER_CS_POL				LP_SPI_CK_DIS				(reserved)				LP_SPI_CS0_DIS			
31	30	29	28	24				23	22												10	9	7	6	5	1				0									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset							

LP_SPI_CS0_DIS Configures whether or not to disable SPI_CS0 pin.

0: SPI_CS0 signal is from/to SPI_CS0 pin.

1: Disable SPI_CS0 pin.

(R/W)

LP_SPI_CLK_DIS Configures whether or not to disable SPI_CLK output.

0: Enable

1: Disable

(R/W)

LP_SPI_MASTER_CS_POL Configures the polarity of SPI_CS0 line in master transfer.

0: SPI_CS0 is low active.

1: SPI_CS0 is high active.

(R/W)

LP_SPI_SLAVE_CS_POL Configures whether or not invert SPI slave input CS polarity.

0: Not change

1: Invert

(R/W)

LP_SPI_CLK_IDLE_EDGE Configures the level of SPI_CLK line when LP-SPI is in idle.

0: Low

1: High

(R/W)

LP_SPI_CS_KEEP_ACTIVE Configures whether or not to keep the SPI_CS line low.

0: Not keep low

1: Keep low

(R/W)

Register 43.85. LP_SPI_DMA_CONF_REG (0x0030)

[illegible]

LP_SPI_RX_AFIFO_RST Configures to reset spi_rx_afifo as shown in Figure 43.5-3 and Figure 43.5-4.

0: Not reset

1: Reset

spi_rx_afifo is used to receive data in SPI master and slave transfer.

(WT)

LP_SPI_BUF_AFIFO_RST Configures to reset buf_tx_afifo as shown in Figure 43.5-3 and Figure 43.5-4.

0: Not reset

1: Reset

buf_tx_afifo is used to send data out in CPU-controlled master and slave transfer.

(WT)

Register 43.86. LP_SPI_SLEEP_CONFO_REG (0x0044)

(reserved)																LP_SPI_SLEEP_WK_DATA_SEL				LP_SPI_SLEEP_DIS_RXFIFO_WR_EN				LP_SPI_SLEEP_EN				LP_SPI_SLV_WK_MODE_SEL				LP_SPI_SLV_WK_CHAR_MASK				LP_SPI_SLV_WK_CHAR_NUM				LP_SPI_SLV_WK_CHARO			
31																20				19	18	17	16	15	11				10	8	7	0											
0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0	0	0	0x0				0x0				0xa				Reset											

LP_SPI_SLV_WK_CHARO Configures wake-up character 0.
(R/W)

LP_SPI_SLV_WK_CHAR_NUM Configures the amount of the enabled wake-up characters.
(R/W)

LP_SPI_SLV_WK_CHAR_MASK Configures the bit map for the characters to be masked.
Bit 0 ~ bit 4 correspond to wake-up character 0 ~ character 4, respectively.
If the bit value is:

- 0: The corresponding wake-up character is not masked.
- 1: The corresponding wake-up character is masked.

(R/W)

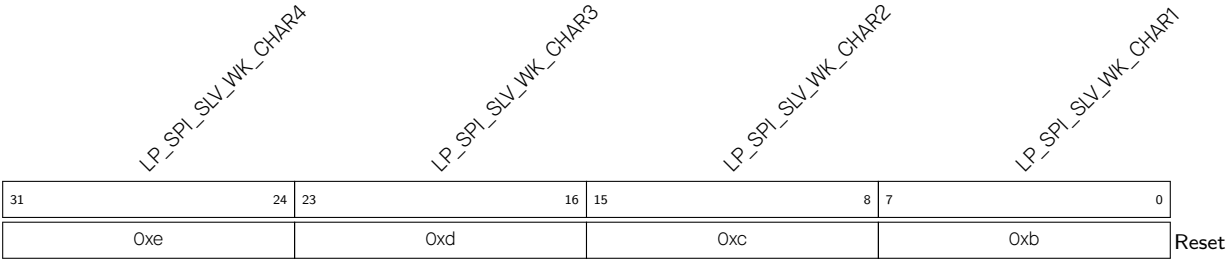
LP_SPI_SLV_WK_MODE_SEL Configures the wake-up mode.
0: Wakes up the chip once the LP-SPI slave detects a start bit.
1: Wakes up the chip once the LP-SPI slave detects a specific sequence of characters.
(R/W)

LP_SPI_SLEEP_EN Configures whether or not to enable sleep mode.
0: Not enable
1: Enable
(R/W)

LP_SPI_SLEEP_DIS_RXFIFO_WR_EN Configures whether or not to store the data before wake-up.
0: Store the data
1: Not store the data
(R/W)

LP_SPI_SLEEP_WK_DATA_SEL Configures in which states to detect the wake-up characters.
0: Only detect wake-up characters from RX data in DATA state.
1: Detect wake-up characters from RX data in all states.
(R/W)

Register 43.87. LP_SPI_SLEEP_CONF1_REG (0x0048)



LP_SPI_SLV_WK_CHAR1 Configures wake-up character 1.
(R/W)

LP_SPI_SLV_WK_CHAR2 Configures wake-up character 2.
(R/W)

LP_SPI_SLV_WK_CHAR3 Configures wake-up character 3.
(R/W)

LP_SPI_SLV_WK_CHAR4 Configures wake-up character 4.
(R/W)

Register 43.88. LP_SPI_SLAVE_REG (0x00E0)

(reserved)				LP_SPI_SOFT_RESET LP_SPI_SLAVE_MODE								(reserved)				LP_SPI_SLV_WRBUF_BITLEN_EN LP_SPI_SLV_RDBUF_BITLEN_EN				(reserved)				LP_SPI_RSCK_DATA_OUT LP_SPI_CLK_MODE_13 LP_SPI_CLK_MODE			
31	28	27	26	25								12	11	10	9		4	3	2	1	0						
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset				

LP_SPI_CLK_MODE Configures LP-SPI clock mode.

- 0: LP-SPI clock is off when CS becomes inactive.
- 1: LP-SPI clock is delayed one cycle after CS becomes inactive.
- 2: LP-SPI clock is delayed two cycles after CS becomes inactive.
- 3: LP-SPI clock is always on.

(R/W)

LP_SPI_CLK_MODE_13 Configures clock mode.

- 0: SPI clock mode 1 and mode 3. See Table [43.7-2](#).
- 1: SPI clock mode 0 and mode 2. See Table [43.7-2](#).

(R/W)

LP_SPI_RSCK_DATA_OUT Configures the edge of output data.

- 0: Output data at TSCK rising edge.
- 1: Output data at RSCK rising edge.

(R/W)

LP_SPI_SLV_RDBUF_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Rd_BUF transfer.

- 0: Not use
- 1: Use

(R/W)

LP_SPI_SLV_WRBUF_BITLEN_EN Configures whether or not to use SPI_SLV_DATA_BITLEN to store the data bit length of Wr_BUF transfer.

- 0: Not use
- 1: Use

(R/W)

LP_SPI_SLAVE_MODE Configures SPI work mode.

- 0: Master
- 1: Slave

(R/W)

LP_SPI_SOFT_RESET Configures whether to reset the SPI clock line, CS line, and data line via software.

- 0: Not reset
- 1: Reset

(WT)

Register 43.89. LP_SPI_SLAVE1_REG (0x00E4)

LP_SPI_SLV_LAST_ADDR										
LP_SPI_SLV_LAST_COMMAND										
LP_SPI_SLV_DATA_BITLEN										
31	26	25	18	17	0					
0		0		0						Reset

Register 43.90. LP_SPI_CLOCK_REG (0x000C)

[illegible]

LP_SPI_CLKCNT_L Configures clock duty cycles, together with LP_SPI_CLKCNT_H and LP_SPI_CLKCNT_N.

In master transfer, this field must be equal to LP_SPI_CLKCNT_N.

In slave mode, it must be 0.

(R/W)

LP_SPI_CLKCNT_H Configures the duty cycle of LP_SPI_CLK (high level) in master transfer.

It's recommended to configure this value to $\text{floor}((\text{LP_SPI_CLKCNT_N} + 1)/2 - 1)$. `floor()` here is to round a number down, e.g., $\text{floor}(2.2) = 2$.

In slave mode, it must be 0.

(R/W)

LP_SPI_CLKCNT_N Configures the divider of LP_SPI_CLK in master transfer.

LP_SPI_CLK frequency is $f_{lp_spi_pclk} / (LP_SPI_CLKDIV_PRE + 1) / (LP_SPI_CLKCNT_N + 1)$.

(R/W)

LP_SPI_CLKDIV_PRE Configures the pre-divider of LP_SPI_CLK in master transfer.

(R/W)

LP_SPI_CLK_EQU_SYSCLK Configures whether or not the LP_SPI_CLK is equal to lp_spi_pclk in master transfer.

0: LP_SPI_CLK is divided from lp_spi_pclk.

1: LP_SPI_CLK is equal to lp_spi_pclk.

(R/W)

Register 43.91. LP_SPI_CLK_GATE_REG (0x00E8)

[illegible]

LP_SPI_CLK_EN Configures whether or not to enable clock gate.

0: Disable

1: Enable

(R/W)

Register 43.92. LP_SPI_DIN_MODE_REG (0x0024)

[illegible]

LP_SPI_DINO_MODE Configures the input mode for LP_SPI_D signal.

0: Input without delay

1: Input at the (LP_SPI_DINO_NUM + 1)th falling edge of clk_spi_mst

2: Input at the (LP_SPI_DIN0_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (LP_SPI_DINO_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

LP_SPI_DIN1_MODE Configures the input mode for LP_SPI_Q signal.

0: Input without delay

1: Input at the (LP_SPI_DIN1_NUM+1)th falling edge of clk_spi_mst

2: Input at the (LP_SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst rising edge cycle

3: Input at the (LP_SPI_DIN1_NUM + 1)th rising edge of clk_hclk plus one clk_spi_mst falling edge cycle

(R/W)

LP_SPI_TIMING_HCLK_ACTIVE Configures whether or not to enable HCLK (high-frequency clock) in LP-SPI input timing module.

0: Disable

1: Enable

(R/W)

Register 43.93. LP_SPI_DIN_NUM_REG (0x0028)

(reserved)																								(reserved)				(reserved)		LP_SPI_DIN1_NUM		LP_SPI_DINO_NUM																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
31																								8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LP_SPI_DINO_NUM Configures the delays to input signal LP_SPI_D based on the setting of LP_SPI_DINO_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

LP_SPI_DIN1_NUM Configures the delays to input signal LP_SPI_Q based on the setting of LP_SPI_DIN1_MODE.

- 0: Delayed by 1 clock cycle
 - 1: Delayed by 2 clock cycles
 - 2: Delayed by 3 clock cycles
 - 3: Delayed by 4 clock cycles
- (R/W)

Register 43.94. LP_SPI_DOUT_MODE_REG (0x002C)

(reserved)																												(reserved)		(reserved)		LP_SPI_DOUT1_MODE		LP_SPI_DOUT0_MODE																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
31																												4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LP_SPI_DOUT0_MODE Configures the output mode for LP_SPI_D signal.

- 0: Output without delay
 - 1: Output with a delay of an LP-SPI module clock cycle at its falling edge
- (R/W)

LP_SPI_DOUT1_MODE Configures the output mode for LP_SPI_Q signal.

- 0: Output without delay
 - 1: Output with a delay of an LP-SPI module clock cycle at its falling edge
- (R/W)

Register 43.95. LP_SPI_DMA_INT_ENA_REG (0x0034)

[illegible]

LP_SPI_SLV_RD_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/W)

LP_SPI_SLV_WR_BUF_DONE_INT_ENA Write 1 to enable SPI_SLV_WR_BUF_DONE_INT interrupt.
(R/W)

LP_SPI_TRANS_DONE_INT_ENA Write 1 to enable SPI_TRANS_DONE_INT interrupt.
(R/W)

LP_SPI_WAKEUP_INT_ENA Write 1 to enable LP_SPI_WAKEUP_INT interrupt.
(R/W)

LP_SPI_SLV_BUF_ADDR_ERR_INT_ENA Write 1 to enable SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(R/W)

LP_SPI_SLV_CMD_ERR_INT_ENA Write 1 to enable SPI_SLV_CMD_ERR_INT interrupt.
(R/W)

LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_ENA Write 1 to enable SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(R/W)

LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ENA Write 1 to enable SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(R/W)

LP_SPI_APP2_INT_ENA Write 1 to enable SPI_APP2_INT interrupt.
(R/W)

LP_SPI_APP1_INT_ENA Write 1 to enable SPI_APP1_INT interrupt.
(R/W)

Register 43.96. LP_SPI_DMA_INT_CLR_REG (0x0038)

(reserved)												LP_SPI_APP1_INT_CLR LP_SPI_APP2_INT_CLR LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_CLR LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_CLR LP_SPI_SLV_BUF_ADDR_ERR_INT_CLR LP_SPI_SLV_CMD_ERR_INT_CLR LP_SPI_WAKEUP_INT_CLR (reserved) LP_SPI_TRANS_DONE_INT_CLR LP_SPI_SLV_WR_BUF_DONE_INT_CLR LP_SPI_SLV_RD_BUF_DONE_INT_CLR												(reserved)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																								
31													21	20	19	18	17	16	15	14	13	12	11	10	9													0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LP_SPI_SLV_RD_BUF_DONE_INT_CLR Write 1 to clear SPI_SLV_RD_BUF_DONE_INT interrupt.
(WT)

LP_SPI_SLV_WR_BUF_DONE_INT_CLR Write 1 to clear SPI_SLV_WR_BUF_DONE_INT interrupt.
(WT)

LP_SPI_TRANS_DONE_INT_CLR Write 1 to clear SPI_TRANS_DONE_INT interrupt.
(WT)

LP_SPI_WAKEUP_INT_CLR Write 1 to clear LP_SPI_WAKEUP_INT interrupt.
(WT)

LP_SPI_SLV_BUF_ADDR_ERR_INT_CLR Write 1 to clear SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

LP_SPI_SLV_CMD_ERR_INT_CLR Write 1 to clear SPI_SLV_CMD_ERR_INT interrupt.
(WT)

LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_CLR Write 1 to clear SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_CLR Write 1 to clear SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

LP_SPI_APP2_INT_CLR Write 1 to clear SPI_APP2_INT interrupt.
(WT)

LP_SPI_APP1_INT_CLR Write 1 to clear SPI_APP1_INT interrupt.
(WT)

Register 43.97. LP_SPI_DMA_INT_RAW_REG (0x003C)

(reserved)												LP_SPI_APP1_INT_RAW LP_SPI_APP2_INT_RAW LP_SPI_MST_TX_INT_RAW LP_SPI_MST_RX_AFIFO_EMPTY_ERR_INT_RAW LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_RAW LP_SPI_SLV_CMD_ERR_INT_RAW LP_SPI_SLV_BUF_ADDR_ERR_INT_RAW (reserved) LP_SPI_TRANS_DONE_INT_RAW LP_SPI_SLV_WR_BUF_DONE_INT_RAW LP_SPI_SLV_RD_BUF_DONE_INT_RAW												(reserved)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
31											21	20	19	18	17	16	15	14	13	12	11	10	9											0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

LP_SPI_SLV_RD_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(R/WTC/SS)

LP_SPI_SLV_WR_BUF_DONE_INT_RAW The raw interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.
(R/WTC/SS)

LP_SPI_TRANS_DONE_INT_RAW The raw interrupt status of SPI_TRANS_DONE_INT interrupt.
(R/WTC/SS)

LP_SPI_WAKEUP_INT_RAW The raw interrupt status of LP_SPI_LP_SPI_WAKEUP_INT interrupt.
(R/WTC/SS)

LP_SPI_SLV_BUF_ADDR_ERR_INT_RAW The raw interrupt status of SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(R/WTC/SS)

LP_SPI_SLV_CMD_ERR_INT_RAW The raw interrupt status of SPI_SLV_CMD_ERR_INT interrupt.
(R/WTC/SS)

LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_RAW The raw interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(R/WTC/SS)

LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_RAW The raw interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(R/WTC/SS)

LP_SPI_APP2_INT_RAW The raw interrupt status of SPI_APP2_INT interrupt.
The value is only controlled by the application.
(R/WTC/SS)

LP_SPI_APP1_INT_RAW The raw interrupt status of SPI_APP1_INT interrupt.
The value is only controlled by the application.
(R/WTC/SS)

Register 43.98. LP_SPI_DMA_INT_ST_REG (0x0040)

[illegible]

LP_SPI_SLV_RD_BUF_DONE_INT_ST	The interrupt status of SPI_SLV_RD_BUF_DONE_INT interrupt.
(RO)	

LP_SPI_SLV_WR_BUF_DONE_INT_ST The interrupt status of SPI_SLV_WR_BUF_DONE_INT interrupt.
(RO)

LP_SPI_TRANS_DONE_INT_ST The interrupt status of SPI_TRANS_DONE_INT interrupt.
(RO)

LP_SPI_WAKEUP_INT_ST The interrupt status bit of LP_SPI_WAKEUP_INT interrupt.
(RO)

LP_SPI_SLV_BUF_ADDR_ERR_INT_ST The interrupt status bit of SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(RO)

LP_SPI_SLV_CMD_ERR_INT_ST The interrupt status of SPI_SLV_CMD_ERR_INT interrupt.
(RO)

LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_ST The interrupt status of SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(RO)

LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_ST The interrupt status of SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(RO)

LP_SPI_APP2_INT_ST The interrupt status of SPI_APP2_INT interrupt.
(RO)

LP_SPI_APP1_INT_ST The interrupt status of SPI_APP1_INT interrupt.
(RO)

Register 43.99. LP_SPI_DMA_INT_SET_REG (0x004C)

(reserved)										LP_SPI_APP1_INT_SET LP_SPI_APP2_INT_SET LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_SET LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_SET LP_SPI_SLV_CMD_ERR_INT_SET LP_SPI_SLV_BUF_ADDR_ERR_INT_SET (reserved) LP_SPI_TRANS_DONE_INT_SET LP_SPI_SLV_WR_BUF_DONE_INT_SET LP_SPI_SLV_RD_BUF_DONE_INT_SET										(reserved)														
31											21	20	19	18	17	16	15	14	13	12	11	10	9											0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

LP_SPI_SLV_RD_BUF_DONE_INT_SET Write 1 to set SPI_SLV_RD_BUF_DONE_INT interrupt.
(WT)

LP_SPI_SLV_WR_BUF_DONE_INT_SET Write 1 to set SPI_SLV_WR_BUF_DONE_INT interrupt.
(WT)

LP_SPI_TRANS_DONE_INT_SET Write 1 to set SPI_TRANS_DONE_INT interrupt.
(WT)

LP_SPI_SLV_BUF_ADDR_ERR_INT_SET Write 1 to set SPI_SLV_BUF_ADDR_ERR_INT interrupt.
(WT)

LP_SPI_SLV_CMD_ERR_INT_SET Write 1 to set SPI_SLV_CMD_ERR_INT interrupt.
(WT)

LP_SPI_MST_RX_AFIFO_WFULL_ERR_INT_SET Write 1 to set SPI_MST_RX_AFIFO_WFULL_ERR_INT interrupt.
(WT)

LP_SPI_MST_TX_AFIFO_EMPTY_ERR_INT_SET Write 1 to set SPI_MST_TX_AFIFO_EMPTY_ERR_INT interrupt.
(WT)

LP_SPI_APP2_INT_SET Write 1 to set SPI_APP2_INT interrupt.
(WT)

LP_SPI_APP1_INT_SET Write 1 to set SPI_APP1_INT interrupt.
(WT)

Register 43.100. LP_SPI_WO_REG (0x0098)

<i>LP_SPI_BUF0</i>	
31	0
0	
Reset	

LP_SPI_BUF0 32-bit data buffer 0.
(R/W/SS)

Register 43.101. LP_SPI_W1_REG (0x009C)

<i>LP_SPI_BUF1</i>	
31	0
0	
Reset	

LP_SPI_BUF1 32-bit data buffer 1.
(R/W/SS)

Register 43.102. LP_SPI_W2_REG (0x00A0)

<i>LP_SPI_BUF2</i>	
31	0
0	
Reset	

LP_SPI_BUF2 32-bit data buffer 2.
(R/W/SS)

Register 43.103. LP_SPI_W3_REG (0x00A4)

<i>LP_SPI_BUF3</i>	
31	0
0	
Reset	

LP_SPI_BUF3 32-bit data buffer 3.
(R/W/SS)

Register 43.104. LP_SPI_W4_REG (0x00A8)

<i>LP_SPI_BUF4</i>	
31	0
0	
Reset	

LP_SPI_BUF4 32-bit data buffer 4.
(R/W/SS)

Register 43.105. LP_SPI_W5_REG (0x00AC)

<i>LP_SPI_BUF5</i>	
31	0
0	
Reset	

LP_SPI_BUF5 32-bit data buffer 5.
(R/W/SS)

Register 43.106. LP_SPI_W6_REG (0x00B0)

<i>LP_SPI_BUF6</i>	
31	0
0	
Reset	

LP_SPI_BUF6 32-bit data buffer 6.
(R/W/SS)

Register 43.107. LP_SPI_W7_REG (0x00B4)

<i>LP_SPI_BUF7</i>	
31	0
0	
Reset	

LP_SPI_BUF7 32-bit data buffer 7.
(R/W/SS)

Register 43.108. LP_SPI_W8_REG (0x00B8)

LP_SPI_BUF8	
31	0
0	
Reset	

LP_SPI_BUF8 32-bit data buffer 8.
(R/W/SS)

Register 43.109. LP_SPI_W9_REG (0x00BC)

LP_SPI_BUF9	
31	0
0	
Reset	

LP_SPI_BUF9 32-bit data buffer 9.
(R/W/SS)

Register 43.110. LP_SPI_W10_REG (0x00C0)

LP_SPI_BUF10	
31	0
0	
Reset	

LP_SPI_BUF10 32-bit data buffer 10.
(R/W/SS)

Register 43.111. LP_SPI_W11_REG (0x00C4)

LP_SPI_BUF11	
31	0
0	
Reset	

LP_SPI_BUF11 32-bit data buffer 11.
(R/W/SS)

Register 43.112. LP_SPI_W12_REG (0x00C8)

<i>LP_SPI_BUF12</i>	
31	0
0	
Reset	

LP_SPI_BUF12 32-bit data buffer 12.
(R/W/SS)

Register 43.113. LP_SPI_W13_REG (0x00CC)

<i>LP_SPI_BUF13</i>	
31	0
0	
Reset	

LP_SPI_BUF13 32-bit data buffer 13.
(R/W/SS)

Register 43.114. LP_SPI_W14_REG (0x00D0)

<i>LP_SPI_BUF14</i>	
31	0
0	
Reset	

LP_SPI_BUF14 32-bit data buffer 14.
(R/W/SS)

Register 43.115. LP_SPI_W15_REG (0x00D4)

<i>LP_SPI_BUF15</i>	
31	0
0	
Reset	

LP_SPI_BUF15 32-bit data buffer 15.
(R/W/SS)

Register 43.116. LP_SPI_DATE_REG (0x00F0)

(reserved)				LP_SPI_DATE																										
31	28	27	0																											
0	0	0	0	0x2009040																										Reset

LP_SPI_DATE Version control register.
(R/W)